

Building Mini-Cordis

A Laboratory Handbook for Reversible Effects, Reactive Coeffects, and Dynamic Composition

A paper companion to *A Programming Paradigm for Spatiotemporal Composability*

August 2026

Contents

Preface	i
What you will build	ii
Audience and prerequisites	iii
How the labs are organised	iv
Milestones	iv
Suggested schedule	iv
Assessment model	v
Development Environment	vi
Recommended project layout	vi
Conventions used in the code	vii
0 Lab 0: Reading Programs as Mathematics	1
0.1 Purpose and outcomes	1
0.2 Motivation: prose is not enough	1
0.3 Types, sets, and state spaces	2
0.3.1 Worked example 0.1: enumerating a finite state space	2
0.4 Functions and composition	2
0.4.1 Worked example 0.2: order matters	3
0.5 Identity and monoids	3
0.5.1 Worked example 0.3: arrays under concatenation	4
0.6 Partial functions	4
0.6.1 Counterexample 0.1: pretending a partial function is total	5
0.7 Relations and equivalence	5
0.7.1 Worked example 0.4: generated identifiers	5
0.8 Directed graphs and dependency order	6
0.9 Transition systems	6
0.9.1 Worked example 0.5: invalid transition	7
0.10 Executable theory	7
0.11 Exercises	8
0.11.1 Core exercises	8
0.11.2 Explanation exercises	8
0.11.3 Counterexample workshop	8
0.12 Deliverable	8
0.13 Reading guide	9
0.13.1 Required	9
0.13.2 Bridge reading	9
0.13.3 Read with this question	9

1	Lab 1: Reversible Effects as State-Dependent Inverses	10
1.1	Purpose and outcomes	10
1.2	Motivation: cleanup separated from setup	10
1.3	From impure functions to explicit context transformations	11
1.4	What kind of inverse do we need?	12
1.4.1	Worked example 1.1: setting a value	13
1.5	The effect-function type	14
1.5.1	Worked example 1.2: command registration with a precondition	15
1.6	Sequential composition	16
1.6.1	Worked example 1.3: two assignments	17
1.7	Twisted composition	17
1.7.1	Worked example 1.4: deriving the inverse order symbolically	18
1.8	The effect monoid	18
1.8.1	Proof sketch: witnessing survives composition	19
1.9	API design: atomic operations should return inverses	19
1.9.1	Worked example 1.5: event subscription	19
1.10	Executable recovery laws	20
1.10.1	Example-based law	20
1.10.2	Property-based law	20
1.10.3	Composition property	21
1.11	Counterexample workshop: a plausible but wrong inverse	21
1.11.1	Counterexample 1.2: logging	22
1.11.2	Acquisition versus emission	22
1.12	Exercises	22
1.12.1	Core implementation	22
1.12.2	Mathematical exercises	23
1.12.3	Property-testing exercises	23
1.12.4	Counterexample exercises	23
1.13	Deliverable	23
1.14	Reading guide	24
1.14.1	Required	24
1.14.2	Applied comparison	24
1.14.3	Deepening	24
1.14.4	Read with these questions	24
2	Lab 2: The Effect Context and Automatic Recovery	25
2.1	Purpose and outcomes	25
2.2	Motivation: helper functions should not leak cleanup obligations	25
2.3	The effect context	26
2.4	Tracking one effect	27
2.4.1	Worked example 2.1: tracking two effects	28
2.5	Recovery	28
2.5.1	Worked example 2.2: exact recovery	28
2.6	The soundness invariant	29
2.6.1	Why each tracked effect preserves the invariant	29
2.7	Lifting state-dependent effect functions	29
2.7.1	Worked example 2.3: activation with nested helpers	30
2.8	An imperative Scope facade	30

2.8.1	API signatures for the first useful runtime	31
2.9	Real resource wrappers	32
2.9.1	Command registry	32
2.9.2	Timer	32
2.9.3	Event listener	32
2.10	Nested scopes and hierarchical composition	33
2.10.1	Worked example 2.4: parent and child order	34
2.11	Total recovery versus selective withdrawal	34
2.12	Failure during setup	35
2.13	Executable invariants	35
2.13.1	Recovery test	35
2.13.2	LIFO trace test	36
2.13.3	At-most-once disposal	36
2.13.4	Intermediate soundness	36
2.14	Counterexample workshop	36
2.14.1	Counterexample 2.1: appending inverses in the wrong order	36
2.14.2	Counterexample 2.2: bypassing the context	37
2.14.3	Counterexample 2.3: disposal after an external overwrite	37
2.15	Exercises	37
2.15.1	Core implementation	37
2.15.2	Mathematical exercises	37
2.15.3	Design exercises	38
2.15.4	Counterexample exercises	38
2.16	Milestone A: reversible computation	38
2.17	Reading guide	38
2.17.1	Required	38
2.17.2	Applied comparison	39
2.17.3	Deepening	39
2.17.4	Read with these questions	39
3	Lab 3: Reactive Coeffects and a Typed Service Context	40
3.1	Purpose and outcomes	40
3.2	Motivation: optimistic lookup is not dependency management	40
3.3	Effects and coeffects	41
3.4	The coeffect context as a partial dependent map	41
3.4.1	A TypeScript approximation	42
3.4.2	Worked example 3.1: typed lookup	43
3.5	Requirements and satisfaction	43
3.5.1	Worked example 3.2: a two-key requirement	44
3.6	Reactive notification	44
3.6.1	Worked example 3.3: one change, different classifications	45
3.7	Provision is itself a revertible effect	45
3.8	A first reactive runtime	46
3.9	Worked system: database, repository, and API	47
3.9.1	Components	48
3.9.2	Insertion trace	49
3.9.3	Withdrawal trace	49
3.10	Requirements should authorise access	49

3.11 Provider uniqueness and collisions	50
3.12 Dependency cycles	51
3.12.1 Worked decomposition	51
3.13 Executable theory	51
3.13.1 Satisfaction test	51
3.13.2 Classification table test	52
3.13.3 Reactivity invariant	52
3.13.4 Provision recovery	52
3.14 Counterexample workshop	52
3.14.1 Counterexample 3.1: hidden dependency	53
3.14.2 Counterexample 3.2: notification only on addition	53
3.14.3 Counterexample 3.3: comparing only satisfaction	53
3.14.4 Counterexample 3.4: string-key collision	53
3.15 Exercises	53
3.15.1 Core implementation	53
3.15.2 Worked-system exercises	54
3.15.3 Mathematical exercises	54
3.15.4 Counterexample exercises	54
3.16 Deliverable	54
3.17 Reading guide	54
3.17.1 Required	54
3.17.2 Applied comparison	55
3.17.3 Deepening	55
3.17.4 Read with these questions	55
Optional Studio 3A: Isolation, Interception, and Derived Contexts	56
Purpose and outcomes	56
3A.1 Motivation: one logical key, several local meanings	56
3A.2 In-place and derived realisation	56
3A.3 Isolation realms	57
TypeScript representation	57
Worked example 3A.1: two tenant databases	58
Recovery	58
3A.4 Interception metadata	59
Provider shape	59
Example metadata	59
3A.5 Component and context metadata	60
Worked example 3A.2: read-only database context	60
3A.6 Isolation versus interception	61
3A.7 Security boundary	61
3A.8 Executable laws	61
Realm isolation	62
Parent preservation	62
Metadata monoid	62
3A.9 Counterexample workshop	62
3A.10 Exercises and deliverable	62
Reading	63

4	Lab 4: Components, Fibers, and Operational Semantics	64
4.1	Purpose and outcomes	64
4.2	Motivation: availability is not identity	64
4.3	Component descriptions	65
4.3.1	Worked example 4.1: a command component	65
4.4	Fibers: live component instances	66
4.4.1	Component versus fiber	66
4.5	The registry and provider relation	67
4.6	Provider views	67
4.6.1	Worked example 4.2: replacement with equal values	68
4.7	Comparing views	69
4.8	The base lifecycle	69
4.9	Operational semantics: rules as executable definitions	70
4.9.1	The reload rule	70
4.9.2	The unload rule	71
4.10	Translating rules into code	71
4.11	Orchestration versus lifecycle steps	72
4.11.1	Why retirement and removal are separate	73
4.11.2	Worked example 4.3: safe retirement	73
4.12	Quiescence	73
4.13	Confinement and attribution	74
4.13.1	Reading through the committed view	74
4.14	Worked trace: provider replacement	75
4.15	Executable invariants	75
4.15.1	Unique provider	75
4.15.2	Active fibers have total committed views	75
4.15.3	Committed providers exist	76
4.15.4	Quiescence after settling	76
4.16	Counterexample workshop	76
4.16.1	Counterexample 4.1: compare dependency values instead of providers	76
4.16.2	Counterexample 4.2: remove before unload	76
4.16.3	Counterexample 4.3: read globally rather than through committed view	76
4.16.4	Counterexample 4.4: allow duplicate providers	76
4.17	Exercises	76
4.17.1	Core implementation	76
4.17.2	Operational-semantics exercises	77
4.17.3	Scenario exercises	77
4.17.4	Counterexample exercises	77
4.18	Milestone B: dynamic composition	77
4.19	Reading guide	78
4.19.1	Required	78
4.19.2	Programming-languages foundations	78
4.19.3	Applied comparison	78
4.19.4	Read with these questions	78
5	Lab 5: Independence, Commutation, and Observational Equivalence	79
5.1	Purpose and outcomes	79
5.2	Motivation: one stack per component is not enough	79

5.3	Commutation	80
5.3.1	Worked example 5.1: disjoint keys commute	80
5.3.2	Worked example 5.2: increments on one counter commute	81
5.3.3	Counterexample 5.1: set and increment	81
5.4	The transformations generated by an effect	81
5.4.1	Why inverses must be included	82
5.4.2	Worked example 5.3: forward maps commute, inverses do not preserve contributions	82
5.5	Independence of effects	82
5.5.1	Why clause 2 is needed	82
5.6	Safe selective withdrawal	83
5.6.1	Worked example 5.4: command registrations	83
5.6.2	Counterexample 5.2: same command key	84
5.7	Operations at distinct service keys	84
5.7.1	Worked example 5.5: a registration service	84
5.7.2	Counterexample 5.3: ordered middleware	84
5.8	Building a commutation matrix	85
5.9	Testing commutation	85
5.9.1	Property test over disjoint keys	86
5.9.2	Shrinking a failure	87
5.10	Why literal equality is too strong	87
5.11	Observers and observational equivalence	87
5.11.1	Finite executable approximation	88
5.12	An equivalence must be compatible with operations	88
5.12.1	Counterexample 5.4: an equivalence that ignores command handlers	89
5.12.2	Worked example 5.6: map order	89
5.13	Observational equivalence per service key	89
5.13.1	Worked example 5.7: a set-valued registry	89
5.14	Independence up to observation	89
5.14.1	Worked example 5.8: allocation up to renaming	90
5.15	A whole-fiber recovery experiment	90
5.16	Trace equivalence	91
5.17	Counterexample workshop	91
5.17.1	Counterexample 5.5: snapshot inverse	91
5.17.2	Counterexample 5.6: state-dependent inverse changes	91
5.17.3	Counterexample 5.7: overly coarse observation	91
5.17.4	Counterexample 5.8: middleware order	91
5.18	Exercises	92
5.18.1	Core implementation	92
5.18.2	Commutation-matrix exercises	92
5.18.3	Mathematical exercises	92
5.18.4	Counterexample exercises	92
5.19	Milestone C, part 1: safe interleaving	92
5.20	Reading guide	93
5.20.1	Required	93
5.20.2	Applied comparison	93
5.20.3	Deepening	93
5.20.4	Read with these questions	93

6	Lab 6: Iteration, Asynchrony, Failure, and Safe Withdrawal	94
6.1	Purpose and outcomes	94
6.2	Motivation: transitions take time	94
6.3	The four-state lifecycle	95
6.4	Multi-step effects and iterators	96
6.4.1	A runner	97
6.4.2	Worked example 6.1: interruption at a boundary	97
6.5	Accumulation across iterations	98
6.5.1	Worked example 6.2: partial activation	98
6.6	Beginning and finishing activation	98
6.7	Target-change diversion	100
6.7.1	Worked example 6.3: replacement during loading	100
6.8	Asynchrony and inertia	100
6.8.1	Worked example 6.4: landing after staleness	101
6.9	Failure and rollback	101
6.9.1	Why failed fibers do not automatically retry	101
6.9.2	Worked example 6.5: port conflict	102
6.10	Why provider withdrawal is harder than notification	102
6.11	Visibility versus physical presence	102
6.12	The relied guard	102
6.13	Why the guard should not deadlock in an acyclic graph	104
6.13.1	Counterexample 6.1: cyclic dependency	104
6.14	Committed access during teardown	104
6.14.1	Worked example 6.6: connection pool teardown	105
6.15	Notification ordering	105
6.16	Chaining transitions	106
6.17	Executable invariants	106
6.17.1	Installed consumers name installed providers	106
6.17.2	No active provider is advertised while unloading	107
6.17.3	Committed binding remains accessible	107
6.17.4	Prefix rollback on failure	107
6.17.5	No overlapping transition per fiber	107
6.18	Counterexample workshop	107
6.18.1	Counterexample 6.2: delete binding before notifying	107
6.18.2	Counterexample 6.3: resolve globally during teardown	107
6.18.3	Counterexample 6.4: ignore an in-flight result	108
6.18.4	Counterexample 6.5: retry failure automatically	108
6.18.5	Counterexample 6.6: wait on all declared consumers rather than committed consumers	108
6.19	Exercises	108
6.19.1	Core implementation	108
6.19.2	Scenario exercises	108
6.19.3	Mathematical exercises	109
6.19.4	Counterexample exercises	109
6.20	Milestone C, part 2: robust lifecycle	109
6.21	Reading guide	109
6.21.1	Required	109
6.21.2	Applied comparison	109

6.21.3 Deepening	110
6.21.4 Read with these questions	110
7 Lab 7: Turning Metatheory into a Model Checker	111
7.1 Purpose and outcomes	111
7.2 Motivation: schedules hide bugs	111
7.3 What metatheory means here	112
7.4 A compact pure model	112
7.5 Rules as data	113
7.5.1 Worked example 7.1: L-Begin	114
7.6 Well-formedness	114
7.6.1 Worked example 7.2: a malformed state	115
7.7 Preservation	115
7.7.1 Connecting premises to clauses	116
7.8 Temporal composability as a trace property	116
7.8.1 Worked example 7.3	117
7.8.2 Assumption switch	117
7.9 Spatial composability as trace properties	117
7.9.1 Activation ordering	117
7.9.2 Withdrawal ordering	117
7.9.3 Resolution coherence	118
7.10 Progress	118
7.10.1 Why the unload guard seems dangerous	118
7.10.2 Assumptions behind progress	118
7.11 Termination and quiescence	119
7.12 Breadth-first exploration	119
7.13 Canonicalising states	120
7.13.1 Worked example 7.4: alpha-equivalent names	120
7.14 Fault switches and counterexample assignments	120
7.14.1 Switch: guardProviderUnload = false	121
7.14.2 Switch: useProviderIdentity = false	121
7.14.3 Switch: rollbackOnFailure = false	121
7.14.4 Switch: enforceUniqueProvision = false	121
7.14.5 Switch: preserveCommittedAccess = false	121
7.14.6 Switch: enforceIndependence = false	121
7.15 Reporting counterexamples	122
7.16 Property-based generation versus exhaustive exploration	122
7.17 Exercises	123
7.17.1 Pure model	123
7.17.2 Property checks	123
7.17.3 Counterexample assignments	123
7.17.4 Written theory	123
7.18 Deliverable	124
7.19 Reading guide	124
7.19.1 Required	124
7.19.2 Programming-languages foundations	124
7.19.3 Testing foundations	124
7.19.4 Read with these questions	124

8	Lab 8: Confluence, Reconciliation, and Hot Replacement	125
8.1	Purpose and outcomes	125
8.2	Motivation: dynamic history should not accumulate scars	125
8.3	Normal forms and confluence	126
8.3.1	What confluence does not claim	127
8.4	From independent transpositions to schedule equivalence	127
8.5	From-scratch assembly	127
8.5.1	Worked example 8.1: delete a closed episode	128
8.6	Support and total provision	128
8.6.1	Counterexample 8.1: conditional provision	128
8.7	Confluence experiment in the model checker	129
8.7.1	Assumption toggles	129
8.8	Declarative configuration	129
8.9	Keyed reconciliation	130
8.9.1	Worked example 8.2	131
8.10	Least-disruptive updates	131
8.11	Configuration reconciliation property	132
8.12	Hot module replacement as component replacement	132
8.13	Transactional replacement	133
8.13.1	Worked example 8.3: syntax failure	133
8.14	Rolling replacement through a broker	134
8.15	Capstone choices	134
8.15.1	Option A: CLI plugin host	134
8.15.2	Option B: agent tool registry	134
8.15.3	Option C: HTTP route framework	135
8.15.4	Option D: chatbot plugin host	135
8.15.5	Option E: data-processing pipeline	135
8.16	Capstone requirements	135
8.17	Final report structure	136
8.17.1	1. Runtime boundary	136
8.17.2	2. Component algebra	136
8.17.3	3. Lifecycle invariants	136
8.17.4	4. Theorem-to-system mapping	136
8.17.5	5. Counterexamples	136
8.17.6	6. Limitations	137
8.18	Exercises	137
8.18.1	Confluence experiments	137
8.18.2	Reconciliation implementation	137
8.18.3	HMR implementation	137
8.18.4	Written theory	137
8.19	Milestone D: metatheoretic runtime and capstone	138
8.20	Reading guide	138
8.20.1	Required	138
8.20.2	Applied comparison	138
8.20.3	Deepening	138
8.20.4	Read with these questions	138
9	Theory Companion and Reading Path	139

9.1	Three reading tracks	139
9.1.1	Minimum track	139
9.1.2	Standard track	139
9.1.3	Deepening track	139
9.2	Discrete mathematics for the paper	140
9.2.1	Motivation	140
9.2.2	Core definitions	140
9.2.3	Applied example: provider precedence	140
9.2.4	Recommended resource	140
9.2.5	Reading exercise	141
9.3	Operational semantics	141
9.3.1	Motivation	141
9.3.2	Definition	141
9.3.3	Worked example	141
9.3.4	Preservation and progress	141
9.3.5	Recommended resources	142
9.3.6	Applied exercise	142
9.4	Effects: three different ideas that share a name	142
9.4.1	Motivation	142
9.4.2	Runtime effect in this handbook	142
9.4.3	Static effect system	142
9.4.4	Algebraic effects and handlers	143
9.4.5	Relationship to the paper	143
9.4.6	Recommended resources	143
9.4.7	Reading questions	143
9.5	Coeffects	143
9.5.1	Motivation	143
9.5.2	Runtime interpretation in Mini-Cordis	144
9.5.3	Comonads: how much do you need?	144
9.5.4	Recommended resources	144
9.5.5	Classification exercise	144
9.6	The unified context and the context paradigm	144
9.6.1	Motivation	144
9.6.2	Definition	145
9.6.3	TypeScript approximation	145
9.6.4	Worked example: nested plugin groups	145
9.6.5	The context paradigm	146
9.6.6	Counterexample: escaping the assigned context	146
9.6.7	Exercises	146
9.7	Category theory: the minimum useful map	147
9.7.1	Motivation	147
9.7.2	Category	147
9.7.3	Endomorphism and monoid	147
9.7.4	Homomorphism	147
9.7.5	Opposite	147
9.7.6	Functor	148
9.7.7	Monad	148
9.7.8	Comonad	148

9.7.9	Fixed-point type	148
9.7.10	Quotient	148
9.7.11	Paper-to-programming table	148
9.7.12	Recommended resources	149
9.7.13	Counterexample to bad CT usage	149
9.8	Reversible computation versus revertible effects	149
9.9	Trace theory and concurrency	150
9.9.1	Motivation	150
9.9.2	Worked example	150
9.9.3	Recommended reading	150
9.9.4	Applied exercise	150
9.10	Applied comparison readings	150
9.10.1	React effects	151
9.10.2	Dependency injection and service dynamics	151
9.10.3	Hot module replacement	151
9.10.4	Property-based testing	151
9.11	Reading matrix by lab	152
9.12	Theory study exercises	152
10	Reference Architecture	153
10.1	Core algebra	153
10.2	Typed keys and bindings	154
10.3	Component and fiber	154
10.4	Lifecycle	155
10.5	Fiber context	156
10.6	Runtime orchestration	156
10.7	Lifecycle algorithm	157
10.8	Disposable scope algorithm	158
10.9	Tracing	158
10.10	Invariant checklist	159
10.10.1	Registry	159
10.10.2	Dependencies	159
10.10.3	Effects	159
10.10.4	Lifecycle	159
10.10.5	Quiescence	159
10.11	Suggested repository milestones	160
A	Notation and Glossary	161
A.1	Mathematical notation	161
A.2	Core terms	161
B	Proof and Property Templates	165
B.1	Proving an invariant	165
B.1.1	Example skeleton	165
B.2	Proving composite recovery	165
B.3	Proving selective withdrawal	166
B.4	Designing a property test	166
B.5	Designing a counterexample	167

C Lab Submission Checklists	168
C.1 Milestone A checklist	168
C.2 Milestone B checklist	168
C.3 Milestone C checklist	168
C.4 Milestone D checklist	168
C.5 Suggested rubric	169
D Bibliography and Resource Guide	170
D.1 Mathematical and PL foundations	170
D.2 Effects	170
D.3 Coeffects	170
D.4 Category theory and compositionality	171
D.5 Reversibility, traces, and concurrency	171
D.6 Applied comparisons	171
D.7 Testing and tooling	171
D.8 How to read research papers in this course	171
Closing Perspective	173

Preface

Modern software increasingly needs to add, remove, replace, and reconfigure components while the process remains alive. A plugin may register commands, start timers, provide a database service, subscribe to events, and create child components. Removing the plugin safely means withdrawing all of those effects without disturbing unrelated components. Replacing a service safely also means noticing which consumers depend on it, deactivating those consumers in the right order, and reactivating them against the replacement.

The paper *A Programming Paradigm for Spatiotemporal Composability* gives these requirements a formal account. It separates the problem into two dimensions:

- **Temporal composability:** when a component leaves, the modifications attributable to it can be recovered.
- **Spatial composability:** dependencies between components are declared, resolved, and reacted to as providers appear, disappear, or change identity.

The paper develops revertible effects, reactive coeffects, a unified context, a component calculus, and metatheoretic results about preservation, recovery, ordering, progress, and confluence. It then relates these ideas to Cordis, a TypeScript implementation.

This handbook is a student-facing companion to that work. Its purpose is not to simplify the paper into a collection of slogans. Its purpose is to help you build enough of the machinery that the notation acquires operational meaning. Every major idea is approached through the same sequence:

1. a practical failure that motivates the idea;
2. a precise definition;
3. a worked example small enough to calculate by hand;
4. an implementation in TypeScript;
5. an executable law or invariant;
6. a counterexample showing why an assumption matters.

The recurring relationship is:

paper notation $\longleftrightarrow$ small executable model $\longleftrightarrow$ property or invariant.

For example, the recovery equation

$$g(f(\gamma)) \simeq \gamma$$

will become both a mathematical statement and a property-based test that generates a context γ , applies an effect, applies the inverse returned by that effect, and checks that the recovered state is observationally equivalent to the original.

Scope of this handbook. The implementation is intentionally smaller than Cordis. It omits several production concerns until the core ideas are understood. It also uses simpler TypeScript types than the dependent types in the paper. Whenever the implementation is an approximation rather than a literal transcription, the text says so explicitly.

What you will build

By the end of the labs, you will have a small runtime capable of hosting dynamically managed CLI commands, services, event handlers, or agent tools.

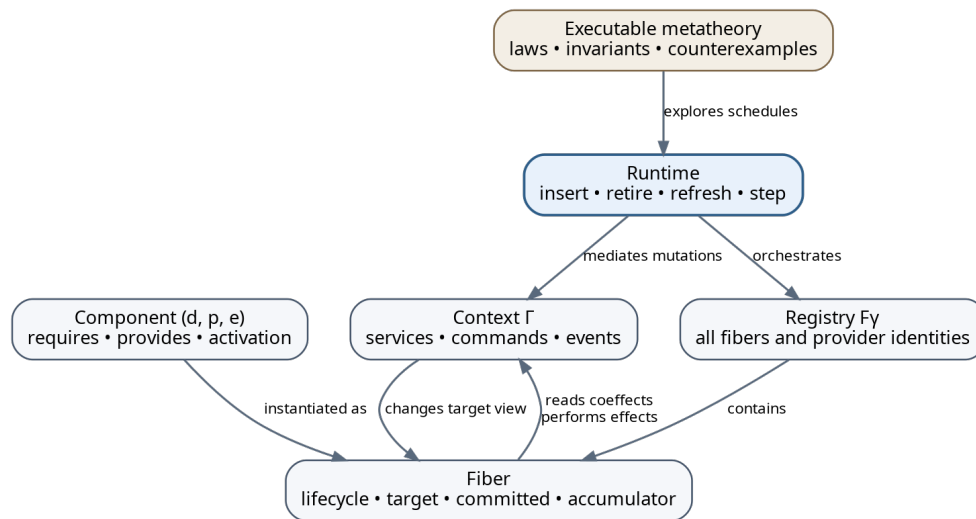


Figure 1: The final runtime separates context, component descriptions, live fibers, orchestration, and executable metatheory.

```

Runtime
├── Context Γ
│   ├── typed service bindings
│   ├── commands and event handlers
│   └── other context-mediated state
├── Component descriptions
│   ├── requirements d
│   └── provisions p

```



```

|   └─ activation program e
|
|   └─ Fiber registry Fy
|       └─ lifecycle state
|       └─ target view
|       └─ committed dependency view
|       └─ accumulated inverses
|
|   └─ Verification harness
|       └─ algebraic properties
|       └─ lifecycle invariants
|       └─ schedule exploration
|       └─ minimal counterexamples

```

The runtime will support:

- dynamic component insertion and retirement;
- automatic cleanup derived from setup operations;
- typed service keys and declared requirements;
- reactive activation and deactivation;
- provider replacement detection;
- asynchronous multi-step activation;
- rollback after interruption or failure;
- dependency-safe provider withdrawal;
- model-based checks of the paper’s main system properties;
- declarative reconciliation and a small hot-replacement experiment.

Audience and prerequisites

This handbook is aimed at advanced undergraduate students, graduate students, and experienced programmers entering programming-languages research. You should be comfortable with:

- TypeScript or another typed programming language;
- functions as values and closures;
- maps, sets, discriminated unions, and asynchronous functions;
- ordinary unit testing;
- basic mathematical notation for functions and sets.

You do **not** need prior category theory. The relevant categorical vocabulary is introduced only where it clarifies composition. You also do not need prior knowledge of effect systems, coeffect systems, or operational semantics.

How the labs are organised

Each lab contains the following sections.

- **Motivation** describes a concrete runtime failure.
- **Definitions** introduce the minimum mathematical vocabulary needed to state the problem precisely.
- **Worked examples** calculate a small case by hand before code hides the structure.
- **Implementation** gives API signatures, pseudocode, and design constraints.
- **Executable theory** converts definitions or theorems into tests and invariants.
- **Counterexample workshop** removes an assumption and asks you to make the failure observable.
- **Exercises and deliverables** define the assessed work.
- **Reading guide** states what to read and what question the reading should answer.

Study advice. Do not read a theorem as a block of symbols and then immediately look for a proof technique. First ask: What failure would this theorem rule out? Which runtime state would witness the failure? Which test would detect it?

Milestones

Milestone	Labs	Demonstrated capability
A. Reversible computation	0-2	State transformations, state-dependent inverses, twisted composition, LIFO recovery
B. Dynamic composition	3-4	Typed services, reactive requirements, components, fibers, target and committed views
C. Safe interleaving	5-6	Independence, observational equivalence, asynchronous rollback, guarded withdrawal
D. Metatheoretic runtime	7-8	Preservation checks, progress exploration, confluence experiments, reconciliation

Suggested schedule

A typical 10-12 week module can use the following rhythm.

Week	Main work
1	Orientation and Lab 0
2	Lab 1
3	Lab 2; Milestone A
4	Lab 3
5	Lab 4; Milestone B
6-7	Lab 5
8-9	Lab 6; Milestone C
10	Lab 7
11	Lab 8
12	Capstone demonstration and Milestone D report

Assessment model

A balanced assessment rewards both implementation and understanding.

Component	Suggested weight
Runtime implementation	35%
Property-based and model-based tests	25%
Mathematical explanations	20%
Counterexamples when assumptions are removed	15%
Capstone demonstration	5%

A counterexample is not a failed implementation. In this course it is evidence that you understand the boundary of a claim.

Development Environment

The reference language is TypeScript. It is a good fit because closures represent captured inverses, Map and Set model finite contexts, discriminated unions model lifecycle states, async generators model effect iterators, promises model in-flight work, and property-testing libraries are readily available.

Recommended project layout

```
mini-cordis/
├── src/
│   ├── algebra.ts
│   ├── effects.ts
│   ├── context.ts
│   ├── component.ts
│   ├── runtime.ts
│   ├── model.ts
│   └── examples/
├── test/
│   ├── algebra.test.ts
│   ├── effects.test.ts
│   ├── lifecycle.test.ts
│   └── model.test.ts
├── package.json
├── tsconfig.json
└── README.md
```

A minimal setup is:

```
npm init -y
npm install -D typescript vitest fast-check tsx @types/node
npx tsc --init
```

Add scripts similar to:

```
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest",
```

```
"typecheck": "tsc --noEmit",  
"demo": "tsx src/examples/demo.ts"  
}  
}
```

The labs use two kinds of tests:

- **example tests**, which check a chosen scenario;
- **property tests**, which generate many states and operations and check a law over all generated cases.

The examples use Vitest-style assertions and `fast-check`, but another test runner and property-testing library are acceptable.

Conventions used in the code

Early labs use immutable state because equality and composition are easier to inspect. Later labs may use internal mutation behind a controlled context API. The public semantics should remain clear even if the representation changes.

We use these basic aliases repeatedly:

```
export type Endo<S> = (state: S) => S;  
export type Dispose = () => void | Promise<void>;  
export type AsyncDispose = () => Promise<void>;
```

The word **effect** is overloaded in computing. In this handbook:

- a *plain state transformation* has type `Endo<S>`;
- a *reversible effect function* returns a successor and an inverse;
- an *external emission* such as sending a message may not be reversible at all;
- an *effect system* is the wider PL concept discussed in the reading notes.

Lab 0: Reading Programs as Mathematics

0.1 Purpose and outcomes

The source paper moves quickly between program behaviour and algebraic structure. Before building the runtime, you need a small vocabulary for describing state, sequencing, identity, partiality, relations, graphs, and transition systems.

After this lab, you should be able to:

- interpret a type or set as a collection of possible states;
- read $f : X \rightarrow Y$ as a function with a specified domain and codomain;
- calculate function composition in the same order that programs execute;
- state and test the monoid laws;
- distinguish a total function from a partial function;
- distinguish equality from an equivalence relation;
- draw a transition system and identify reachable states;
- explain why these ideas are relevant to plugin lifecycles.

0.2 Motivation: prose is not enough

Suppose a plugin host executes the following operations:

```
register command "hello"  
subscribe to event "message"  
start timer T
```

A vague cleanup requirement might say, “remove everything the plugin added.” That sounds clear until another plugin registers a different command, subscribes to the same event bus, and starts another timer. Now several questions appear.

- What counts as the state of the host?
- What exactly did the plugin change?
- In which order should changes be reversed?
- When are two recovered states considered the same?
- Which other components may move the state before cleanup runs?

Mathematics helps because it forces each of these questions to have a type and a law.

0.3 Types, sets, and state spaces

Motivation. A program cannot be reasoned about until we say what counts as a possible state.

Definition 0.1 (type or state space). A type X is treated extensionally as a collection of possible values. A **state space** Γ is the type of all contexts the runtime may occupy.

A small state space might be:

```
export type Context = Readonly<{
  commands: ReadonlyMap<string, string>;
  counters: ReadonlyMap<string, number>;
}>;
```

A particular value $\gamma \in \Gamma$ is one concrete state:

```
const gamma0: Context = {
  commands: new Map(),
  counters: new Map([["requests", 0]])
};
```

The Greek letter Γ names the type; lowercase γ names a value of that type. This convention appears throughout the paper.

Fundamentals: intensional versus extensional views. A TypeScript type also has syntax, declaration sites, and compiler behaviour. In the mathematics of this handbook, we usually care about the values it admits and the functions between those values. That is the extensional view.

0.3.1 Worked example 0.1: enumerating a finite state space

Let $B = \{0, 1\}$. A state with two Boolean flags has type $B \times B$. Its complete state space is:

$$B \times B = \{(0, 0), (0, 1), (1, 0), (1, 1)\}.$$

If the first flag means “database active” and the second means “API active,” then $(0, 1)$ may be invalid because an API requires a database. This distinction foreshadows a later one:

- the **state space** says which representations are possible;
- an **invariant** says which possible representations are well formed.

0.4 Functions and composition

Motivation. Sequential program execution is function composition.

Definition 0.2 (function). A function $f : X \rightarrow Y$ maps every value of X to exactly one value of Y .

Definition 0.3 (composition). Given $g : X \rightarrow Y$ and $f : Y \rightarrow Z$, their composition is the function

$$f \circ g : X \rightarrow Z$$

defined by

$$(f \circ g)(x) = f(g(x)).$$

The right-hand function executes first. If a program runs `increment` and then `double`, its mathematical composition is `double after increment`:

$$\text{double} \circ \text{increment}.$$

```
export const compose = <A, B, C>(  
  f: (b: B) => C,  
  g: (a: A) => B  
) : ((a: A) => C) =>  
  (a) => f(g(a));
```

0.4.1 Worked example 0.2: order matters

Let

$$\text{inc}(n) = n + 1, \quad \text{double}(n) = 2n.$$

Then

$$(\text{double} \circ \text{inc})(3) = 8,$$

while

$$(\text{inc} \circ \text{double})(3) = 7.$$

The functions do not commute. This simple calculation becomes important when we later ask whether effects from different components can be reordered.

0.5 Identity and monoids

Motivation. A runtime needs a meaningful “do nothing” operation, and it needs long sequences of operations to associate consistently.

Definition 0.4 (identity function). For a type X , the identity function is

$$\text{id}_X : X \rightarrow X, \quad \text{id}_X(x) = x.$$

Definition 0.5 (monoid). A monoid is a set M , a binary operation $\star : M \times M \rightarrow M$, and an identity element $e \in M$ satisfying:

1. **closure:** $a \star b \in M$ for all $a, b \in M$;
2. **associativity:** $(a \star b) \star c = a \star (b \star c)$;
3. **identity:** $e \star a = a = a \star e$.

The endomorphisms on Γ ,

$$\Gamma \rightarrow \Gamma,$$

form a monoid under composition, with id_Γ as identity.

Category theory connection. In any category, the endomorphisms of one object form a monoid under morphism composition. For this handbook, the programming reading is enough: state transformations compose, composition is associative, and doing nothing is an identity.

0.5.1 Worked example 0.3: arrays under concatenation

Arrays of strings form a monoid:

- operation: concatenation;
- identity: the empty array `[]`;
- associativity: regrouping concatenations does not change the resulting sequence.

Integers under subtraction do not form a monoid because subtraction is not associative:

$$(5 - 3) - 1 = 1, \quad 5 - (3 - 1) = 3.$$

One counterexample is enough to refute a universal law.

0.6 Partial functions

Motivation. Service lookup may fail because the requested key is absent.

Definition 0.6 (partial function). A partial function $f : X \rightharpoonup Y$ may be undefined for some values of X .

A `Map.get` operation is naturally partial. TypeScript represents the possibility of absence using `undefined`:

```
function lookup<K, V>(map: ReadonlyMap<K, V>, key: K): V | undefined {
  return map.get(key);
}
```

Later, the coefficient context will be modelled as a finite partial map from typed keys to values.

0.6.1 Counterexample 0.1: pretending a partial function is total

```
const service = services.get("database")!;
service.query("select 1");
```

The non-null assertion does not make the lookup total. It only suppresses the compiler's warning. A reactive runtime addresses the actual problem by activating a component only when all required bindings are present.

0.7 Relations and equivalence

Motivation. Recovery often cannot recreate the exact physical representation of a state, but it may recreate everything observable about that state.

Definition 0.7 (binary relation). A relation R on X is a predicate $R(x, y)$ over pairs of values in X .

Definition 0.8 (equivalence relation). A relation $\simeq$ is an equivalence relation when it is:

1. reflexive: $x \simeq x$;
2. symmetric: $x \simeq y$ implies $y \simeq x$;
3. transitive: $x \simeq y$ and $y \simeq z$ imply $x \simeq z$.

Equality is an equivalence relation, but it is often finer than needed.

0.7.1 Worked example 0.4: generated identifiers

Suppose two contexts contain the same commands but have different internal counters:

```
const a = {
  nextId: 41,
  commands: new Map([["hello", "plugin-a"]])
};

const b = {
  nextId: 99,
  commands: new Map([["hello", "plugin-a"]])
};
```

If the counter is not exposed by any operation, an observer may be unable to distinguish a from b. We can define:

```
function observe(ctx: ContextWithIds) {
  return [...ctx.commands.keys()].sort();
}
```

and then say:

$$a \simeq b \text{ iff } \text{observe}(a) = \text{observe}(b).$$

The full treatment appears in Lab 5.

0.8 Directed graphs and dependency order

Motivation. Dependencies impose an order on activation and withdrawal.

Definition 0.9 (directed graph). A directed graph consists of vertices and directed edges. In a component graph, an edge

$$A \longrightarrow B$$

will mean that B depends on something provided by A .

Definition 0.10 (directed acyclic graph, DAG). A DAG is a directed graph with no directed cycle.

Definition 0.11 (topological order). A topological order of a DAG lists every vertex so that each provider appears before each consumer that depends on it.

For the chain

```
Database -> Repository -> WebAPI
```

one topological order is exactly that sequence. Withdrawal must proceed in the reverse dependency direction:

```
WebAPI -> Repository -> Database
```

A cycle such as $A \rightarrow B \rightarrow A$ has no topological order.

0.9 Transition systems

Motivation. A lifecycle is not merely a set of states; it is a set of legal moves between states.

Definition 0.12 (transition system). A transition system consists of:

- a set of states S ;
- a transition relation $\rightarrow \subseteq S \times S$;
- optionally, labels identifying the rule that caused each transition.

A two-state component lifecycle is:

```
INACTIVE --reload--> ACTIVE
ACTIVE   --unload--> INACTIVE
```

A trace is a sequence

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n.$$

A state is **reachable** when some trace from the initial state ends there.

0.9.1 Worked example 0.5: invalid transition

If the only rules are reload and unload, the transition

```
INACTIVE --unload--> INACTIVE
```

is not legal merely because its source and destination are representable states. Operational semantics specify legal transitions, not just possible values.

0.10 Executable theory

Write ordinary tests for composition:

```
import { describe, expect, it } from "vitest";

const id = <A>(x: A): A => x;

it("composition is associative for this example", () => {
  const f = (x: number) => x + 1;
  const g = (x: number) => x * 2;
  const h = (x: number) => x - 4;

  const left = compose(f, compose(g, h));
  const right = compose(compose(f, g), h);

  expect(left(10)).toBe(right(10));
});

it("identity changes nothing", () => {
  const f = (x: number) => x * 3;
  expect(compose(id, f)(7)).toBe(f(7));
  expect(compose(f, id)(7)).toBe(f(7));
});
```

An example test does not prove the law for all functions, but it trains you to connect a law to executable behaviour. Later labs use property generators over restricted families of functions and states.

0.11 Exercises

0.11.1 Core exercises

1. Implement `compose`, `identity`, and `pipe` without using a utility library.
2. For each structure below, identify the carrier, operation, identity, and whether the operation is associative:
 - integers under addition;
 - strings under concatenation;
 - arrays under concatenation;
 - finite sets under union;
 - integers under subtraction;
 - non-empty strings under concatenation.
3. Give a concrete counterexample for every failed monoid law.
4. Define a four-state transition system for a download task: `IDLE`, `RUNNING`, `SUCCEEDED`, `FAILED`. List legal transitions.
5. Draw the component dependency graph for a small application containing a database, repository, HTTP server, metrics exporter, and health endpoint.

0.11.2 Explanation exercises

6. Explain in 150-250 words why function composition writes the operation that executes last on the left.
7. Explain the difference between “a state is representable” and “a state is reachable.”
8. Give an example of two program states that are unequal but reasonably treated as equivalent by a chosen observer.

0.11.3 Counterexample workshop

9. Construct a transition rule that appears harmless but makes an invalid state reachable. State the invariant it violates.
10. Define an observation function that is too coarse: it considers two states equivalent even though an allowed operation can distinguish them. Show the distinguishing operation.

0.12 Deliverable

Submit:

- `src/algebra.ts` containing the core definitions;
- `test/algebra.test.ts` containing at least eight example tests;
- a one-page note with your dependency graph, transition system, and two counterexamples.

0.13 Reading guide

0.13.1 Required

- Source paper: Sections 1.1, 2.3, and the opening of 3.1.
- *Mathematics for Computer Science*: selected material on functions, relations, directed graphs, invariants, and proof by counterexample.

0.13.2 Bridge reading

- Milewski, *Category Theory for Programmers*: “Category: The Essence of Composition” and “Types and Functions.”

0.13.3 Read with this question

Which facts about sequencing can be derived from the structure of composition, without inspecting the implementation of each individual operation?

Lab 1: Reversible Effects as State-Dependent Inverses

1.1 Purpose and outcomes

A conventional cleanup API separates setup from teardown. A plugin registers a command in one place and later tries to remember how to unregister it. The paper instead pairs every context-changing operation with the information needed to reverse that particular application.

After this lab, you should be able to:

- model a side effect as a transformation of an explicit context;
- distinguish a globally invertible function from an inverse witnessed at one application state;
- implement a reversible effect function of type $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$;
- derive the inverse of a sequential composition;
- explain twisted composition and its reverse ordering;
- test recovery laws with generated states;
- identify effects that fall outside the recoverable system boundary.

1.2 Motivation: cleanup separated from setup

Consider a command registry:

```
commands.set("hello", helloHandler);
```

A conventional component lifecycle may later call:

```
commands.delete("hello");
```

This looks harmless, but it hides several assumptions.

- Was "hello" absent before setup?
- Did another component replace the handler?
- Is deleting the key the correct restoration, or should an older value be restored?
- Will every successful setup operation be remembered in teardown?
- If setup fails halfway through, which subset should be cleaned up?

The locality problem is fundamental: the code that knows what changed is the code performing the change. A robust primitive should return its inverse at that point.

1.3 From impure functions to explicit context transformations

Motivation. An ordinary impure function hides the environment it changes. To reason about the change, make the environment explicit.

Suppose an impure operation has a programming type:

```
function register(name: string, handler: Handler): void
```

Its mathematical effect can be exposed by threading an explicit state:

$$f : \Gamma \times X \rightarrow \Gamma \times Y.$$

For a fixed input $x \in X$, the state-changing part is an endomorphism:

$$f_x : \Gamma \rightarrow \Gamma.$$

In the first implementation, keep the context immutable:

```
export type Handler = (input: string) => string;

export type Context = Readonly<{
  commands: ReadonlyMap<string, Handler>;
}>;

export type Endo<S> = (state: S) => S;
```

A plain transformation that installs a command is:

```
export function installCommand(
  name: string,
  handler: Handler
): Endo<Context> {
  return (ctx) => {
    const commands = new Map(ctx.commands);
    commands.set(name, handler);
    return { ...ctx, commands };
  };
}
```

The explicit representation does not itself provide cleanup. It only gives us a value on which composition and equality can be discussed.

1.4 What kind of inverse do we need?

The word *inverse* has several meanings. This lab uses a deliberately weak and practical one.

Definition 1.1 (global left inverse). Given $f, g : \Gamma \rightarrow \Gamma$, g is a global left inverse of f when

$$g \circ f = \text{id}_{\Gamma}.$$

Equivalently, for every $\gamma \in \Gamma$,

$$g(f(\gamma)) = \gamma.$$

A global inverse is convenient, but many useful operations do not have one. For example, `set x = 42` maps many previous states to the same successor. Once only the successor is known, the previous value has been lost.

The runtime can recover the previous value by capturing it *when the effect is applied*.

Definition 1.2 (inverse witnessed at an application state). Let $f : \Gamma \rightarrow \Gamma$. At a particular state γ , a function $g : \Gamma \rightarrow \Gamma$ witnesses reversal of f when

$$g(f(\gamma)) = \gamma.$$

No claim is made about $g(f(\gamma'))$ for unrelated states γ' .

This is weaker than global invertibility and is exactly what a closure can provide.

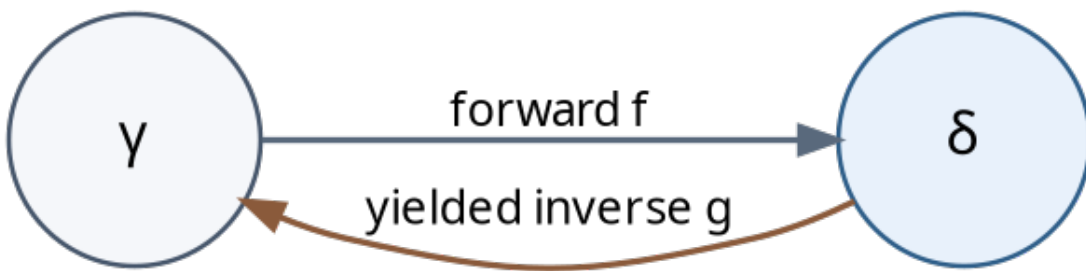


Figure 1.1: A reversible effect returns both the successor state and an inverse intended for that application.

```
application state γ
  |
  | forward effect f
```

```

      v
  successor state  $\delta = f(\gamma)$ 
      |
      | yielded inverse  $g$ 
      v
  recovered state  $\gamma$ 

```

Critical distinction. The paper uses a one-sided recovery condition. Do not assume that $f(g(\delta)) = \delta$, and do not assume that the yielded g reverses f at every possible state. The condition is local to the application that produced it.

1.4.1 Worked example 1.1: setting a value

Let the context be a map from strings to numbers. We want an operation `setValue("x", 42)`.

If `x` was absent, the inverse should delete it. If `x` held 7, the inverse should restore 7.

```

const MISSING = Symbol("missing");

type NumberContext = ReadonlyMap<string, number>;

export type Reversible<S> = (state: S) => readonly [S, Endo<S>];

export function setValue(
  key: string,
  value: number
): Reversible<NumberContext> {
  return (before) => {
    const old = before.has(key) ? before.get(key)! : MISSING;

    const after = new Map(before);
    after.set(key, value);

    const undo: Endo<NumberContext> = (current) => {
      const restored = new Map(current);
      if (old === MISSING) {
        restored.delete(key);
      } else {
        restored.set(key, old);
      }
      return restored;
    };
  };
}

```

```

    return [after, undo];
  };
}

```

For the state

$$\gamma = \{x \mapsto 7, y \mapsto 3\},$$

the operation produces

$$\delta = \{x \mapsto 42, y \mapsto 3\}$$

and an inverse closure containing the old value 7. Applying that inverse to δ recovers γ .

Notice that the inverse is not necessarily safe at an arbitrary future state. If another operation changes x before this inverse runs, blindly restoring 7 may overwrite that later contribution. Lab 5 studies the independence conditions that make selective withdrawal sound.

1.5 The effect-function type

Definition 1.3 (reversible effect function). A reversible effect function over Γ has type

$$\mathcal{E}_\Gamma = \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma).$$

Applied to γ , it returns a pair (δ, g) where:

- δ is the successor context;
- g is an inverse intended to recover the application.

In TypeScript:

```

export type Effect<S> = (
  state: S
) => readonly [next: S, inverse: Endo<S>];

```

Definition 1.4 (witnessed effect function). An effect is witnessed when, for every state at which it successfully applies, the inverse it returns satisfies

$$g(\delta) \simeq \gamma.$$

For now, $\simeq$ is ordinary equality. Lab 5 replaces it with observational equivalence.

TypeScript cannot prove this semantic property from the function type alone. It is an obligation on the effect author and a target for testing.

Type versus law. A type can require an operation to *return some function*. It cannot, in ordinary TypeScript, prove that the returned function is the correct inverse. The pair “API shape plus semantic law” recurs throughout programming-languages design.

1.5.1 Worked example 1.2: command registration with a precondition

There are two reasonable semantics for registering a command.

1. **Overwrite semantics:** installing `hello` replaces any previous binding, and the inverse restores the old binding.
2. **Fresh-key semantics:** installing `hello` is allowed only when the key is absent, and the inverse deletes it.

The paper’s coeffect set operation uses a fresh-key precondition. We can model partiality explicitly:

```
export type Result<T, E> =
  | Readonly<{ ok: true; value: T }>
  | Readonly<{ ok: false; error: E }>;

export type PartialEffect<S, E> = (
  state: S
) => Result<readonly [S, Endo<S>], E>;

export function registerFreshCommand(
  name: string,
  handler: Handler
): PartialEffect<Context, string> {
  return (before) => {
    if (before.commands.has(name)) {
      return { ok: false, error: `command already exists: ${name}` };
    }

    const commands = new Map(before.commands);
    commands.set(name, handler);
    const after = { ...before, commands };

    const undo: Endo<Context> = (current) => {
      const restored = new Map(current.commands);
      restored.delete(name);
      return { ...current, commands: restored };
    };

    return { ok: true, value: [after, undo] };
  };
}
```

```
};
}
```

The failed case produces no transition. An alternative formalisation could return an optional successor and compose effects in a `Maybe/Result` structure. The paper notes this choice but keeps failure outside the basic effect algebra until later.

1.6 Sequential composition

Motivation. A component performs more than one effect. We need to derive one composite inverse from the atomic inverses.

Suppose effect e_2 runs first:

$$e_2(\gamma) = (\delta, s),$$

then e_1 runs:

$$e_1(\delta) = (\varepsilon, t).$$

To recover γ from ε , first apply t to undo e_1 , then apply s to undo e_2 :

$$(s \circ t)(\varepsilon) = \gamma.$$

Definition 1.5 (effect composition). Define $e_1 \diamond e_2$ by

$$\begin{aligned} (e_1 \diamond e_2)(\gamma) = & \text{let } (\delta, s) = e_2(\gamma) \text{ in} \\ & \text{let } (\varepsilon, t) = e_1(\delta) \text{ in} \\ & (\varepsilon, s \circ t). \end{aligned}$$

The right-hand effect runs first, matching ordinary function composition.

```
export function composeEffects<S>(  
  later: Effect<S>,  
  earlier: Effect<S>  
) : Effect<S> {  
  return (initial) => {  
    const [middle, undoEarlier] = earlier(initial);  
    const [final, undoLater] = later(middle);  
  
    const undoComposite: Endo<S> = (state) =>  
      undoEarlier(undoLater(state));
```

```

    return [final, undoComposite];
  };
}

```

1.6.1 Worked example 1.3: two assignments

Start with:

$$\gamma_0 = \{x \mapsto 1, y \mapsto 2\}.$$

Run:

```

E1: set x = 10
E2: set y = 20

```

The forward path is:

```

{x=1,y=2}
--E1-->
{x=10,y=2}
--E2-->
{x=10,y=20}

```

The inverse path is:

```

{x=10,y=20}
--undo E2-->
{x=10,y=2}
--undo E1-->
{x=1,y=2}

```

Even though the two assignments happen to commute because they touch different keys, the derived inverse does not rely on that fact. It reverses the actual execution order.

1.7 Twisted composition

The paper first studies pairs of context transformations (f, g) where g is supplied uniformly. These pairs compose as:

$$(f_1, g_1) \circledast (f_2, g_2) = (f_1 \circ f_2, g_2 \circ g_1).$$

The forward components compose in ordinary order. The inverse components compose in the opposite order. This structure is sometimes described as a product of a monoid with its **opposite monoid**.

Definition 1.6 (opposite monoid). Given a monoid $(M, \star, e)$, its opposite has the same elements and identity but reverses multiplication:

$$a \star^{\text{op}} b = b \star a.$$

Fundamentals: why “twisted”? Nothing mysterious happens. A sequential computation writes a stack: the last successful action must be the first one undone. The algebra records that stack discipline.

1.7.1 Worked example 1.4: deriving the inverse order symbolically

Assume:

$$g_1 \circ f_1 = \text{id}, \quad g_2 \circ f_2 = \text{id}.$$

For the composite $f_1 \circ f_2$, test the candidate inverse $g_2 \circ g_1$:

$$\begin{aligned} (g_2 \circ g_1) \circ (f_1 \circ f_2) &= g_2 \circ (g_1 \circ f_1) \circ f_2 \\ &= g_2 \circ \text{id} \circ f_2 \\ &= g_2 \circ f_2 \\ &= \text{id}. \end{aligned}$$

Associativity lets us regroup without changing the operation order.

1.8 The effect monoid

The identity effect returns the state unchanged and yields the identity inverse:

$$\eta_{\Gamma}(\gamma) = (\gamma, \text{id}_{\Gamma}).$$

```
export const identityEffect = <S>(): Effect<S> =>
  (state) => [state, (current) => current];
```

Under $\diamond$, effect functions form a monoid:

- identity effect is the unit;
- effect composition is associative;
- witnessed effects remain witnessed under composition.

1.8.1 Proof sketch: witnessing survives composition

Suppose:

$$s(\delta) = \gamma, \quad t(\varepsilon) = \delta.$$

Then:

$$(s \circ t)(\varepsilon) = s(t(\varepsilon)) = s(\delta) = \gamma.$$

The proof is short because the operation was designed so the inverse order matches the data flow.

1.9 API design: atomic operations should return inverses

A useful Mini-Cordis API starts with a controlled set of atomic operations.

```
export interface EffectApi<S> {
  apply(effect: Effect<S>): void;
}
```

Domain-specific operations can hide direct state manipulation:

```
export interface CommandApi {
  registerCommand(name: string, handler: Handler): void;
}

export interface EventApi<E> {
  subscribe(listener: (event: E) => void): void;
}
```

Each method performs a primitive effect and records the inverse. The component author uses `registerCommand`; they do not separately write `unregisterCommand` in a distant deactivation callback.

1.9.1 Worked example 1.5: event subscription

A mutable event bus often returns an unsubscribe closure:

```
interface EventBus<E> {
  subscribe(listener: (event: E) => void): () => void;
}
```

This is already close to a reversible effect. The acquisition is registration; the returned closure is the inverse. To fit the explicit-state model, one could model the listener set as part of Γ . In an imperative runtime, the operation can mutate the bus and return `unsubscribe` directly.

The abstraction is more important than the representation:

```
perform one atomic acquisition
return the cleanup for that exact acquisition
record the cleanup immediately
```

1.10 Executable recovery laws

1.10.1 Example-based law

```
import { expect, it } from "vitest";

it("setValue returns an inverse for its application", () => {
  const initial = new Map<string, number>([["x", 7], ["y", 3]]);
  const [changed, undo] = setValue("x", 42)(initial);
  expect(undo(changed)).toEqual(initial);
});
```

1.10.2 Property-based law

```
import fc from "fast-check";

const mapArbitrary = fc.dictionary(fc.string(), fc.integer()).map(
  (record) => new Map(Object.entries(record))
);

it("setValue recovers every generated application state", () => {
  fc.assert(
    fc.property(
      mapArbitrary,
      fc.string(),
      fc.integer(),
      (initial, key, value) => {
        const [changed, undo] = setValue(key, value)(initial);
        expect(undo(changed)).toEqual(initial);
      }
    )
  );
});
```

Property testing is useful here because the law quantifies over states:

$$\forall \gamma. \text{ if } e(\gamma) = (\delta, g), \text{ then } g(\delta) = \gamma.$$

The generator approximates that universal quantifier over a chosen finite representation.

1.10.3 Composition property

```
it("the composition of witnessed effects is witnessed", () => {
  fc.assert(
    fc.property(
      mapArbitrary,
      fc.string(), fc.integer(),
      fc.string(), fc.integer(),
      (initial, k1, v1, k2, v2) => {
        const composite = composeEffects(
          setValue(k2, v2),
          setValue(k1, v1)
        );
        const [changed, undo] = composite(initial);
        expect(undo(changed)).toEqual(initial);
      }
    )
  );
});
```

1.11 Counterexample workshop: a plausible but wrong inverse

Consider:

```
export function brokenSetValue(
  key: string,
  value: number
): Effect<NumberContext> {
  return (before) => {
    const after = new Map(before);
    after.set(key, value);

    const undo = (current: NumberContext) => {
      const restored = new Map(current);
      restored.delete(key);
      return restored;
    };

    return [after, undo];
  };
}
```

The inverse works only when the key was initially absent. A generated counterexample is:

```
initial = { x -> 7 }  
apply set(x, 42) = { x -> 42 }  
apply undo      = { }
```

The runtime did not recover its application state.

1.11.1 Counterexample 1.2: logging

```
console.log("plugin activated");
```

What inverse could make an observer forget that the line appeared? Deleting a record from an internal log is not the same as removing text already seen by a user or forwarded to an external collector.

This motivates the **system boundary**.

Definition 1.7 (recoverable system boundary). A location is inside the recoverable context when the runtime can:

1. attribute modifications to components;
2. control the location sufficiently to restore it, at least up to the chosen equivalence.

Operations outside that boundary cannot be guaranteed reversible by the context mechanism.

1.11.2 Acquisition versus emission

Many real operations contain two phases.

- **Acquisition:** obtain a resource or install a local record, such as opening a socket, allocating a block, or subscribing a listener. The acquisition can often be undone by closing, freeing, or unsubscribing.
- **Emission:** send information through the acquired channel, such as transmitting bytes or charging a payment. The information has crossed the boundary and may require withholding, transactions, or compensation rather than an inverse.

Design note. “Has a cleanup function” is not equivalent to “undoes everything that happened.” Closing a network connection reclaims the connection; it does not retract packets already received by another process.

1.12 Exercises

1.12.1 Core implementation

1. Implement `setValue`, `deleteValue`, `incrementValue`, and `appendValue` as reversible effects.
2. For `deleteValue`, decide whether deletion of an absent key is an error, a neutral effect, or a successful effect with the identity inverse. Document the choice.

3. Implement `composeEffects` and `identityEffect`.
4. Implement `sequenceEffects(effects)` that executes a list from left to right and returns one composite inverse.
5. Add a `Result`-based variant for effects with preconditions.

1.12.2 Mathematical exercises

6. Prove the left and right identity laws for $\diamond$.
7. Prove associativity of $\diamond$ by expanding three effects. Track both successor states and inverse functions.
8. Show that if two uniform pairs (f_1, g_1) and (f_2, g_2) are globally witnessed, their twisted composite is globally witnessed.
9. Give an example of a state-dependent inverse that is not a global inverse.
10. Explain why the function type alone cannot express the witnessing law in ordinary TypeScript.

1.12.3 Property-testing exercises

11. Write generators for finite maps and lists.
12. Test recovery for every atomic effect.
13. Test that `sequenceEffects([])` is observationally identical to `identityEffect()`.
14. Mutate one inverse so it is wrong only on an edge case. Verify that shrinking finds a small counterexample.

1.12.4 Counterexample exercises

15. Find a pair of effects for which each inverse is locally correct but applying the first inverse after the second effect corrupts the state. Keep this counterexample for Lab 5.
16. Classify each operation as acquisition, emission, or a mixture:
 - allocate memory;
 - write to a private scratch file;
 - send an email;
 - register an HTTP route;
 - publish a message to a broker;
 - start a child process;
 - increment an external analytics counter.
17. For two operations outside the boundary, propose either a withholding strategy or a compensating action. State why it is weaker than exact recovery.

1.13 Deliverable

Submit:

- the effect type and at least four atomic effects;
- effect composition and identity;
- at least four property tests;
- one minimal counterexample demonstrating a wrong inverse;

- a 500-word boundary analysis of one real API used in a project you know.

1.14 Reading guide

1.14.1 Required

- Source paper: Sections 3.1.1 and 3.1.2, especially Definitions 1-16.
- Wadler, *Monads for Functional Programming*: the motivation and state examples.

1.14.2 Applied comparison

- React documentation on `useEffect` setup and cleanup. Compare its structural pairing with this lab's atomic effects.

1.14.3 Deepening

- Pretnar, *An Introduction to Algebraic Effects and Handlers*: read for the distinction between declaring an effect operation and interpreting it.
- Heunen, Kaarsgaard, and Karvonen, *Reversible Effects as Inverse Arrows*: read only the introduction at this stage.

1.14.4 Read with these questions

1. What does the paper require from an inverse that reversible-computing models may require globally?
2. Which part of effect composition follows from ordinary function composition?
3. Which part appears only because recovery must run in reverse order?

Lab 2: The Effect Context and Automatic Recovery

2.1 Purpose and outcomes

Lab 1 produced an inverse for one application and a composite inverse for a fixed sequence. A running component needs the runtime to accumulate those inverses as effects occur, including effects buried inside helper functions. This lab builds the first useful abstraction: a scope whose teardown is derived from its setup.

After this lab, you should be able to:

- define the effect context $\partial\Gamma$;
- explain the roles of current state and inverse accumulator;
- implement `track` and `recover`;
- explain why prepending new inverses yields LIFO recovery;
- build a component-local Scope API;
- compose child scopes into parent scopes;
- distinguish total recovery from selective withdrawal;
- state the soundness invariant of an effect context.

2.2 Motivation: helper functions should not leak cleanup obligations

Suppose plugin activation is written using helper functions:

```
function installUserCommands(ctx: Context) {
  ctx.command("user:add", addUser);
  ctx.command("user:remove", removeUser);
}

function installUserEvents(ctx: Context) {
  ctx.on("user.created", sendWelcomeEmail);
  ctx.on("user.deleted", removeCachedProfile);
}

function activate(ctx: Context) {
```

```
installUserCommands(ctx);
installUserEvents(ctx);
ctx.interval(refreshCache, 30_000);
}
```

A handwritten deactivation function must duplicate the structure of activation. If a helper is extended, teardown must also be extended. If activation fails halfway through, teardown must know exactly how far execution progressed.

A tracking scope changes the contract:

```
Every successful context mutation records its inverse immediately.
The component owns one accumulator containing all recorded inverses.
Unloading the component applies the accumulator once.
```

The component author composes setup operations. The runtime composes their inverses.

2.3 The effect context

Definition 2.1 (effect context). Given a context type Γ , define

$$\partial\Gamma = \Gamma \times (\Gamma \rightarrow \Gamma).$$

An element (γ, φ) contains:

- γ : the current context state;
- φ : an accumulator that recovers the state before the tracked effects.

The initial effect context is:

$$(\gamma_0, \text{id}_\Gamma).$$

A direct TypeScript representation is:

```
export type EffectContext<S> = Readonly<{
  current: S;
  recover: Endo<S>;
}>;

export function initialEffectContext<S>(state: S): EffectContext<S> {
  return {
    current: state,
    recover: (current) => current
  }
}
```

```
};
}
```

The identity accumulator says that no effects have yet been recorded.

Notation note. The symbol ∂ is not differentiation here. It marks a context extended with recovery information.

2.4 Tracking one effect

Suppose we have a uniform forward map f and candidate inverse g . The tracker should:

1. transform the current state with f ;
2. add g to the accumulator.

Definition 2.2 (track).

$$\text{track}_{\Gamma}(f, g)(\gamma, \varphi) = (f(\gamma), \varphi \circ g).$$

```
export function track<S>(  
  forward: Endo<S>,  
  inverse: Endo<S>  
) : Endo<EffectContext<S>> {  
  return ({ current, recover }) => ({  
    current: forward(current),  
    recover: (state) => recover(inverse(state))  
  });  
}
```

Why is the new accumulator $\varphi \circ g$ rather than $g \circ \varphi$? Because the newly performed effect must be undone first. If `recover` already undoes earlier work, the new inverse runs before the old accumulator:

```
current final state  
-> g           undo newest effect  
->  $\varphi$          undo all earlier effects  
-> original state
```


2.4.1 Worked example 2.1: tracking two effects

Start with:

$$(\gamma_0, \text{id}).$$

Track (f_1, g_1) :

$$(f_1(\gamma_0), g_1).$$

Track (f_2, g_2) :

$$(f_2(f_1(\gamma_0)), g_1 \circ g_2).$$

Applying the accumulator executes g_2 first and g_1 second because function application is right-to-left:

$$(g_1 \circ g_2)(\gamma_2) = g_1(g_2(\gamma_2)).$$

2.5 Recovery

Definition 2.3 (recover).

$$\text{recover}_\Gamma(\gamma, \varphi) = (\varphi(\gamma), \text{id}_\Gamma).$$

```
export function recover<S>(ctx: EffectContext<S>): EffectContext<S> {
  return {
    current: ctx.recover(ctx.current),
    recover: (state) => state
  };
}
```

Recovery both applies the accumulator and resets it. Resetting matters: applying an inverse twice is generally unsound because the second application is no longer acting on the state produced by the corresponding forward effect.

2.5.1 Worked example 2.2: exact recovery

Let:

```
E1 = set x to 10, inverse restores x to 1
E2 = set y to 20, inverse restores y to 2
```

Starting from $\{x=1, y=2\}$, tracking gives:

```
current = {x=10,y=20}
recover = restore-x-to-1 after restore-y-to-2
```

Calling `recover` produces:

```
current = {x=1,y=2}
recover = identity
```

2.6 The soundness invariant

Definition 2.4 (soundness invariant). If an effect context began at γ_0 , then a reachable pair (γ, φ) is sound when

$$\varphi(\gamma) \simeq \gamma_0.$$

The invariant says that although the current state changes, the accumulator still points back to the beginning of the scope.

2.6.1 Why each tracked effect preserves the invariant

Assume:

$$\varphi(\gamma) \simeq \gamma_0$$

and a new effect returns (δ, g) with

$$g(\delta) \simeq \gamma.$$

The new accumulator is $\varphi \circ g$, so:

$$(\varphi \circ g)(\delta) = \varphi(g(\delta)) \simeq \varphi(\gamma) \simeq \gamma_0.$$

When $\simeq$ is not equality, we additionally need inverses and accumulators to respect the relation. Lab 5 introduces that condition.

Proof pattern. State an invariant, show it holds initially, then show each transition preserves it. This pattern returns in Lab 7 under the name *preservation*.

2.7 Lifting state-dependent effect functions

Lab 1's effect function chooses its inverse after seeing the current state:

$$e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma).$$

The tracking operation therefore consumes an `Effect<S>` directly:

```
export function applyEffect<S>(
  ctx: EffectContext<S>,
  effect: Effect<S>
): EffectContext<S> {
  const [next, inverse] = effect(ctx.current);
  return {
    current: next,
    recover: (state) => ctx.recover(inverse(state))
  };
}
```

This is the core of the runtime effect primitive.

2.7.1 Worked example 2.3: activation with nested helpers

```
function activate(initial: Context): EffectContext<Context> {
  let scope = initialEffectContext(initial);

  scope = applyEffect(scope, registerCommand("hello", hello));
  scope = applyEffect(scope, subscribe("message", onMessage));
  scope = applyEffect(scope, startTimer("refresh", 30_000));

  return scope;
}
```

The caller does not separately construct teardown. It receives an effect context whose accumulator is teardown.

2.8 An imperative Scope facade

Explicit state threading is useful for proofs but awkward for application code. A runtime can preserve the same semantics behind an imperative facade.

```
export class Scope<S> {
  #current: S;
  #recover: Endo<S> = (state) => state;
  #armed = true;

  constructor(initial: S) {
    this.#current = initial;
  }

  get current(): S {
```

```

    return this.#current;
  }

  effect(effect: Effect<S>): void {
    if (!this.#armed) {
      throw new Error("scope is already disposed");
    }

    const [next, inverse] = effect(this.#current);
    const previousRecover = this.#recover;

    this.#current = next;
    this.#recover = (state) => previousRecover(inverse(state));
  }

  dispose(): S {
    if (!this.#armed) {
      return this.#current;
    }

    this.#armed = false;
    this.#current = this.#recover(this.#current);
    this.#recover = (state) => state;
    return this.#current;
  }
}

```

The armed flag makes disposal idempotent at the API level. The inverse stack itself is not assumed idempotent; the scope simply refuses to run it twice.

2.8.1 API signatures for the first useful runtime

```

export interface MiniContext {
  readonly state: ContextState;

  effect(effect: Effect<ContextState>): void;
  command(name: string, handler: Handler): void;
  on<E>(topic: string, listener: (event: E) => void): void;
  interval(name: string, milliseconds: number): void;
  provide<T>(key: Key<T>, value: T): void;
}

```

Each domain method should reduce to effect.

```
command(name: string, handler: Handler): void {  
  this.effect(registerCommand(name, handler));  
}
```

The discipline is:

Every mutation visible to the runtime must pass through the context effect primitive.

A direct write that bypasses the context lies outside the tracking guarantee.

2.9 Real resource wrappers

The immutable model is a teaching tool. Real APIs commonly return disposal closures directly.

2.9.1 Command registry

```
function registerCommandImperatively(  
  registry: Map<string, Handler>,  
  name: string,  
  handler: Handler  
) : Dispose {  
  if (registry.has(name)) {  
    throw new Error(`duplicate command: ${name}`);  
  }  
  registry.set(name, handler);  
  return () => registry.delete(name);  
}
```

2.9.2 Timer

```
function startInterval(  
  callback: () => void,  
  milliseconds: number  
) : Dispose {  
  const handle = setInterval(callback, milliseconds);  
  return () => clearInterval(handle);  
}
```

2.9.3 Event listener

```
function subscribe<E>(  
  bus: EventBus<E>,  
  listener: (event: E) => void
```

```

): Dispose {
  return bus.subscribe(listener);
}

```

An imperative effect runner can accumulate these closures:

```

export class DisposableScope {
  #dispose: AsyncDispose = async () => {};
  #armed = true;

  add(inverse: Dispose): void {
    const previous = this.#dispose;
    this.#dispose = async () => {
      await inverse();
      await previous();
    };
  }

  async dispose(): Promise<void> {
    if (!this.#armed) return;
    this.#armed = false;
    await this.#dispose();
    this.#dispose = async () => {};
  }
}

```

This is the practical interpretation of the accumulator.

2.10 Nested scopes and hierarchical composition

Motivation. A component may create child components or subordinate resources. Disposing the parent should dispose the child.

A child scope's disposer can itself be recorded as an effect of the parent:

```

const parent = new DisposableScope();
const child = new DisposableScope();

parent.add(() => child.dispose());

```

If the child records:

```

command A
listener B
interval C

```

then disposing the parent triggers the child's accumulator, which triggers C, B, then A cleanup.

This is the practical meaning of a tower such as $\partial^2\Gamma$: recovery at one level can be an effect tracked by the level above. The paper later unifies the recursive tower into one self-similar context type.

2.10.1 Worked example 2.4: parent and child order

```
Parent setup:
  P1
  create child
    C1
    C2
  P2
```

A single LIFO stack should recover:

```
undo P2
  dispose child
    undo C2
    undo C1
  undo P1
```

This preserves lexical nesting even though the component lifetime may be much longer than a lexical block.

2.11 Total recovery versus selective withdrawal

One accumulator is enough to undo the entire sequence in reverse order. It is not enough to remove an arbitrary earlier effect while retaining later effects.

Consider:

```
E1: set x = 1
E2: set x = 2
```

The stack can safely undo E2 and then E1. But attempting to undo only E1 while keeping E2 raises a question: what should the value of x become?

- Restoring the state before E1 gives the value before both effects.
- Leaving $x = 2$ means the inverse for E1 should somehow ignore the location changed by E2.

Selective withdrawal becomes sound when effects are independent, or when an explicit ordering discipline ensures the inverse meets the state it expects. Lab 5 formalises this distinction.

Checkpoint. LIFO recovery requires only locally witnessed inverses. Arbitrary removal order requires additional independence assumptions.

2.12 Failure during setup

Even before asynchronous iterators, the accumulator improves failure handling.

```
async function activate(scope: DisposableScope) {
  scope.add(registerCommandImperatively(commands, "hello", hello));
  scope.add(subscribe(bus, onMessage));

  // This may throw.
  const connection = await connectDatabase();
  scope.add(() => connection.close());
}
```

A caller can recover completed acquisitions:

```
const scope = new DisposableScope();
try {
  await activate(scope);
} catch (error) {
  await scope.dispose();
  throw error;
}
```

Lab 6 makes partial progress and failure explicit in the lifecycle state machine.

2.13 Executable invariants

2.13.1 Recovery test

```
it("disposing a scope recovers its initial state", () => {
  const initial = new Map<string, number>([["x", 1]]);
  const scope = new Scope(initial);

  scope.effect(setValue("x", 10));
  scope.effect(setValue("y", 20));
  scope.effect(setValue("z", 30));

  expect(scope.dispose()).toEqual(initial);
});
```


2.13.2 LIFO trace test

```
it("runs inverses in reverse order", async () => {
  const trace: string[] = [];
  const scope = new DisposableScope();

  scope.add(() => { trace.push("undo A"); });
  scope.add(() => { trace.push("undo B"); });
  scope.add(() => { trace.push("undo C"); });

  await scope.dispose();
  expect(trace).toEqual(["undo C", "undo B", "undo A"]);
});
```

2.13.3 At-most-once disposal

```
it("does not apply an accumulator twice", async () => {
  let count = 0;
  const scope = new DisposableScope();
  scope.add(() => { count += 1; });

  await scope.dispose();
  await scope.dispose();

  expect(count).toBe(1);
});
```

2.13.4 Intermediate soundness

For the immutable model, expose a test-only method returning the accumulated recovery function and check after every generated step:

```
expect(scope.recoveryTarget()).toEqual(initial);
```

Do not expose this method in the application-facing API. It exists to connect the implementation to the invariant.

2.14 Counterexample workshop

2.14.1 Counterexample 2.1: appending inverses in the wrong order

A common bug is:

```
this.#recover = (state) => inverse(previousRecover(state));
```

This applies the older accumulator before the newest inverse. Use two operations on the same key to expose the error.

```
initial x = 0
E1 sets x = 1
E2 sets x = 2
```

The correct recovery is:

```
undo E2: x = 1
undo E1: x = 0
```

The wrong order may run undo E1 at $x = 2$, then undo E2, ending at $x = 1$ rather than 0.

2.14.2 Counterexample 2.2: bypassing the context

```
scope.effect(registerCommand("safe", safeHandler));
rawCommands.set("leaked", leakedHandler); // untracked
```

After disposal, safe is gone but leaked remains. The abstraction can guarantee only what passes through its boundary.

2.14.3 Counterexample 2.3: disposal after an external overwrite

```
Component A registers command x.
Component B overwrites command x directly.
Component A disposes and deletes x.
```

A's inverse has now removed B's contribution. The local witnessing law was true at A's application state, but foreign interference invalidated selective withdrawal. This is the motivating counterexample for independence.

2.15 Exercises

2.15.1 Core implementation

1. Implement immutable EffectContext, track, applyEffect, and recover.
2. Implement Scope<S> with at-most-once disposal.
3. Implement DisposableScope for synchronous and asynchronous disposal closures.
4. Wrap at least four real resources: a command, event listener, timer, and service binding.
5. Implement a child scope whose disposer is recorded in its parent.

2.15.2 Mathematical exercises

6. Prove that $\text{track}(\text{id}, \text{id})$ is the identity transformation on $\partial\Gamma$.
7. Show that tracking two uniform pairs separately equals tracking their twisted composite.
8. Prove that one locally witnessed application preserves the soundness invariant.
9. Generalise the proof to a finite sequence by induction.

10. Explain why resetting the accumulator after recovery is part of the semantics rather than only an optimisation.

2.15.3 Design exercises

11. Decide whether `dispose()` should ignore, aggregate, or stop at cleanup errors. Compare the consequences for recovery guarantees.
12. Design a trace record that attributes every acquired resource to the scope that owns it.
13. Add diagnostic output showing the inverse stack without exposing the inverse closures themselves.
14. Explain which direct host APIs must be hidden or wrapped to preserve the context discipline.

2.15.4 Counterexample exercises

15. Implement the wrong composition order and find the smallest failing sequence.
16. Construct a helper function that mutates the host without using the scope. Demonstrate the leak.
17. Construct two individually witnessed effects where out-of-order selective withdrawal fails.
18. Show why applying the same inverse twice is not justified by the witness condition.

2.16 Milestone A: reversible computation

Your first milestone should demonstrate:

- at least four atomic effect primitives;
- automatic accumulation of inverses;
- exact or observational recovery after complete disposal;
- LIFO ordering;
- nested scope recovery;
- property tests for atomic and composite recovery;
- a counterexample for wrong inverse order;
- a written system-boundary analysis.

A successful demonstration should be able to load a small plugin that installs commands, listeners, and timers, then unload it with no plugin-specific teardown function.

2.17 Reading guide

2.17.1 Required

- Source paper: Definitions 2-16 and the conclusion of Section 3.1.3.
- Source paper: Section 5.1.1 for the correspondence with `ctx.effect` and its accumulator.

2.17.2 Applied comparison

- React `useEffect`: focus on the relationship between setup and returned cleanup.
- RAII or bracket-style resource management in a language you know: compare lexical lifetime with component lifetime.

2.17.3 Deepening

- Source paper Section 6.1 on system boundaries, acquisition, emission, withholding, and compensation.

2.17.4 Read with these questions

1. Which guarantee is supplied by stack discipline alone?
2. Which guarantee requires effects from different components to commute?
3. What must an application force through the context before it can claim complete recovery?

Lab 3: Reactive Coeffects and a Typed Service Context

3.1 Purpose and outcomes

Reversible effects answer, “What did this component change?” They do not answer, “What must already be available before this component can run?” A dynamic component system also needs requirements that can become satisfied or unsatisfied while the process is running.

After this lab, you should be able to:

- explain the effect/coeffect distinction;
- model a coeffect context as a finite partial map from typed keys to values;
- implement typed service keys in TypeScript;
- define and compute dependency satisfaction;
- classify a context transition as activating, deactivating, or neutral;
- make service provision itself a reversible effect;
- reactivate components when providers appear, disappear, or change;
- identify dependency cycles and explain why they remain inactive.

3.2 Motivation: optimistic lookup is not dependency management

Consider a reporting plugin:

```
function activate(ctx: Context) {  
  const database = ctx.get(DatabaseKey);  
  const clock = ctx.get(ClockKey);  
  // Build report service from both.  
}
```

If the database has not yet been loaded, there are several bad options.

- throw an error and hope the orchestrator retries;
- return undefined and scatter checks through the component;
- poll until the service appears;
- impose a global load order manually;
- restart the process whenever the configuration changes.

A reactive component model instead lets the component declare:

```
requires: database, clock
```

The runtime activates it only when both are available. If either disappears, the runtime deactivates it and later reactivates it when the requirement becomes satisfied again.

3.3 Effects and coeffects

The source paper uses two complementary questions.

Effect question: What does the computation do to its environment?

Coeffect question: What does the computation require from its environment?

A simple programming analogy is:

```
State-like computation: S -> (A, S)
Reader-like computation: R -> A
```

The first threads a changing state. The second consumes an environment. This analogy is useful but incomplete: the paper's contribution is to make requirements dynamic and reactive at runtime rather than merely static function parameters.

Definition 3.1 (coeffect). In this handbook, a coeffect is a declared environmental requirement mediated by the runtime. A component may run only while the required bindings are resolved according to its committed view.

Terminology warning. Coeffect systems in programming-language theory include liveness, resource usage, implicit parameters, dataflow neighbourhoods, permissions, and other context requirements. Mini-Cordis implements one concrete runtime interpretation: dynamically resolved component dependencies.

3.4 The coeffect context as a partial dependent map

The paper defines a coeffect context as:

$$\Sigma = (k : K) \multimap \mathcal{V}_k.$$

Read this in pieces.

- K is the type of keys.
- For each key k , $\mathcal{V}_k$ is the value type associated with that key.
- $\multimap$ means the map is partial: some keys may be absent.

The phrase **dependent map** means that the type of the stored value depends on the key.

3.4.1 A TypeScript approximation

TypeScript cannot directly store an arbitrary dependent function, but a generic key can carry its value type:

```
export class Key<T> {
  readonly id: symbol;

  constructor(readonly description: string) {
    this.id = Symbol(description);
  }
}

export const DatabaseKey = new Key<Database>("database");
export const ClockKey = new Key<Clock>("clock");
```

A service store can keep values under symbol identity:

```
export class ServiceStore {
  #values = new Map<symbol, unknown>();

  has<T>(key: Key<T>): boolean {
    return this.#values.has(key.id);
  }

  get<T>(key: Key<T>): T | undefined {
    return this.#values.get(key.id) as T | undefined;
  }

  set<T>(key: Key<T>, value: T): void {
    this.#values.set(key.id, value);
  }

  delete<T>(key: Key<T>): void {
    this.#values.delete(key.id);
  }

  clone(): ServiceStore {
    const copy = new ServiceStore();
    for (const [id, value] of this.#values) {
      copy.#values.set(id, value);
    }
    return copy;
  }
}
```

The cast is isolated inside the store. The public API preserves the relationship between `Key<T>` and `T`.

Design note: key identity. Use unique nominal identity, such as a `symbol` or object reference, rather than unqualified strings. Strings make accidental collisions between independently developed components much easier. Versioning and structural compatibility remain separate problems.

3.4.2 Worked example 3.1: typed lookup

```
interface Database {
  query(sql: string): Promise<readonly unknown[]>;
}

interface Clock {
  now(): Date;
}

const store = new ServiceStore();
const clock: Clock = { now: () => new Date() };
store.set(ClockKey, clock);

const resolved = store.get(ClockKey); // Clock | undefined
resolved?.now();
```

Trying to store a number under `ClockKey` is rejected at the call site:

```
store.set(ClockKey, 42); // type error
```

The runtime cast is not a proof against malicious or untyped code, but it gives ordinary components a typed interface.

3.5 Requirements and satisfaction

Definition 3.2 (coeffect specification). A component requirement is a finite set of keys:

$$d \subseteq K.$$

```
export type Requirement = ReadonlySet<Key<unknown>>;
```

Because `Set<Key<unknown>>` loses some per-key value precision, a practical API may store a readonly tuple or object. For satisfaction, only key identity is needed.

Definition 3.3 (satisfaction). A coeffect context σ satisfies a requirement d when every required key is present:

$$\sigma \models d \iff \forall k \in d. k \in \text{dom}(\sigma).$$

```
export function satisfies(
  store: ServiceStore,
  requirements: Requirement
): boolean {
  for (const key of requirements) {
    if (!store.has(key)) return false;
  }
  return true;
}
```

3.5.1 Worked example 3.2: a two-key requirement

Let:

$$d = \{\text{database}, \text{clock}\}.$$

Available keys	Satisfies d ?
none	no
database	no
clock	no
database, clock	yes
database, clock, metrics	yes

Satisfaction is monotone with respect to adding unrelated keys in this simple model. Later, provider identity and isolation make resolution richer than membership alone.

3.6 Reactive notification

A requirement matters dynamically because context changes are classified against it.

Definition 3.4 (notification class). Given a requirement d and old/new contexts σ, σ' , define:

$$\text{notify}_d(\sigma, \sigma') = \begin{cases} \text{activating,} & \sigma \not\models d \wedge \sigma' \models d, \\ \text{deactivating,} & \sigma \models d \wedge \sigma' \not\models d, \\ \text{neutral,} & \text{otherwise.} \end{cases}$$

```

export type Notification =
  | "activating"
  | "deactivating"
  | "neutral";

export function classify(
  before: ServiceStore,
  after: ServiceStore,
  requirements: Requirement
): Notification {
  const wasSatisfied = satisfies(before, requirements);
  const isSatisfied = satisfies(after, requirements);

  if (!wasSatisfied && isSatisfied) return "activating";
  if (wasSatisfied && !isSatisfied) return "deactivating";
  return "neutral";
}

```

3.6.1 Worked example 3.3: one change, different classifications

Suppose database is added.

- Component A requires {database}: activating.
- Component B requires {database, clock} and clock is absent: neutral.
- Component C requires {metrics}: neutral.
- Component D was already satisfied by {clock}: neutral.

Notification is always relative to a specification. A context transition is not intrinsically “activating” for the whole system.

3.7 Provision is itself a revertible effect

Providing a service changes the coeffect context. Therefore service provision should use the effect machinery from Labs 1-2.

For fresh-key semantics:

```

export function provideService<T>(
  key: Key<T>,
  value: T
): PartialEffect<ServiceStore, string> {
  return (before) => {
    if (before.has(key)) {
      return {
        ok: false,

```

```

        error: `service already provided: ${key.description}`
    };
}

const after = before.clone();
after.set(key, value);

const undo: Endo<ServiceStore> = (current) => {
    const restored = current.clone();
    restored.delete(key);
    return restored;
};

return { ok: true, value: [after, undo] };
}

```

If your `ServiceStore` is mutable, the same semantics can return a disposal closure:

```

function provide<T>(store: ServiceStore, key: Key<T>, value: T): Dispose {
    if (store.has(key)) {
        throw new Error(`duplicate service: ${key.description}`);
    }
    store.set(key, value);
    return () => store.delete(key);
}

```

This is an important unification:

```

coeffect operation: install or withdraw a dependency binding
effect interpretation: context mutation paired with its inverse

```

The dependency table is not a separate magical subsystem. It is part of the same tracked context.

3.8 A first reactive runtime

Define a component description:

```

export interface SimpleComponent {
    readonly name: string;
    readonly requires: Requirement;
    activate(ctx: ComponentContext): Dispose | Promise<Dispose>;
}

```

Track whether it is active:

```
export interface SimpleInstance {
  readonly component: SimpleComponent;
  active: boolean;
  dispose?: Dispose;
}
```

A simple refresh operation is:

```
async function refresh(instance: SimpleInstance, ctx: ComponentContext) {
  const shouldRun = satisfies(ctx.services, instance.component.requires);

  if (shouldRun && !instance.active) {
    instance.dispose = await instance.component.activate(ctx);
    instance.active = true;
    return;
  }

  if (!shouldRun && instance.active) {
    await instance.dispose?();
    instance.dispose = undefined;
    instance.active = false;
  }
}
```

Whenever a service changes, refresh affected components. A deliberately simple implementation may refresh all instances:

```
async function refreshAll(instances: readonly SimpleInstance[], ctx: ComponentContext) {
  for (const instance of instances) {
    await refresh(instance, ctx);
  }
}
```

Later, notification will target only fibers whose declared keys changed.

3.9 Worked system: database, repository, and API

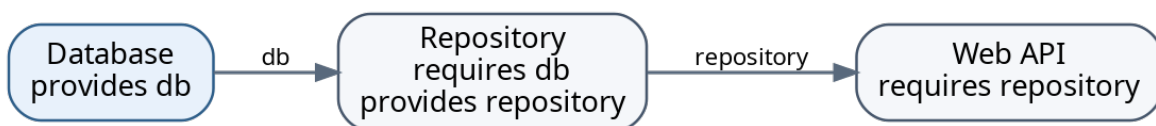


Figure 3.1: A provider chain. Each arrow points from a provider to a consumer.

```

Database
  provides: database
    |
    v
Repository
  requires: database
  provides: repository
    |
    v
WebAPI
  requires: repository

```

3.9.1 Components

```

const DatabaseComponent: SimpleComponent = {
  name: "database",
  requires: new Set(),
  async activate(ctx) {
    const db = await openDatabase();
    const unprovide = ctx.provide(DatabaseKey, db);
    return async () => {
      await unprovide();
      await db.close();
    };
  }
};

```

```

const RepositoryComponent: SimpleComponent = {
  name: "repository",
  requires: new Set([DatabaseKey]),
  activate(ctx) {
    const db = ctx.require(DatabaseKey);
    const repository = makeRepository(db);
    return ctx.provide(RepositoryKey, repository);
  }
};

```

```

const ApiComponent: SimpleComponent = {
  name: "api",
  requires: new Set([RepositoryKey]),
  activate(ctx) {
    const repository = ctx.require(RepositoryKey);
    return ctx.command("users:list", () => repository.listUsers());
  }
};

```

```
}
};
```

3.9.2 Insertion trace

Insert the components in the least helpful order:

```
insert WebAPI      -> requirement unsatisfied; remains inactive
insert Repository  -> database absent; remains inactive
insert Database    -> activates and provides database
refresh Repository -> activates and provides repository
refresh WebAPI     -> activates and registers command
```

The dependency declaration constrains **activation order**, not module discovery order.

3.9.3 Withdrawal trace

A naive implementation might do:

```
remove database binding
refresh Repository -> deactivates
refresh WebAPI     -> deactivates
```

This detects the loss of satisfaction, but it is not yet dependency-safe. Repository teardown may still need the database. Lab 6 separates “stop advertising” from “destroy the binding” and introduces guarded withdrawal.

3.10 Requirements should authorise access

A requirement declaration is useful only if access is mediated.

```
export class ComponentContext {
  constructor(
    readonly services: ServiceStore,
    readonly declared: Requirement
  ) {}

  require<T>(key: Key<T>): T {
    if (!this.declared.has(key as Key<unknown>)) {
      throw new Error(`undeclared dependency: ${key.description}`);
    }

    const value = this.services.get(key);
    if (value === undefined) {
      throw new Error(`inactive dependency: ${key.description}`);
    }
  }
}
```

```

    return value;
  }
}

```

This runtime check establishes a capability-like discipline for context-mediated dependencies:

- declaring a key requests authority to access it;
- the context mediates the request;
- undeclared access is rejected;
- absent access is rejected.

It is not a security sandbox. Code with a direct reference to the underlying store can bypass the check. Sandboxing requires a stronger execution boundary.

3.11 Provider uniqueness and collisions

The simplest calculus gives each key at most one active provider. This makes resolution deterministic.

Invariant 3.1 (unique provision). For distinct active components A and B :

$$p_A \cap p_B = \emptyset.$$

A runtime can enforce the invariant at insertion or activation.

```

function assertDisjointProvision(
  incoming: ReadonlySet<Key<unknown>>,
  existing: readonly ReadonlySet<Key<unknown>>[]
): void {
  for (const provision of existing) {
    for (const key of incoming) {
      if (provision.has(key)) {
        throw new Error(`duplicate provider for ${key.description}`);
      }
    }
  }
}

```

This restriction is relaxed by isolation realms or service brokers, but those are later extensions. For now, one key has one possible provider.

3.12 Dependency cycles

Consider:

```
A requires b and provides a
B requires a and provides b
```

Initially neither a nor b is present.

- A cannot activate until b is present.
- B cannot activate until a is present.
- Neither component can create its provision before activation.

The system is not deadlocked in the concurrency sense; it is simply unsatisfied. The cycle can be detected from declarations.

3.12.1 Worked decomposition

Suppose a server and access controller appear mutually dependent:

```
Server requires policy and provides server
Policy requires server and provides policy
```

Separate core services from integration behaviour:

```
ServerCore      provides server
PolicyCore      provides policy
RequestMediation requires server, policy
PolicyManagement requires server, policy
```

The core providers no longer depend on each other. Two integration components express the bidirectional interactions.

Design lesson. A dependency cycle often indicates that components are too coarse. Splitting stable capabilities from integration logic can restore an acyclic provider graph.

3.13 Executable theory

3.13.1 Satisfaction test

```
it("requires every declared key", () => {
  const store = new ServiceStore();
  const requirement = new Set<Key<unknown>>([DatabaseKey, ClockKey]);

  expect(satisfies(store, requirement)).toBe(false);
  store.set(DatabaseKey, fakeDatabase);
  expect(satisfies(store, requirement)).toBe(false);
});
```



```
store.set(ClockKey, fakeClock);
expect(satisfies(store, requirement)).toBe(true);
});
```

3.13.2 Classification table test

```
it.each([
  [false, false, "neutral"],
  [false, true, "activating"],
  [true, false, "deactivating"],
  [true, true, "neutral"]
] as const)("classifies %s -> %s as %s", (before, after, expected) => {
  expect(classifyBooleans(before, after)).toBe(expected);
});
```

3.13.3 Reactivity invariant

After the runtime reaches a stable point:

```
for (const instance of instances) {
  expect(instance.active).toBe(
    satisfies(ctx.services, instance.component.requires)
  );
}
```

This is a first approximation. Lab 4 replaces the Boolean with target and committed provider views.

3.13.4 Provision recovery

```
it("unproviding a service reverses provision", async () => {
  const store = new ServiceStore();
  const undo = provide(store, ClockKey, fakeClock);
  expect(store.has(ClockKey)).toBe(true);

  await undo();
  expect(store.has(ClockKey)).toBe(false);
});
```

3.14 Counterexample workshop

3.14.1 Counterexample 3.1: hidden dependency

```
const MetricsComponent = {
  requires: new Set(),
  activate(ctx: ComponentContext) {
    const db = ctx.services.get(DatabaseKey)!; // bypasses declaration
    // ...
  }
};
```

The runtime believes this component is independent of the database. Removing the database will not trigger deactivation, and the hidden reference may become stale.

3.14.2 Counterexample 3.2: notification only on addition

If the runtime refreshes consumers when a service is added but not when it is removed, it establishes activation ordering but not safe continued operation. Components remain active after their requirements become unsatisfied.

3.14.3 Counterexample 3.3: comparing only satisfaction

Suppose Database V1 is replaced immediately by Database V2. The key remains present throughout:

```
before: database provided by fiber 17
after:  database provided by fiber 29
```

A Boolean satisfaction check sees `true -> true`, hence neutral. But consumers may hold resources or caches tied to V1. Lab 4 records provider identity in a target view so replacement becomes observable.

3.14.4 Counterexample 3.4: string-key collision

Two packages independently choose the string "cache" for unrelated interfaces. A consumer may resolve the wrong value. Unique key objects prevent accidental nominal collision, though interface versioning remains an ecosystem concern.

3.15 Exercises

3.15.1 Core implementation

1. Implement `Key<T>` and `ServiceStore`.
2. Implement typed `get`, `has`, `set`, and `delete` operations.
3. Implement `Requirement`, `satisfies`, and notification classification.
4. Make `provide` a tracked effect whose inverse withdraws the binding.
5. Implement a simple runtime that refreshes all components after each service change.
6. Enforce declared access in `ComponentContext.require`.
7. Enforce disjoint provisions.

3.15.2 Worked-system exercises

8. Implement Database, Repository, and WebAPI components.
9. Insert them in all six possible orders and show that the same components eventually become active.
10. Remove each provider in turn and record the resulting activation/deactivation trace.
11. Add an optional Metrics component that can activate independently.
12. Replace Database V1 with V2 without changing key presence. Record what the Boolean model fails to notice.

3.15.3 Mathematical exercises

13. Prove that satisfaction is decidable for a finite requirement and finite store.
14. Show that adding an unrelated key cannot make a satisfied requirement unsatisfied.
15. Show that removing an unrelated key cannot change satisfaction.
16. Draw the dependency graph induced by provisions and requirements.
17. State a condition under which the graph has a topological order.

3.15.4 Counterexample exercises

18. Introduce a hidden dependency and write a test that exposes stale access.
19. Disable removal notifications and show a component remaining active illegally.
20. Create a two-component cycle and explain why no activation rule applies.
21. Refactor the cycle into core and integration components.
22. Replace nominal key objects with strings and demonstrate a collision.

3.16 Deliverable

Submit:

- typed key and store implementation;
- reactive satisfaction and notification logic;
- three-component dependency-chain demo;
- access mediation and unique-provider checks;
- tests for all notification classes;
- one dependency-cycle analysis;
- one counterexample involving a hidden dependency or provider replacement.

3.17 Reading guide

3.17.1 Required

- Source paper: Sections 2.2-2.3 and 3.2.1-3.2.2.
- Petricek, Orchard, and Mycroft, *Coeffects: Unified Static Analysis of Context-Dependence*: introduction and motivating examples.

3.17.2 Applied comparison

- Martin Fowler, *Inversion of Control Containers and the Dependency Injection Pattern*.
- OSGi Declarative Services: read the introduction and service-reference lifecycle at a conceptual level.

3.17.3 Deepening

- Source paper Section 3.2.3 on isolation and interception.
- Milewski, *Category Theory for Programmers*: the chapter on comonads only after the effect/coeffect distinction is already clear.

3.17.4 Read with these questions

1. What is gained by declaring requirements before activation rather than failing at lookup time?
2. In what sense is a dependency table a coeffect context?
3. Why is service provision simultaneously a coeffect operation and a revertible effect?

Optional Studio 3A: Isolation, Interception, and Derived Contexts

This studio extends Lab 3 with the two mechanisms from the paper’s Section 3.2.3. Complete it when the course needs multi-tenant contexts, tests with local service substitutions, or policy metadata applied without editing components.

Purpose and outcomes

After this studio, you should be able to:

- distinguish in-place effects from derived contexts;
- resolve one logical key to different values in different realms;
- implement context-local service isolation;
- define interception metadata with a monoid operation;
- merge component-declared and context-carried metadata;
- explain why isolation/interception need not mutate the shared parent store;
- distinguish access mediation from security sandboxing.

3A.1 Motivation: one logical key, several local meanings

A flat service table permits one binding per key. That is insufficient for:

- two tenants using different databases;
- a test component substituting a fake clock;
- a child application overriding configuration locally;
- a sandbox attenuating filesystem access;
- tracing metadata applied to all calls in one subtree.

Copying every component and renaming every key would make declarations brittle. Instead, derive a child context that changes how a key resolves.

3A.2 In-place and derived realisation

Definition 3A.1 (in-place realisation). An operation mutates the current context and returns a non-trivial inverse. Service provision is the main example.

Definition 3A.2 (derived realisation). An operation leaves the parent intact and returns a child context with adjusted resolution. Recovery discards the child; no mutation of the parent needs to be undone.

```

parent context
  |
  | derive with local override
  v
child context

recover child = stop using/discard child
parent was never changed

```

This distinction is useful in imperative hosts. The semantic operation still has a clear context transformation, but the implementation may realise it by persistent inheritance rather than mutation plus inverse.

3A.3 Isolation realms

Motivation. A logical key such as `DatabaseKey` should resolve through a context-specific realm identifier.

Definition 3A.3 (isolated coeffect context). An isolated context contains:

$$\Sigma_{\text{iso}} = (K \multimap R) \times ((r : R) \multimap \mathcal{V}_r).$$

Interpretation:

- $\rho : K \multimap R$ maps a logical key to a realm;
- σ maps a realm to its actual value;
- keys not explicitly isolated may use their own identity as the default realm.

Resolution is two-stage:

$$k \mapsto \rho(k) \mapsto \sigma(\rho(k)).$$

TypeScript representation

```

export type Realm = symbol;

export class IsolatedStore {
  constructor(
    readonly parent?: IsolatedStore,
    readonly realms = new Map<symbol, Realm>(),
    readonly values = new Map<Realm, Binding<unknown>>()
  ) {}

  realmOf<T>(key: Key<T>): Realm {

```

```

    return this.realms.get(key.id)
    ?? this.parent?.realmOf(key)
    ?? key.id;
  }

  get<T>(key: Key<T>): Binding<T> | undefined {
    return this.getByRealm<T>(this.realmOf(key));
  }

  getByRealm<T>(realm: Realm): Binding<T> | undefined {
    return (
      this.values.get(realm) as Binding<T> | undefined
    ) ?? this.parent?.getByRealm<T>(realm);
  }

  deriveIsolation<T>(key: Key<T>, realm: Realm = Symbol()): IsolatedStore {
    const realms = new Map<symbol, Realm>();
    realms.set(key.id, realm);
    return new IsolatedStore(this, realms);
  }
}

```

A complete implementation must ensure that parent lookup uses the resolved realm rather than re-resolving the logical key against the parent's realm table.

Worked example 3A.1: two tenant databases

```

root context:
  DatabaseKey -> realm global-db
  global-db   -> administration database

child tenant A:
  DatabaseKey -> realm tenant-a-db
  tenant-a-db -> database A

child tenant B:
  DatabaseKey -> realm tenant-b-db
  tenant-b-db -> database B

```

The same Repository component declares DatabaseKey in all three contexts. Each fiber resolves a different provider according to its local realm mapping.

Recovery

Discarding tenant A's child context removes its realm override and locally provided binding. Root and tenant B remain unchanged.

3A.4 Interception metadata

Motivation. Sometimes the binding should remain the same while the way it is used changes.

Examples:

- add a tracing span name;
- restrict filesystem paths;
- mark database access read-only;
- attach a tenant ID;
- impose a timeout or rate limit.

Definition 3A.4 (metadata monoid). For each key k , metadata values form a monoid:

$$(\mathcal{M}_k, \oplus_k, \epsilon_k).$$

The operation combines metadata; ϵ_k is empty metadata.

A component may declare metadata, and an enclosing context may carry additional metadata. The provider receives their merge.

Provider shape

Instead of storing a value directly, store a provider function:

```
export interface Provider<T, M> {
  resolve(metadata: M): T;
}
```

Example metadata

```
interface FilePolicy {
  readonly readable: ReadonlySet<string>;
  readonly writable: ReadonlySet<string>;
  readonly readOnly: boolean;
}

const emptyFilePolicy: FilePolicy = {
  readable: new Set(),
  writable: new Set(),
  readOnly: false
};
```

Define an associative merge with identity. For example, path sets may intersect to attenuate authority, while `readOnly` may combine with logical OR.


```
function mergeFilePolicy(left: FilePolicy, right: FilePolicy): FilePolicy {
  return {
    readable: intersect(left.readable, right.readable),
    writable: intersect(left.writable, right.writable),
    readOnly: left.readOnly || right.readOnly
  };
}
```

You must define the empty/unconstrained representation carefully so it is truly an identity. An empty set interpreted as “no permitted paths” is not an identity for intersection; use an explicit `allPaths` value or optional constraint.

Counterexample in the definition. A merge operation that is not associative makes nested contexts depend on parenthesisation. That breaks compositional reasoning. Test the monoid laws for metadata.

3A.5 Component and context metadata

A requirement can carry metadata per key:

```
export interface Required<T, M> {
  readonly key: Key<T>;
  readonly metadata: M;
}
```

A derived context can add enclosing metadata:

```
ctx.intercept(FileSystemKey, {
  mode: "read-only",
  allowedPrefix: "/workspace/plugin-a"
});
```

On access:

```
component metadata
  ⊕ context-carried metadata
  -> provider.resolve(merged metadata)
```

The paper gives enclosing context metadata priority where the per-key merge is right-biased. For security attenuation, choose a merge that cannot accidentally widen authority.

Worked example 3A.2: read-only database context

The root provider exposes a database session factory:

```
interface DbMetadata {
  readonly readOnly: boolean;
  readonly trace?: string;
}
```

A reporting component declares {readOnly: true}. An enclosing production context adds {trace: "nightly-report"}. The provider returns a session configured with both restrictions.

Interception changes how the binding is used, not whether the dependency key is present. Therefore updating tracing metadata need not deactivate consumers if the provider consults metadata on each access.

3A.6 Isolation versus interception

Mechanism	Changes	Typical use	Triggers dependency replacement?
provision	actual binding/store	publish a service	yes, when provider appears/disappears
isolation	key-to-realm resolution	tenant/test-local substitution	yes, if resolved provider changes
interception	metadata supplied on access	policy, tracing, timeout	not necessarily

3A.7 Security boundary

Interception can mediate capabilities only when component code cannot bypass the context and reach raw host objects. A malicious component running in the same unrestricted language runtime may access filesystem or network APIs directly.

Therefore:

- dependency declarations and interception are useful access-control mechanisms for cooperative code;
- untrusted code requires an external sandbox, process, VM, WebAssembly boundary, or other isolation mechanism;
- the bridge into the sandbox can itself expose attenuated effects.

3A.8 Executable laws

Realm isolation

```
it("resolves the same key differently in sibling contexts", () => {
  expect(tenantA.require(DatabaseKey)).toBe(databaseA);
  expect(tenantB.require(DatabaseKey)).toBe(databaseB);
  expect(root.require(DatabaseKey)).toBe(adminDatabase);
});
```

Parent preservation

```
it("deriving isolation does not mutate the parent", () => {
  const before = root.observeResolution(DatabaseKey);
  root.deriveIsolation(DatabaseKey, Symbol("test"));
  expect(root.observeResolution(DatabaseKey)).toEqual(before);
});
```

Metadata monoid

```
fc.assert(fc.property(metadataArb, (m) => {
  expect(merge(empty, m)).toEqual(m);
  expect(merge(m, empty)).toEqual(m);
}));

fc.assert(fc.property(metadataArb, metadataArb, metadataArb, (a, b, c) => {
  expect(merge(merge(a, b), c)).toEqual(merge(a, merge(b, c)));
}));
```

3A.9 Counterexample workshop

1. Mutate the root realm table instead of deriving a child. Show sibling contexts unexpectedly switching providers.
2. Use a non-associative metadata merge such as subtraction or “keep the middle value.” Show nesting order becoming observable.
3. Treat an empty permission set as an identity for intersection and expose the failed identity law.
4. Claim interception is a sandbox, then demonstrate direct access to an unmediated host API.
5. Change isolation but fail to refresh affected consumers. Show a committed view referring to the old provider indefinitely.

3A.10 Exercises and deliverable

1. Implement isolated realm resolution with parent inheritance.
2. Add `deriveIsolation(key, realm?)` and local provision.
3. Implement one metadata monoid and test identity/associativity.

4. Add `intercept(key, metadata)` to derived contexts.
5. Build a two-tenant example using the same component descriptions.
6. Build a read-only or traced service view using interception.
7. Document whether an interception update should trigger reload in your chosen provider API.
8. Submit two counterexamples: one broken realm implementation and one broken metadata monoid.

Reading

- Source paper: Section 3.2.3.
- Source paper: Sections 6.3 and 6.4 for access control, sandboxing, and language support.

Lab 4: Components, Fibers, and Operational Semantics

4.1 Purpose and outcomes

Lab 3 used a component description plus a Boolean active flag. That model cannot distinguish a service that remains present but changes provider, cannot represent a component halfway through a transition, and cannot attribute each activation's effects to a stable runtime identity. This lab introduces the paper's central runtime unit: the **fiber**.

After this lab, you should be able to:

- define a component as requirements, provisions, and a witnessed effect program;
- distinguish a component description from a live fiber;
- define a registry and provider identity;
- compute a target view from the whole runtime state;
- explain why a committed view records provider identities rather than values;
- implement the two-state base lifecycle;
- read and write small-step operational rules;
- define quiescence for the base runtime;
- translate between a transition rule and TypeScript code.

4.2 Motivation: availability is not identity

Suppose a component requires `DatabaseKey`. It activates against Database V1 and creates a prepared-statement cache tied to V1's connection pool. Later V1 is replaced with Database V2.

A Boolean model records:

```
before: database available = true
after:  database available = true
```

No transition is triggered. Yet the consumer is still wired to the old provider.

The runtime must remember not only that a key was satisfied, but **which provider satisfied it when the component activated**.

4.3 Component descriptions

Definition 4.1 (component). A component over context Γ is a triple:

$$C = (d, p, e)$$

where:

- d is the finite set of required coeffect keys;
- p is the finite set of keys the component may provide;
- e is a witnessed revertible effect program executed while the component is active.

A TypeScript description is:

```
export interface Component {
  readonly name: string;
  readonly requires: ReadonlySet<Key<unknown>>;
  readonly provides: ReadonlySet<Key<unknown>>;
  readonly activate: (
    ctx: FiberContext
  ) => void | Promise<void>;
}
```

The activation function need not return one disposer if every context mutation is already tracked by the fiber's scope. Its semantic result is still one composite accumulator.

Interface interpretation. The pair (d, p) is a two-sided component interface. Requirements describe what the component reads from the environment. Provisions describe what it may write into the shared service context.

4.3.1 Worked example 4.1: a command component

```
const Greeter: Component = {
  name: "greeter",
  requires: new Set([ClockKey]),
  provides: new Set(),
  activate(ctx) {
    const clock = ctx.require(ClockKey);
    ctx.command("hello", () => `Hello at ${clock.now().toISOString()}`);
  }
};
```

The component has no runtime identity yet. It is reusable data describing how an instance should behave.

4.4 Fibers: live component instances

Motivation. The same component description may be instantiated more than once, and each instance needs its own lifecycle state and accumulator.

Definition 4.2 (fiber). A fiber is a live instantiation of a component. It carries at least:

- a fresh identity;
- the component description;
- its lifecycle state;
- its component-local scope/accumulator;
- a committed dependency view;
- a retirement flag;
- optionally a parent fiber and child context.

```
export type FiberId = symbol;

export type BaseLifecycle =
  | Readonly<{ tag: "inactive" }>
  | Readonly<{
    tag: "active";
    committed: ProviderView;
    scope: DisposableScope;
  }>;

export interface Fiber {
  readonly id: FiberId;
  readonly component: Component;
  readonly parent?: FiberId;
  retired: boolean;
  state: BaseLifecycle;
}
```

The word *fiber* here does not mean an operating-system thread. It names an instantiated component carrying its own lifecycle.

4.4.1 Component versus fiber

Component	Fiber
reusable description	one live instance
no runtime identity	fresh <code>FiberId</code>
declares requirements/provisions	carries resolved providers and active effects
can be instantiated repeatedly	has one lifecycle history
analogous to a class/module	analogous to an object/process instance

4.5 The registry and provider relation

Definition 4.3 (registry). A runtime state carries a finite partial map from fiber identities to fibers:

$$F_\gamma : \mathcal{N} \rightharpoonup \mathcal{F}_\Gamma.$$

```
export class Registry {
  readonly fibers = new Map<FiberId, Fiber>();
}
```

The active fibers jointly determine the service context. In the simple single-provider model, each key is provided by at most one active fiber.

Definition 4.4 (provider). $\text{provider}(k, \text{gamma})$ is the identity of the active fiber whose current provisions contain key k .

A practical store entry can preserve both value and provider identity:

```
export interface Binding<T> {
  readonly key: Key<T>;
  readonly value: T;
  readonly provider: FiberId;
}
```

```
export class BindingStore {
  #bindings = new Map<symbol, Binding<unknown>>();

  get<T>(key: Key<T>): Binding<T> | undefined {
    return this.#bindings.get(key.id) as Binding<T> | undefined;
  }
}
```

The value supports ordinary service calls. The provider identity supports lifecycle coherence.

4.6 Provider views

Definition 4.5 (provider view). For a requirement d , a provider view is a total map from every required key to the identity of the fiber currently providing it:

$$\omega : d \rightarrow \mathcal{N}.$$

```
export type ProviderView = ReadonlyMap<Key<unknown>, FiberId>;
```


Definition 4.6 (target view). The target view of fiber n in state γ is:

$$\text{target}_n(\gamma) = \begin{cases} \perp, & \text{if } n \text{ is retired or a requirement is unresolved,} \\ (k \mapsto \text{provider}_k(\gamma)), & \text{otherwise.} \end{cases}$$

Here $\perp$ means that the fiber should not be running.

```
export type TargetView = ProviderView | undefined;

export function targetView(
  fiber: Fiber,
  bindings: BindingStore
): TargetView {
  if (fiber.retired) return undefined;

  const view = new Map<Key<unknown>, FiberId>();
  for (const key of fiber.component.requires) {
    const binding = bindings.get(key);
    if (binding === undefined) return undefined;
    view.set(key, binding.provider);
  }
  return view;
}
```

Definition 4.7 (committed view). When a fiber begins activation, it records the target view it is activating against. That recorded map is its committed view ω .

The committed view answers:

Which exact providers did this activation assume?

4.6.1 Worked example 4.2: replacement with equal values

Suppose both provider objects expose the same observable data:

```
const databaseV1 = { version: "compatible", query };
const databaseV2 = { version: "compatible", query };
```

Even if a structural comparison says they are equal, the provider views differ:

```
committed: DatabaseKey -> fiber-17
target:    DatabaseKey -> fiber-29
```

The mismatch triggers unload and reload. This is deliberate: identity captures lifecycle provenance that value equality does not.

4.7 Comparing views

Map identity is not enough. Define extensional equality:

```
export function equalViews(
  left: TargetView,
  right: TargetView
): boolean {
  if (left === undefined || right === undefined) {
    return left === right;
  }
  if (left.size !== right.size) return false;

  for (const [key, provider] of left) {
    if (right.get(key) !== provider) return false;
  }
  return true;
}
```

A change in an unrelated service leaves the view unchanged. A change in provider identity for a declared key changes the view even if the key remains satisfied.

4.8 The base lifecycle

The first lifecycle has only two states.

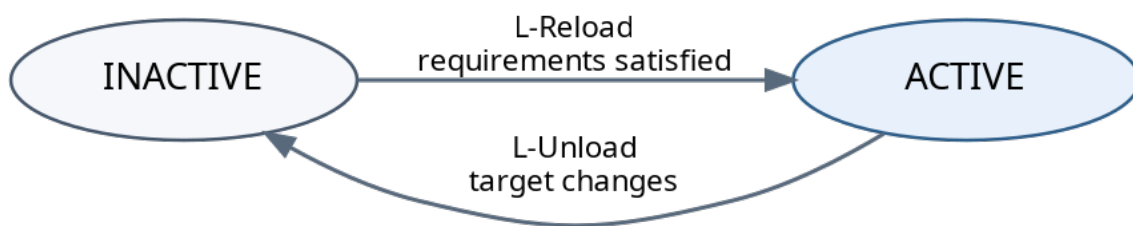


Figure 4.1: *The two-state base lifecycle.*

```
INACTIVE
|
| requirements satisfied and not retired
v
ACTIVE(committed view, accumulator)
|
| target differs from committed
```

v
INACTIVE

Definition 4.8 (reload condition). An inactive fiber may activate when its target view is not $\perp$.

Definition 4.9 (unload condition). An active fiber must deactivate when its current target differs from its committed view.

The conditions include both loss of satisfaction and provider replacement.

4.9 Operational semantics: rules as executable definitions

Operational semantics defines system behaviour using transition rules. A rule has:

- **premises** above a horizontal line;
- a **conclusion** below the line.

Read:

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{\gamma \longrightarrow \delta}$$

as:

If all premises P_i hold, the system may take a step from γ to δ .

The rules are not comments about an implementation. They define legal behaviour abstractly.

4.9.1 The reload rule

A simplified reload rule is:

$$\frac{\theta_n = \text{Inactive} \quad \omega = \text{target}_n(\gamma) \neq \perp \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g, \omega)]} \text{ L-RELOAD}$$

Read it line by line.

1. Fiber n is inactive.
2. Its target view exists and is called ω .
3. Its activation effect transforms the state to δ and yields accumulator g .
4. The conclusion stores $\text{Active}(g, \omega)$ on the fiber.

4.9.2 The unload rule

$$\frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}]} \text{ L-UNLOAD}$$

The committed view and accumulator appear together: a target mismatch decides *that* unloading is required, and the accumulator determines *how* the effects are withdrawn.

Fundamentals: small-step semantics. Each rule performs one conceptual system step. A complete run is a sequence of steps. Nondeterminism means several rules may be legal at once; the semantics permits any of them unless an ordering condition forbids it.

4.10 Translating rules into code

The two rules become:

```
async function stepFiber(runtime: Runtime, fiber: Fiber): Promise<boolean> {
  const target = targetView(fiber, runtime.bindings);

  if (fiber.state.tag === "inactive" && target !== undefined) {
    const scope = new DisposableScope();
    const ctx = runtime.contextFor(fiber, scope, target);

    await fiber.component.activate(ctx);
    fiber.state = {
      tag: "active",
      committed: target,
      scope
    };
    return true;
  }

  if (
    fiber.state.tag === "active" &&
    !equalViews(target, fiber.state.committed)
  ) {
    await fiber.state.scope.dispose();
    fiber.state = { tag: "inactive" };
    return true;
  }

  return false;
}
```

The return value reports whether a transition occurred.

A scheduler may repeatedly choose any fiber with a legal step:

```
async function settle(runtime: Runtime): Promise<void> {
  while (true) {
    let progressed = false;

    for (const fiber of runtime.registry.fibers.values()) {
      if (await stepFiber(runtime, fiber)) {
        progressed = true;
        break;
      }
    }

    if (!progressed) return;
  }
}
```

This deterministic loop chooses the first applicable fiber, but the semantics should not depend on that particular policy under the later confluence assumptions.

4.11 Orchestration versus lifecycle steps

The paper distinguishes external orchestration actions from automatic lifecycle reactions.

Orchestration actions express user or configuration input:

- insert a fiber;
- retire a fiber;
- remove a fully inactive retired fiber.

Lifecycle actions are reactive:

- reload when a target appears;
- unload when a committed target changes.

```
export class Runtime {
  insert(component: Component, parent?: FiberId): FiberId;
  retire(id: FiberId): void;
  remove(id: FiberId): void;
  step(): Promise<boolean>;
  settle(): Promise<void>;
}
```

4.11.1 Why retirement and removal are separate

If an active fiber is removed immediately, its accumulator disappears with it and its effects leak. Retirement is a request:

```
retire -> target becomes bottom -> lifecycle unloads -> remove
```

Removal is legal only after the fiber is inactive and any child constraints are satisfied.

4.11.2 Worked example 4.3: safe retirement

```
state 0: fiber A is ACTIVE and owns command "hello"
orchestrator: retire A
state 1: A remains present but target(A) = bottom
lifecycle: unload A; accumulator removes "hello"
state 2: A is INACTIVE and retired
orchestrator: remove A
state 3: A no longer occurs in registry
```

The control record outlives the effects long enough to recover them.

4.12 Quiescence

Motivation. A reactive runtime needs a definition of “settled.”

Definition 4.10 (quiescent base state). A base state is quiescent when every fiber agrees with its target:

- an inactive fiber has target $\perp$;
- an active fiber’s committed view equals its target view.

```
export function isQuiescent(runtime: Runtime): boolean {
  for (const fiber of runtime.registry.fibers.values()) {
    const target = targetView(fiber, runtime.bindings);

    if (fiber.state.tag === "inactive") {
      if (target !== undefined) return false;
    } else {
      if (!equalViews(target, fiber.state.committed)) return false;
    }
  }
  return true;
}
```

Quiescence does not mean that the application performs no ordinary work. It means no lifecycle transition is currently demanded by the component configuration and dependency context.

4.13 Confinement and attribution

The paper requires each fiber's effects to be attributable to that fiber. A simplified engineering rule is:

A component may mutate shared runtime state only through the `FiberContext` assigned to its fiber.

That context must:

- record effects in the fiber's scope;
- authorise only declared dependency reads;
- tag provisions with the current fiber identity;
- prevent a component from writing lifecycle control fields directly.

```
export class FiberContext {
  constructor(
    readonly fiber: Fiber,
    readonly committed: ProviderView,
    readonly scope: DisposableScope,
    readonly runtime: Runtime
  ) {}

  require<T>(key: Key<T>): T;
  provide<T>(key: Key<T>, value: T): void;
  command(name: string, handler: Handler): void;
  effect(acquire: () => Dispose | Promise<Dispose>): Promise<void>;
}
```

4.13.1 Reading through the committed view

A subtle but important design is that `require(key)` should resolve the provider recorded in `committed`, not simply whatever provider is globally current.

```
require<T>(key: Key<T>): T {
  const provider = this.committed.get(key as Key<unknown>);
  if (provider === undefined) {
    throw new Error(`undeclared dependency: ${key.description}`);
  }

  return this.runtime.bindingFromProvider(key, provider).value;
}
```

This guarantees that one activation does not silently switch providers halfway through its lifetime. In Lab 6, it also lets teardown continue using the old committed provider after that provider has stopped advertising itself to new consumers.

4.14 Worked trace: provider replacement

Initial active state:

```
Database V1 fiber d1:
  provides database

Repository fiber r:
  committed { database -> d1 }
```

The orchestrator retires d1 and inserts d2.

A possible base trace is:

1. d1 unloads and removes database
2. r sees target = bottom and unloads
3. d2 activates and provides database
4. r sees target {database -> d2} and reloads

Another schedule might activate d2 after d1 leaves but before r unloads. Then r sees a non-bottom target that differs from its committed view. It still unloads and reloads because provider identity changed.

4.15 Executable invariants

4.15.1 Unique provider

```
for (const key of allKeys) {
  const providers = activeProvidersOf(runtime, key);
  expect(providers.length).toBeLessThanOrEqual(1);
}
```

4.15.2 Active fibers have total committed views

```
for (const fiber of runtime.fibers()) {
  if (fiber.state.tag !== "active") continue;

  for (const key of fiber.component.requires) {
    expect(fiber.state.committed.has(key)).toBe(true);
  }
}
```


4.15.3 Committed providers exist

```
for (const [key, provider] of fiber.state.committed) {
  expect(runtime.registry.fibers.has(provider)).toBe(true);
  expect(runtime.providerOwns(provider, key)).toBe(true);
}
```

The base calculus cannot yet guarantee that every committed provider remains installed during a consumer's teardown, because teardown is atomic. Lab 6 refines this.

4.15.4 Quiescence after settling

```
await runtime.settle();
expect(isQuiescent(runtime)).toBe(true);
```

4.16 Counterexample workshop

4.16.1 Counterexample 4.1: compare dependency values instead of providers

Create two providers that return the same object or equal values. Replace one with the other. If the target digest uses value equality, the consumer fails to reload. Add a cache tied to the old provider to make the error visible.

4.16.2 Counterexample 4.2: remove before unload

Delete an active fiber's registry entry before running its scope disposer. Its commands or services remain, but no accumulator remains reachable. This is a permanent leak.

4.16.3 Counterexample 4.3: read globally rather than through committed view

A consumer activates against Database V1. Replace the global binding with V2 without reloading the consumer. A later call through `ctx.require(DatabaseKey)` suddenly reaches V2 even though the consumer's prepared resources belong to V1. This violates resolution coherence.

4.16.4 Counterexample 4.4: allow duplicate providers

If two active fibers provide the same key, `targetView` depends on map iteration or insertion order. A consumer's provider may change without any explicit policy. Either enforce uniqueness, add isolation realms, or introduce a broker that is itself the unique provider.

4.17 Exercises

4.17.1 Core implementation

1. Define `Component`, `FiberId`, `Fiber`, `Registry`, `Binding`, and `BindingStore`.
2. Generate fresh fiber identities that are never reused during one runtime execution.
3. Implement `targetView` and extensional `equalViews`.
4. Implement `insert`, `retire`, and `remove` orchestration operations.

5. Implement the two-state lifecycle and a settle loop.
6. Tag every provision with the providing fiber identity.
7. Make dependency access resolve through the committed view.
8. Enforce unique provision.

4.17.2 Operational-semantics exercises

9. Rewrite L-Reload and L-Unload in your own notation and define every metavariable.
10. Translate L-Reload into TypeScript without referring to the provided implementation.
11. Starting from your TypeScript code, write a rule for retire.
12. Explain the difference between a rule being *applicable* and the scheduler *choosing* it.
13. Draw the reachable state graph for one provider and one consumer in the base lifecycle.

4.17.3 Scenario exercises

14. Insert Database, Repository, and WebAPI in every order and compare the quiescent state.
15. Replace a provider with an equal-valued provider and verify that the consumer reloads.
16. Retire an active provider and show that it cannot be removed before unloading.
17. Instantiate the same consumer component twice. Verify that each fiber has a separate scope and committed view.
18. Add a child component registration effect and make parent disposal retire the child.

4.17.4 Counterexample exercises

19. Change target views to store only Booleans and produce a missed-replacement trace.
20. Change target views to store values and produce a missed-replacement trace.
21. Bypass the committed view during require and demonstrate a mid-episode provider switch.
22. Permit two providers for one key and show nondeterministic resolution.

4.18 Milestone B: dynamic composition

Your second milestone should demonstrate:

- typed requirements and provisions;
- fibers with stable identities;
- target and committed provider views;
- base reactive activation/deactivation;
- provider replacement detection;
- separate retirement and removal;
- per-fiber accumulators;
- a quiescence check;
- at least one rule/code translation exercise.

A strong demonstration loads a provider chain in reverse order, reaches the correct active graph, replaces the root provider, and shows only affected consumers reloading.

4.19 Reading guide

4.19.1 Required

- Source paper: Sections 4.1 and 4.2.
- Source paper: Table 2 in Section 5.1 for theory-to-implementation correspondence.

4.19.2 Programming-languages foundations

- Pierce, *Types and Programming Languages*: selected chapters introducing small-step operational semantics, evaluation rules, and preservation/progress style arguments.
- Harper, *Practical Foundations for Programming Languages*: use as a second account of structural operational semantics.

4.19.3 Applied comparison

- OSGi Declarative Services: component descriptions, service references, and lifecycle reaction.
- Vite HMR API: identify where acceptance and disposal remain framework- or module-authored.

4.19.4 Read with these questions

1. Why is a component description insufficient as the unit of lifecycle state?
2. What information does a committed view preserve that a service value does not?
3. Which parts of the runtime state should component code be unable to read or write directly?

Lab 5: Independence, Commutation, and Observational Equivalence

5.1 Purpose and outcomes

Labs 1-2 proved that a component can undo its own effects in reverse order. Lab 4 placed one accumulator on each fiber. The difficult question is now global:

If other fibers change the context between this fiber's application and recovery, does its accumulator still remove only its own contribution?

The answer requires a disciplined notion of independence. Literal state equality is also too strict for many real resources, so the independence laws must eventually be read up to observational equivalence.

After this lab, you should be able to:

- explain why local recovery does not imply safe selective withdrawal;
- define commutation of state transformations;
- identify the forward and inverse transformations an effect may perform;
- explain why independence is stronger than forward commutation;
- construct a commutation matrix for runtime primitives;
- define observational equivalence from allowed observations;
- explain why equivalence must be respected by effects and inverses;
- test recovery of one component against interleaved effects of others;
- produce minimal counterexamples when independence fails.

5.2 Motivation: one stack per component is not enough

Suppose two components use disjoint command names.

```
A registers command "alpha"  
B registers command "beta"
```

The runtime executes:

```
load A
load B
unload A
```

A's inverse can delete alpha while leaving beta. This is the behaviour we want.

Now suppose both components modify an ordered middleware chain.

```
A inserts middleware A
B inserts middleware B
```

The final order may be $[A, B]$ or $[B, A]$, and the behaviour of each middleware depends on its neighbours. Removing A from the middle may require relinking the chain and may change what B observes. The two effects are not independent merely because each has a cleanup function.

Local reversal says:

```
A immediately followed by undo(A) recovers A's input.
```

Global selective withdrawal asks:

```
A, then arbitrary independent foreign effects, then undo(A)
leaves exactly the foreign effects.
```

That is a stronger property.

5.3 Commutation

Definition 5.1 (commuting transformations). Two transformations $f, g : \Gamma \rightarrow \Gamma$ commute when

$$f \circ g \simeq g \circ f.$$

With equality as $\simeq$, applying them in either order gives the same state.

5.3.1 Worked example 5.1: disjoint keys commute

Let:

```
f = set key x to 10
g = set key y to 20
```

for distinct keys x and y . Starting from any state:

```
f after g: x=10, y=20
g after f: x=10, y=20
```

The operations commute because each reads and writes only its own key.

5.3.2 Worked example 5.2: increments on one counter commute

Let both transformations increment the same counter:

$$f(n) = n + 1, \quad g(n) = n + 1.$$

Then:

$$f(g(n)) = n + 2 = g(f(n)).$$

Shared location does not automatically imply non-commutation.

5.3.3 Counterexample 5.1: set and increment

Let:

$$f(n) = 10, \quad g(n) = n + 1.$$

Then:

$$f(g(3)) = 10, \quad g(f(3)) = 11.$$

They do not commute.

Design lesson. Independence is a property of operations and observations, not merely of memory locations. Disjoint writes are an easy sufficient condition, not the only one.

5.4 The transformations generated by an effect

A state-dependent effect does more than one fixed forward map. It has:

- its forward state transformation;
- every inverse it might return at different application states.

Definition 5.2 (transformation family of an effect). For an effect e , let $\mathcal{M}(e)$ be the monoid generated by:

- the forward transformation $\text{pr}_1 \circ e$;
- every inverse $\text{pr}_2(e(\gamma))$ yielded at any state γ ;
- arbitrary finite compositions of those transformations;
- the identity.

You do not need to enumerate this monoid in production code. It is a reasoning device that prevents us from checking only the pleasant forward path.

5.4.1 Why inverses must be included

Suppose two effects have commuting forward maps but one inverse overwrites data used by the other. They may load in either order but still fail during selective unloading.

5.4.2 Worked example 5.3: forward maps commute, inverses do not preserve contributions

State:

```
type S = { x: number; log: string[] };
```

Effect A increments x and captures the entire prior state as its inverse:

```
A forward: x := x + 1
A inverse: replace the whole state with the captured snapshot
```

Effect B appends to log and removes only its own log entry on inverse.

A's forward map and B's forward map commute because they touch different fields. But after A; B, applying A's snapshot inverse erases B's log entry. The inverse is locally witnessed and globally destructive.

The lesson is that an inverse should normally restore only the footprint its forward operation owns, not a larger snapshot.

5.5 Independence of effects

The paper's definition includes two ideas.

Definition 5.3 (independence, simplified). Effects e_1 and e_2 are independent when:

1. every transformation generated by one commutes with every transformation generated by the other, up to $\simeq$;
2. transformations of one do not change which inverse or continuation the other yields at a corresponding state.

Clause 1 protects state contributions. Clause 2 protects state-dependent control information.

5.5.1 Why clause 2 is needed

Consider an effect whose inverse depends on another field:

```
function conditionalEffect(state: State): [State, Endo<State>] {
  const undoMode = state.mode;
  // ...
}
```

A foreign effect changes `mode` before this effect is applied in a reordered execution. Even if the resulting forward states commute, the effect may now yield a different inverse. A proof that reorders execution needs the inverse and continuation to remain coherent.

Paper connection. Independence is intentionally stronger than the equation $e_1 \diamond e_2 = e_2 \diamond e_1$. The latter compares two whole applications. Independence must support moving a foreign forward transformation past a saved inverse and evaluating an effect at a state changed by another component.

5.6 Safe selective withdrawal

Suppose pairwise independent effects $e_1, \dots, e_n$ are applied in some order. Each yields an inverse g_i . Independence supports this property:

Applying any saved inverse removes the contribution of its own effect while retaining the others, and the remaining saved inverses remain valid.

For two effects:

```
initial γ
--A--> A(γ)
--B--> B(A(γ))
--undo A--> B(γ)
```

The equation behind the last step is:

$$g_A(B(A(\gamma))) \simeq B(g_A(A(\gamma))) \simeq B(\gamma).$$

The first equivalence moves g_A past B using commutation. The second uses A 's local witness.

5.6.1 Worked example 5.4: command registrations

State is a finite map from command names to handlers. Let A register `alpha`, and B register `beta`, with fresh distinct keys.

- A 's forward map inserts `alpha`.
- A 's inverse removes `alpha`.
- B 's forward map inserts `beta`.
- B 's inverse removes `beta`.

Every A transformation commutes with every B transformation because the keys are distinct. Either component may unload first.

5.6.2 Counterexample 5.2: same command key

If both register hello, fresh-key semantics rejects the second effect. Under overwrite semantics, the operations do not commute and an inverse may restore a stale provider. The simplest runtime avoids the problem through unique provision.

5.7 Operations at distinct service keys

The coeffect context gives an especially useful sufficient condition.

If an operation at key k reads and writes only the binding at k , and another operation acts only at $k' \neq k$, their lifted transformations commute. Their yielded inverses and outcomes also depend on disjoint bindings.

This is why decomposing shared state into well-scoped service keys is more than an organisational convention. It creates algebraic independence boundaries.

5.7.1 Worked example 5.5: a registration service

A CommandRegistry service owns a map of command names. Its operations are:

```
interface CommandRegistry {
  register(name: string, handler: Handler): Dispose;
  invoke(name: string, input: string): Promise<string>;
}
```

Two registrations of distinct command names can be independent within the same service if the service's equivalence ignores internal map ordering and the inverse removes only the named entry.

5.7.2 Counterexample 5.3: ordered middleware

A middleware service owns an ordered sequence:

```
interface MiddlewareChain {
  prepend(middleware: Middleware): Dispose;
  handle(request: Request): Promise<Response>;
}
```

Prepending A then B gives [B,A]; prepending B then A gives [A,B]. The handle operation can distinguish the order. The operations are not independent.

The runtime must impose order elsewhere:

- within one component, use LIFO recovery;
- across components, express an explicit dependency or provide an ordering-aware broker.

5.8 Building a commutation matrix

For every primitive your runtime exposes, classify its interaction with the others.

Operation A	Operation B	Expected relation	Reason
register command a	register command b	independent if $a \neq b$	disjoint entries
subscribe listener L1	subscribe listener L2	often independent	set/multiset semantics
prepend middleware A	prepend middleware B	not independent	order is observable
provide service db	provide service clock	independent	distinct keys
allocate timer	register command	independent inside chosen model	distinct resources
set global mode	conditional registration	generally not independent	mode changes yielded behaviour

A good matrix records assumptions. For example, event listeners are independent only if:

- listener order is not observable, or equivalence ignores it;
- removing one listener preserves duplicate registrations correctly;
- listener identifiers are stable;
- callbacks have not emitted irreversible output during registration.

Interface design question. Can the service be redesigned so its operations commute? A set-like registration interface is often more compositional than an ordered mutation interface.

5.9 Testing commutation

A helper can compare forward outcomes:

```
export function forwardCommutes<S>({
  f: Effect<S>,
  g: Effect<S>,
  initial: S,
  equivalent: (a: S, b: S) => boolean
}): boolean {
  const [afterG] = g(initial);
  const [afterFThenG] = f(afterG);

  const [afterF] = f(initial);
  const [afterGThenF] = g(afterF);

  return equivalent(afterFThenG, afterGThenF);
}
```

This is only the first clause and only for one state. A stronger test also checks saved inverses:

```
export function selectiveWithdrawalLaw<S>(
  a: Effect<S>,
  b: Effect<S>,
  initial: S,
  equivalent: (x: S, y: S) => boolean
): boolean {
  const [afterA, undoA] = a(initial);
  const [afterAB] = b(afterA);

  const [afterB] = b(initial);
  const afterRemoveA = undoA(afterAB);

  return equivalent(afterRemoveA, afterB);
}
```

And symmetrically for removing B.

5.9.1 Property test over disjoint keys

```
it("effects on distinct keys support selective withdrawal", () => {
  fc.assert(
    fc.property(
      mapArbitrary,
      fc.string(), fc.integer(),
      fc.string(), fc.integer(),
      (initial, k1, v1, k2, v2) => {
        fc.pre(k1 !== k2);

        expect(
          selectiveWithdrawalLaw(
            setValue(k1, v1),
            setValue(k2, v2),
            initial,
            mapsEqual
          )
        ).toBe(true);
      }
    )
  );
});
```

5.9.2 Shrinking a failure

Remove `fc.pre(k1 != k2)`. The property should find a same-key counterexample. The small counterexample explains the assumption more effectively than a paragraph saying “distinctness is required.”

5.10 Why literal equality is too strong

Imagine a resource allocator:

```
initial nextHandle = 100
allocate -> returns handle 100
free handle 100
final nextHandle = 101
```

The physical state has not returned to its exact representation. Yet no live allocation remains. If handle numbers are not otherwise observable or reusable according to the interface, the application may reasonably consider the states equivalent.

Other examples include:

- heap layouts after allocation and free;
- fresh fiber identities;
- hash-map bucket layouts;
- cache contents that do not affect allowed outcomes;
- timestamps recorded only for diagnostics;
- generated temporary paths after cleanup.

A theory demanding literal equality would reject many practically recovered systems.

5.11 Observers and observational equivalence

Definition 5.4 (observer). An observer is any allowed program built from the operations exposed by the context interface, together with its observable outcomes.

Definition 5.5 (observational equivalence). States γ and γ' are observationally equivalent, written

$$\gamma \simeq \gamma',$$

when no allowed observer can distinguish them.

This definition depends on the interface. If raw allocation counters are exposed, differing counters are observable. If only live resource operations are exposed, they may not be.

5.11.1 Finite executable approximation

A real observational equivalence quantifies over all allowed tests. In a lab model, define a canonical observation:

```
export interface ObservableContext {
  readonly commands: readonly string[];
  readonly services: readonly string[];
  readonly activeFibers: readonly string[];
}

export function observe(runtime: Runtime): ObservableContext {
  return {
    commands: runtime.commandNames().sort(),
    services: runtime.serviceDescriptions().sort(),
    activeFibers: runtime.activeFiberNames().sort()
  };
}

export function equivalent(a: Runtime, b: Runtime): boolean {
  return deepEqual(observe(a), observe(b));
}
```

This is a chosen approximation, not automatically the coarsest valid equivalence.

Fundamentals: quotient intuition. Treating states up to $\simeq$ means grouping indistinguishable representations into equivalence classes. Programs should then act consistently on classes rather than depending on hidden representatives.

5.12 An equivalence must be compatible with operations

A proposed equivalence is unsound if an allowed operation can distinguish related states.

Definition 5.6 (respecting equivalence). A function $f : \Gamma \rightarrow \Gamma$ respects $\simeq$ when:

$$\gamma \simeq \gamma' \implies f(\gamma) \simeq f(\gamma').$$

For an effect function, we need more:

- equivalent inputs produce equivalent successor states;
- yielded inverses are equivalent as functions on relevant states;
- each yielded inverse respects equivalence;
- observable outcomes agree.

5.12.1 Counterexample 5.4: an equivalence that ignores command handlers

Suppose observe records only command names, not command behaviour. Two states both contain `hello`, but one handler returns `"safe"` and the other returns `"compromised"`. The allowed `invoke("hello")` operation distinguishes them. Therefore the proposed equivalence is too coarse.

5.12.2 Worked example 5.6: map order

Two command maps have the same name-to-handler bindings but different insertion order. If the API exposes only lookup by name, insertion order is unobservable and may be quotiented away. If the API exposes `listCommands()` in insertion order, the relation must preserve that order or change the API's specified observation.

5.13 Observational equivalence per service key

The paper assembles context equivalence from equivalences on each coefficient key. This gives service providers control over what counts as observable for their interface.

For a key k , choose:

- value type $\mathcal{V}_k$;
- operations $\mathcal{A}_k$;
- equivalence $\simeq_k$ respected by those operations.

Then two service contexts are equivalent when they:

1. bind the same keys;
2. bind $\simeq_k$ -related values at every key.

5.13.1 Worked example 5.7: a set-valued registry

A listener registry might use an array internally but expose only:

```
register(listener) : Dispose
emit(event) : void
```

If listener invocation order is unspecified, arrays differing only by permutation may be equivalent. If invocation order is specified, they are not.

The interface determines the law.

5.14 Independence up to observation

Two operations may leave physically different states while remaining observationally commuting:

$$f \circ g \simeq g \circ f.$$

5.14.1 Worked example 5.8: allocation up to renaming

Allocate two opaque handles A and B.

```
order 1: A gets 100, B gets 101
order 2: B gets 100, A gets 101
```

The states are not literally equal. If handles are opaque and observers may only use each returned handle with its own resource, the two orders may be equivalent up to a consistent renaming. If callers compare handle numbers, the order becomes observable and allocation is not commutative under that interface.

Category theory connection. Quotienting by observational equivalence is what makes structural laws apply to behaviour rather than representation. The important practical question is not “can I invent a coarse relation?” but “do all permitted operations respect it?”

5.15 A whole-fiber recovery experiment

Build two components with independent effects:

```
A: register command alpha; provide service a
B: register command beta; provide service b
```

Run:

```
insert A
insert B
settle
retire A
settle
```

Compare the result to a fresh runtime that inserted only B:

```
expect(observe(runtimeAfterRemovingA)).toEqual(
  observe(runtimeWithOnlyB)
);
```

This is the executable intuition behind recovery exactness:

A fiber’s accumulator removes its own contribution and leaves the state produced by other fibers.

Repeat with three components and every retirement order.

5.16 Trace equivalence

Independent actions support a useful view of concurrent histories. If actions a and b are independent, adjacent occurrences may be swapped:

$$xaby \sim xbay.$$

Repeated swaps generate an equivalence class of traces. For example:

$$abc \sim bac$$

when a and b are independent.

This is the basic intuition of Mazurkiewicz trace theory: schedules that differ only by the order of independent actions represent the same causal behaviour.

Mini-Cordis will use the same idea in Lab 8. If lifecycle steps of independent fibers can be transposed without changing the endpoint, many nondeterministic schedules converge to one normal form.

5.17 Counterexample workshop

5.17.1 Counterexample 5.5: snapshot inverse

Implement an effect whose inverse restores an entire context snapshot. Compose it with any foreign effect. The forward operations may touch disjoint fields, but selective withdrawal erases the foreign effect.

5.17.2 Counterexample 5.6: state-dependent inverse changes

Design an effect that chooses its inverse based on mode. Let a foreign effect change mode. Compare:

apply foreign, then apply local

with a reordered schedule. Show that the local effect yields different recovery behaviour.

5.17.3 Counterexample 5.7: overly coarse observation

Ignore handler behaviour and compare only command names. Construct two states that your equivalence relates but `invoke` distinguishes.

5.17.4 Counterexample 5.8: middleware order

Insert two middleware components in opposite orders. Send a request and record the trace. The differing outputs witness non-commutation.

5.18 Exercises

5.18.1 Core implementation

1. Implement `forwardCommutes` and `selectiveWithdrawalLaw`.
2. Add the symmetric law for removing the second effect.
3. Define an `Equivalence<S>` interface and use it in all laws.
4. Implement a canonical observe function for your runtime.
5. Make command and service maps compare extensionally rather than by object identity.
6. Add whole-fiber recovery tests against fresh runtimes containing only the remaining components.

5.18.2 Commutation-matrix exercises

7. Classify every primitive in your runtime against every other primitive.
8. State the preconditions for each positive classification.
9. Identify one primitive whose interface could be redesigned to improve commutativity.
10. Add tests for at least three positive and three negative cells.

5.18.3 Mathematical exercises

11. Prove the two-effect selective withdrawal equation using commutation and the witness law.
12. Extend the argument informally to a sequence of pairwise independent effects.
13. Explain why forward commutation alone does not constrain saved inverses.
14. Show that functions respecting an equivalence are closed under composition.
15. Show that equality is always an admissible equivalence, though possibly too fine.
16. Define a candidate equivalence for a listener registry and list the operations that must respect it.

5.18.4 Counterexample exercises

17. Implement the snapshot-inverse failure.
18. Implement an order-sensitive middleware chain.
19. Define an equivalence that ignores too much, then write a distinguishing test.
20. Remove the distinct-key precondition from a property test and inspect the shrunk failure.
21. Create an effect whose yielded inverse depends on foreign state and show a schedule-sensitive failure.

5.19 Milestone C, part 1: safe interleaving

At this checkpoint, demonstrate:

- a documented observational equivalence;
- a commutation matrix;
- selective withdrawal of independent components;

- at least two non-independence counterexamples;
- a property test with a meaningful shrunk failure;
- a trace-equivalence explanation for two schedules.

Do not claim a primitive is independent unless your explanation includes forward maps, inverses, state-dependent yields, and observable outcomes where applicable.

5.20 Reading guide

5.20.1 Required

- Source paper: Section 3.1.3 on independence.
- Source paper: Section 3.3.2 on observational equivalence.
- Source paper: Theorems 40 and 42 on distinct keys and coeffect-mediated effects.

5.20.2 Applied comparison

- Study one registration-style API and one ordered-chain API. Explain why the former is usually easier to make commutative.

5.20.3 Deepening

- Heunen, Kaarsgaard, and Karvonen, *Reversible Effects as Inverse Arrows*.
- An introductory account of Mazurkiewicz traces: focus on equivalence generated by swapping independent adjacent actions.
- Fong and Spivak, *Seven Sketches in Compositionality*: read for the broader practice of identifying interfaces and compositional laws.

5.20.4 Read with these questions

1. Why must an independence definition mention inverses as well as forward maps?
2. Who decides what counts as observable at a service key?
3. How can a more carefully designed interface make more operations commute?

Lab 6: Iteration, Asynchrony, Failure, and Safe Withdrawal

6.1 Purpose and outcomes

The base lifecycle treats activation and deactivation as atomic, immediate, and infallible. Real components perform several operations, await external work, discover target changes mid-transition, and sometimes fail. Providers must also wait for consumers to finish tear-down before destroying dependencies those consumers still need.

After this lab, you should be able to:

- refine the lifecycle with `LOADING` and `UNLOADING` states;
- model activation as an effect iterator;
- explain rollback boundaries and LIFO accumulation across iterations;
- implement target-change diversion;
- explain inertia for in-flight asynchronous work;
- recover completed effects after activation failure;
- separate a provider's visibility from physical withdrawal;
- define and implement the `relied` guard;
- preserve committed dependency access through consumer teardown;
- reason about cascaded deactivation in a provider graph.

6.2 Motivation: transitions take time

A realistic component may activate as follows:

1. `register` a diagnostic command
2. `connect` to a database
3. `run` a schema check
4. `start` an HTTP server
5. `publish` an API service

While step 3 is awaiting I/O:

- the database provider may be replaced;
- the component may be retired;
- a timeout may occur;

- another activation may fail;
- the parent component may unload.

The runtime needs an explicit state representing a transition in progress. Otherwise there is nowhere to store:

- the inverses accumulated so far;
- the dependency view the transition committed to;
- the continuation or remaining work;
- the in-flight promise;
- a failure outcome.

6.3 The four-state lifecycle

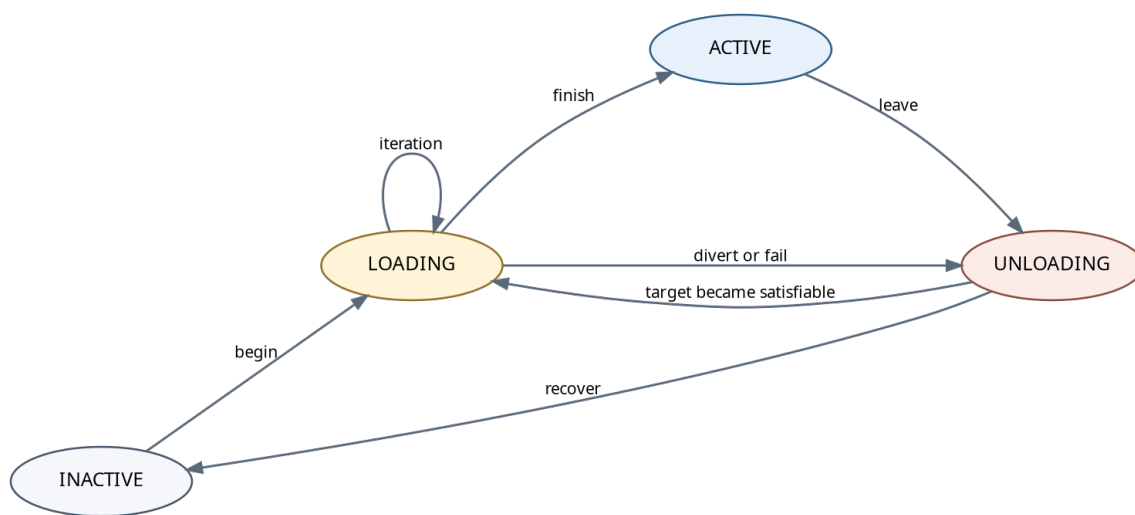
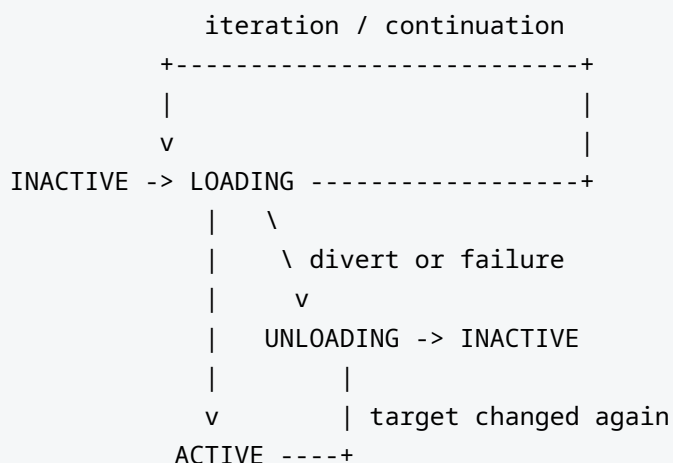


Figure 6.1: *A lifecycle with transitions in progress.*



Definition 6.1 (extended lifecycle). A fiber is in one of four states:

```

export type Lifecycle =
  | Readonly<{
    tag: "inactive";
    error?: unknown;
  }>
  | Readonly<{
    tag: "loading";
    committed: ProviderView;
    scope: DisposableScope;
    transition: Promise<void>;
  }>
  | Readonly<{
    tag: "active";
    committed: ProviderView;
    scope: DisposableScope;
  }>
  | Readonly<{
    tag: "unloading";
    committed: ProviderView;
    scope: DisposableScope;
    transition: Promise<void>;
    outcome?: unknown;
  }>;

```

The paper carries the remaining iterator and accumulator explicitly in the lifecycle state. A practical implementation may store them inside a task object or scope, provided the same information is available.

Definition 6.2 (installed fiber). A fiber is **installed** while it carries an accumulator and committed view: loading, active, or unloading.

An inactive fiber has no installed contribution.

6.4 Multi-step effects and iterators

Motivation. A component should be interruptible between atomic acquisitions, and the runtime should recover exactly the successful prefix.

Definition 6.3 (effect iterator, informal). An effect iterator repeatedly produces:

- a context change;
- an inverse for that change;
- either a continuation or termination.

A TypeScript async generator is a direct implementation tool:

```
export type Acquire = () => Dispose | Promise<Dispose>;

export type EffectIterator = AsyncGenerator<Acquire, void, void>;
```

A component activation may be written:

```
async function* activateApi(ctx: FiberContext): EffectIterator {
  yield () => ctx.acquireCommand("status", statusHandler);
  yield () => ctx.acquireDatabaseConnection();
  yield () => ctx.acquireHttpServer();
  yield () => ctx.acquireService(ApiKey, makeApi());
}
```

Each yielded Acquire is one iteration boundary.

Implementation choice. You may instead make the generator yield disposal closures after performing each acquisition. The key semantic requirement is that the runtime receives one inverse immediately after each successful atomic step.

6.4.1 A runner

```
export async function execute(
  iterator: EffectIterator,
  guard: () => boolean,
  scope: DisposableScope
): Promise<"finished" | "diverted"> {
  while (guard()) {
    const next = await iterator.next();
    if (next.done) return "finished";

    const inverse = await next.value();
    scope.add(inverse);
  }

  return "diverted";
}
```

This design checks the guard before launching each new iteration.

6.4.2 Worked example 6.1: interruption at a boundary

iteration 1: register command	succeeds; inverse recorded
iteration 2: subscribe event	succeeds; inverse recorded
dependency target changes	

```
guard fails before iteration 3
runtime disposes scope
```

Recovery runs:

```
unsubscribe event
unregister command
```

Iteration 3 never began, so there is nothing from it to undo.

6.5 Accumulation across iterations

If iterations yield inverses $g_1, g_2, \dots, g_k$, the accumulator is:

$$g_1 \circ g_2 \circ \dots \circ g_k.$$

Applying it runs g_k first. The soundness invariant is maintained after every successful iteration.

6.5.1 Worked example 6.2: partial activation

```
initial context  $\gamma_0$ 
  --E1-->  $\gamma_1$ , save  $g_1$ 
  --E2-->  $\gamma_2$ , save  $g_2$ 
  --E3 fails before producing a successor/inverse
```

The scope contains $g_1 \circ g_2$. Applying it to γ_2 recovers γ_0 . The failed iteration contributes nothing because it never committed a successful effect boundary.

If an external API can partially mutate and then throw before returning an inverse, it is not atomic at the abstraction boundary. Wrap it in a smaller transaction or arrange for its own exception path to restore partial work.

6.6 Beginning and finishing activation

A fiber begins loading only from a non-failed inactive state whose target exists.

```
async function beginLoad(runtime: Runtime, fiber: Fiber): Promise<void> {
  const target = targetView(fiber, runtime.bindings);
  if (fiber.state.tag !== "inactive" || target === undefined) return;
  if (fiber.state.error !== undefined) return;

  const scope = new DisposableScope();
  const transition = reload(runtime, fiber, target, scope);

  fiber.state = {
```

```

    tag: "loading",
    committed: target,
    scope,
    transition
  };

  await transition;
}

```

The reload task records the target at its start:

```

async function reload(
  runtime: Runtime,
  fiber: Fiber,
  committed: ProviderView,
  scope: DisposableScope
): Promise<void> {
  const ctx = runtime.contextFor(fiber, scope, committed);
  const iterator = fiber.component.activate(ctx);

  try {
    const result = await execute(
      iterator,
      () => equalViews(targetView(fiber, runtime.bindings), committed),
      scope
    );

    if (
      result === "finished" &&
      equalViews(targetView(fiber, runtime.bindings), committed)
    ) {
      fiber.state = { tag: "active", committed, scope };
      runtime.notifyProvidedBy(fiber);
    } else {
      startUnload(runtime, fiber);
    }
  } catch (error) {
    startUnload(runtime, fiber, error);
  }
}

```

The exact code will need careful control of concurrent state assignment. Treat this as pseudocode identifying the state transitions, not a drop-in race-free implementation.

6.7 Target-change diversion

Definition 6.4 (diversion). If the target changes while a fiber is loading, the runtime stops launching new iterations and routes the fiber into unloading with the accumulator built so far.

The target may change because:

- a required provider disappears;
- a provider is replaced;
- the fiber is retired;
- isolation changes resolution.

All cases are handled by comparing current target with committed view.

6.7.1 Worked example 6.3: replacement during loading

```
Repository begins loading against db provider d1.  
It registers one command.  
Database d1 is replaced by d2.  
Repository's target becomes {db -> d2}, not its committed {db -> d1}.  
The next guard check diverts.  
Repository unregisters the command.  
Repository becomes inactive.  
It may then begin loading against d2.
```

One activation never straddles two provider resolutions.

6.8 Asynchrony and inertia

Checking a guard before each iteration does not make an in-flight iteration cancellable.

```
launch connectToPeer() against provider d1  
provider changes to d2 while promise is pending  
connectToPeer() eventually resolves
```

The iteration has already been submitted. Unless the underlying API supports safe cancellation with a clear rollback contract, its result must land.

Definition 6.5 (inertia). Once an asynchronous iteration has begun, it runs to a result. If the target changes during the flight, the result is recorded and then immediately recovered during unloading.

6.8.1 Worked example 6.4: landing after staleness

1. target = view v1; launch E
2. target changes to v2
3. E resolves with inverse g
4. record g in scope
5. enter UNLOADING
6. apply g and earlier inverses
7. optionally reload against v2

Dropping the result would leak the effect because its inverse would never be recorded. Pretending the fiber became active would expose provisions from a transition already known to be stale.

Cancellation note. Cancellation is itself a protocol. It is safe only if the operation guarantees one of: no effect occurred, an inverse is still delivered, or a compensating recovery path runs. A rejected promise alone does not prove atomic cancellation.

6.9 Failure and rollback

Definition 6.6 (activation failure). An iteration may fail instead of yielding its next inverse. The runtime must recover the successful prefix and record the failure on the fiber.

A failure transition is:

```
LOADING(scope, committed)
  --iteration throws error ξ-->
UNLOADING(scope, committed, outcome ξ)
  --dispose scope-->
INACTIVE(error ξ)
```

The failed component contributes no lasting context effects after recovery.

6.9.1 Why failed fibers do not automatically retry

If activation failed against an unchanged environment, immediate retry may loop forever. The reference design records the failure and withholds the fiber until an orchestrator clears or replaces it.

A practical API can expose:

```
runtime.retry(fiberId)
runtime.replace(fiberId, newComponent)
runtime.clearFailure(fiberId)
```

Each is an explicit orchestration decision.

6.9.2 Worked example 6.5: port conflict

E1 registers command "status"	succeeds
E2 subscribes health event	succeeds
E3 binds TCP port 8080	fails: address in use

The fiber moves to unloading. It unsubscribes the health event and unregisters `status`, then becomes inactive with the port error recorded.

6.10 Why provider withdrawal is harder than notification

Suppose Repository depends on Database. Repository's teardown does:

```
await db.flushPendingWrites();
```

A naive provider unload sequence is:

```
delete database binding
notify Repository
Repository teardown tries to use database -> failure
```

Detection is not enough. The consumer must retain its committed access while tearing down, and the provider must defer physical recovery until dependents finish.

6.11 Visibility versus physical presence

The key design is to separate two moments.

1. **Leave:** the provider stops being available to *new target resolution*.
2. **Unload:** after dependents have left, the provider applies its accumulator and physically removes its bindings/resources.

A provider in UNLOADING is therefore:

- absent from the global active-provider view;
- still reachable through committed views held by installed dependents;
- still physically intact until the guard releases.

This separation creates the interval in which consumers can tear down safely.

6.12 The relied guard

Definition 6.7 (relied). A provider fiber n is relied upon when some other installed fiber's committed view names n for one of its required keys.

```
export function reliedUpon(runtime: Runtime, provider: Fiber): boolean {
  for (const consumer of runtime.fibers()) {
    if (consumer.id === provider.id) continue;
    if (!isInstalled(consumer.state)) continue;

    for (const requiredProvider of consumer.state.committed.values()) {
      if (requiredProvider === provider.id) return true;
    }
  }
  return false;
}
```

Unload guard: a provider may apply its accumulator only when `reliedUpon` is false.

```
async function unload(runtime: Runtime, fiber: Fiber): Promise<void> {
  await runtime.waitUntil(() => !reliedUpon(runtime, fiber));
  await fiber.state.scope.dispose();

  const outcome =
    fiber.state.tag === "unloading" ? fiber.state.outcome : undefined;

  fiber.state = outcome === undefined
    ? { tag: "inactive" }
    : { tag: "inactive", error: outcome };
}
```

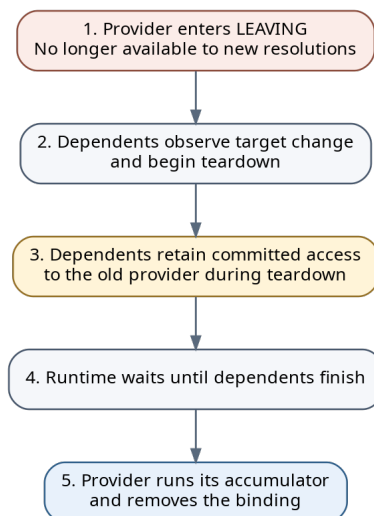


Figure 6.2: Safe withdrawal separates leaving from final recovery.

1. Provider stops advertising the binding.
2. Consumers' target views change.
3. Consumers unload using their committed provider references.
4. Provider waits until no installed consumer names it.
5. Provider runs its accumulator and destroys the binding.

6.13 Why the guard should not deadlock in an acyclic graph

Once provider P leaves, P is no longer active for target computation. Every consumer committed to P now has a mismatching target and must enter unloading. If a consumer is itself a provider, its consumers leave first. In an acyclic dependency graph, this cascade proceeds from leaves back toward roots.

```
Database <- Repository <- WebAPI
```

Withdrawal order is:

```
WebAPI, then Repository, then Database
```

The paper proves progress under additional finiteness and bounded-iteration assumptions. Lab 7 turns the argument into state exploration.

6.13.1 Counterexample 6.1: cyclic dependency

If A relies on B and B relies on A while both are installed through some external bootstrap, guarded unload can wait in a cycle. The formal progress theorem excludes cycles in the precedence relation.

6.14 Committed access during teardown

`require(key)` must consult the fiber's committed view even after the provider has left the global active set.

```
require<T>(key: Key<T>): T {
  const provider = this.committed.get(key as Key<unknown>);
  if (provider === undefined) {
    throw new Error(`undeclared dependency: ${key.description}`);
  }

  const binding = this.runtime.bindingFromProvider(key, provider);
  if (binding === undefined) {
    throw new Error(
      `runtime invariant broken: committed provider vanished early`
    );
  }
}
```

```
    return binding.value;
  }
```

The guard ensures the committed provider's binding remains physically available until the consumer's scope has recovered.

6.14.1 Worked example 6.6: connection pool teardown

```
Database provides pool.
Repository borrows connections and registers a drain hook.
Database begins leaving.
Repository begins unloading.
Repository returns borrowed connections through committed pool reference.
Repository finishes and drops committed view.
Database guard releases.
Database closes pool.
```

Without committed access, Repository might resolve a replacement pool halfway through teardown and return old connections to the wrong provider.

6.15 Notification ordering

A provider should be marked UNLOADING before its recovery task is scheduled. That state change removes it from target resolution immediately, causing dependents to begin leaving before any provider inverse runs.

Pseudocode:

```
function startUnload(runtime: Runtime, fiber: Fiber, outcome?: unknown): void {
  if (!isInstalled(fiber.state)) return;

  const { committed, scope } = fiber.state;
  const task = runtime.createPausedTask(() => unload(runtime, fiber));

  // Publish departure while the recovery task is still unable to run.
  fiber.state = {
    tag: "unloading",
    committed,
    scope,
    transition: task.promise,
    outcome
  };

  runtime.notifyProvidedBy(fiber);
}
```

```
task.start();
}
```

`createPausedTask` is a small scheduler abstraction used here only to make the ordering explicit: constructing the task does not execute its thunk, and `start()` may be called once. A production implementation may obtain the same semantics with an event-loop queue, an actor mailbox, or an explicit transition scheduler. The required conceptual order is:

```
publish UNLOADING state
notify dependents
then allow provider recovery to proceed subject to guard
```

6.16 Chaining transitions

A target can change again during unloading.

- If the target remains $\perp$, recovery ends in `INACTIVE`.
- If a valid target appears, recovery completes first, then a fresh load begins.

Similarly, a stale loading transition lands and unloads before reloading against the new target.

This **inertial chaining** prevents two activation programs or an activation and deactivation from running concurrently on the same fiber.

```
LOADING(v1) --target becomes v2--> UNLOADING(v1)
UNLOADING(v1) --recover--> LOADING(v2)
```

The runtime always crosses the clean inactive boundary semantically, even if implementation code chains tasks directly.

6.17 Executable invariants

6.17.1 Installed consumers name installed providers

```
for (const consumer of runtime.fibers()) {
  if (!isInstalled(consumer.state)) continue;

  for (const providerId of consumer.state.committed.values()) {
    const provider = runtime.getFiber(providerId);
    expect(provider).toBeDefined();
    expect(isInstalled(provider!.state)).toBe(true);
  }
}
```

6.17.2 No active provider is advertised while unloading

```
for (const fiber of runtime.fibers()) {  
  if (fiber.state.tag === "unloading") {  
    for (const key of fiber.component.provides) {  
      expect(runtime.currentProvider(key)).not.toBe(fiber.id);  
    }  
  }  
}
```

6.17.3 Committed binding remains accessible

During every consumer teardown step:

```
for (const [key, provider] of consumer.state.committed) {  
  expect(runtime.bindingFromProvider(key, provider)).toBeDefined();  
}
```

6.17.4 Prefix rollback on failure

Record an event trace:

```
acquire A  
acquire B  
fail C  
release B  
release A
```

Assert no provision or command remains after the fiber reaches failed inactive state.

6.17.5 No overlapping transition per fiber

```
expect(fiber.transitionCountInFlight).toBeLessThanOrEqual(1);
```

6.18 Counterexample workshop

6.18.1 Counterexample 6.2: delete binding before notifying

Remove the provider's service binding first, then notify. A consumer's teardown tries to use its dependency and fails. Record the exact trace.

6.18.2 Counterexample 6.3: resolve globally during teardown

Keep the old provider physically alive but make require use the current global provider. Replace the provider while the consumer unloads. Show teardown crossing from V1 to V2.

6.18.3 Counterexample 6.4: ignore an in-flight result

Launch an asynchronous acquisition, change the target, and discard the eventual disposer. The resource leaks because the successful effect landed without entering the accumulator.

6.18.4 Counterexample 6.5: retry failure automatically

Make a deterministic port-binding failure. Automatic retry produces an infinite transition loop and prevents quiescence.

6.18.5 Counterexample 6.6: wait on all declared consumers rather than committed consumers

A component may declare a key but currently resolve it in another isolation realm, or may be inactive. Waiting on declarations alone is too coarse. The guard should inspect installed committed views.

6.19 Exercises

6.19.1 Core implementation

1. Replace the base lifecycle with the four-state discriminated union.
2. Implement an async-generator activation protocol.
3. Accumulate one inverse after each successful iteration.
4. Add a target-stability guard between iterations.
5. Implement diversion into unloading.
6. Implement failure rollback and a failed inactive state.
7. Implement at-most-one transition per fiber.
8. Implement leave-before-unload visibility.
9. Implement `reliedUpon` from committed views.
10. Delay provider recovery until the guard releases.
11. Preserve committed dependency access throughout consumer teardown.
12. Chain unloading back into loading when a new target appears.

6.19.2 Scenario exercises

13. Interrupt a three-step activation after step two and verify prefix rollback.
14. Change the target while step three is in flight. Let it land, then verify complete rollback.
15. Fail the third step and verify the same context recovery as diversion, differing only in fiber outcome.
16. Build Database -> Repository -> WebAPI and log exact withdrawal order.
17. Make Repository teardown call Database. Verify success under the guard and failure without it.
18. Replace Database while Repository is loading and verify one activation never mixes provider views.
19. Make a target disappear and reappear while a fiber is unloading. Verify unload completes before reload.

6.19.3 Mathematical exercises

20. State the lifecycle invariant relating installed consumers to installed providers.
21. Explain why marking a provider UNLOADING makes every committed consumer's target differ.
22. Give an informal induction argument that guarded withdrawal releases on a finite acyclic provider graph.
23. Explain why inertia is a restriction on which transitions may be declined rather than a new state.
24. Relate an async generator continuation to the paper's effect iterator continuation.
25. Explain why the failure state affects future activation but not recovered context state.

6.19.4 Counterexample exercises

26. Remove the guard and demonstrate premature dependency destruction.
27. Remove committed access and demonstrate provider switching during teardown.
28. Discard an in-flight inverse and demonstrate a leak.
29. Permit overlapping reload/unload tasks and produce a double-registration or double-disposal trace.
30. Enable automatic failure retry and demonstrate non-termination.

6.20 Milestone C, part 2: robust lifecycle

A complete Milestone C demonstration includes:

- observational equivalence and independence tests from Lab 5;
- four lifecycle states;
- multi-step asynchronous activation;
- prefix rollback on diversion and failure;
- inertia for in-flight work;
- safe dependency teardown using committed views;
- guarded provider recovery;
- a trace showing a dependency chain unloading in consumer-first order;
- counterexamples with the guard and committed access disabled.

6.21 Reading guide

6.21.1 Required

- Source paper: Sections 4.3.1-4.3.4.
- Source paper: Section 5.1.3 and Algorithm 5.

6.21.2 Applied comparison

- React documentation on the lifecycle of reactive effects: compare setup/cleanup ordering but note the different component model and restrictions.
- OSGi service-component lifecycle: compare dynamic service availability and teardown.

- Vite HMR: compare replacement and acceptance boundaries with component-scoped recovery.

6.21.3 Deepening

- Source paper: Theorem 63 on ordering and Theorem 64 on resolution coherence.
- Rollback-recovery and saga literature for effects outside the exact-recovery boundary.

6.21.4 Read with these questions

1. Why must a provider become invisible before its resources are destroyed?
2. Why must a consumer retain an old committed view rather than re-resolve during teardown?
3. What must happen to an asynchronous effect that finishes after its transition becomes stale?

Lab 7: Turning Metatheory into a Model Checker

7.1 Purpose and outcomes

The paper’s metatheory is not decorative. It states what the runtime promises despite non-deterministic interleavings. Reproducing every proof is not the best first learning exercise. Instead, this lab builds a finite model checker that treats rules as executable transitions, explores schedules, checks invariants after every step, and returns small counterexamples when assumptions are disabled.

After this lab, you should be able to:

- distinguish a semantic theorem from an example test;
- define well-formed runtime states;
- explain preservation as invariant closure under transitions;
- formulate temporal and spatial composability as trace properties;
- define progress and quiescence;
- explain why acyclicity, finiteness, and bounded iteration enter progress;
- enumerate legal lifecycle steps for a small state;
- canonicalise states up to observation and name renaming;
- use breadth-first exploration to find minimal counterexample traces;
- connect a failed invariant to a missing rule premise.

7.2 Motivation: schedules hide bugs

A deterministic demo often runs components in one convenient order. A lifecycle relation is nondeterministic:

```
Database becomes unavailable.  
Repository and Metrics both have legal transitions.  
The scheduler may step either first.
```

A bug may appear only under one interleaving:

```
provider leaves  
unrelated component mutates context
```

```
consumer unloads
provider recovery runs
```

Testing one schedule is therefore weak evidence. A finite model checker can explore all schedules for small systems and ask whether an invariant survives each one.

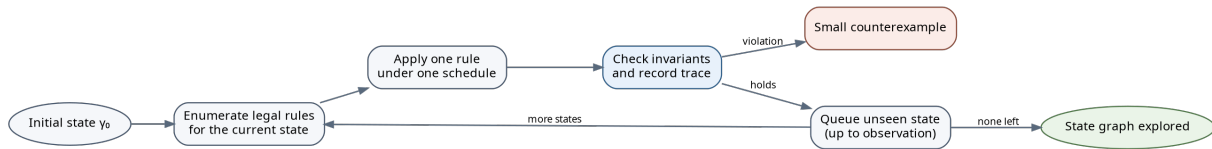


Figure 7.1: A small model checker explores legal rules and reports the first invariant violation.

7.3 What metatheory means here

Definition 7.1 (metatheory). Metatheory studies properties of the formal runtime system rather than one application running inside it. Typical claims quantify over all well-formed states and all legal steps or traces.

Examples:

- every legal transition preserves registry well-formedness;
- unloading a fiber removes its contribution and not other fibers' contributions;
- a consumer's provider remains available throughout teardown;
- a non-quiescent finite acyclic system always has a legal lifecycle step;
- all terminating schedules reach equivalent normal forms under stated assumptions.

A model checker does not prove these claims for unbounded systems. It checks all states within a chosen finite bound and is excellent at finding counterexamples, validating rule translations, and revealing missing hypotheses.

Testing versus proof. Passing a bounded exploration is not a proof of the theorem. A failing exploration is a genuine counterexample to the tested model. The most valuable output is often a short trace explaining why a rule set is wrong.

7.4 A compact pure model

The production runtime contains promises, closures, and host objects that are hard to enumerate. Build a separate pure model with finite identifiers and symbolic effects.

```
export type ModelKey = "db" | "repo" | "api" | "metrics";
export type ModelFiberId = 0 | 1 | 2 | 3;

export type ModelLifecycle =
  | { tag: "inactive"; failed: boolean }
```

```

| { tag: "loading"; committed: ModelView; remaining: number }
| { tag: "active"; committed: ModelView }
| { tag: "unloading"; committed: ModelView };

export interface ModelFiber {
  id: ModelFiberId;
  requires: readonly ModelKey[];
  provides: readonly ModelKey[];
  retired: boolean;
  lifecycle: ModelLifecycle;
  effects: readonly SymbolicEffect[];
}

export interface ModelState {
  fibers: readonly ModelFiber[];
  context: SymbolicContext;
  trace: readonly StepLabel[];
}

```

Symbolic effects can be simple set additions/removals with explicit owners:

```

export interface SymbolicContext {
  registrations: ReadonlyMap<string, ModelFiberId>;
  provisions: ReadonlyMap<ModelKey, ModelFiberId>;
}

```

This representation makes equality and canonicalisation tractable.

7.5 Rules as data

Represent each possible transition as a value:

```

export type Rule =
| { tag: "begin"; fiber: ModelFiberId }
| { tag: "iterate"; fiber: ModelFiberId }
| { tag: "finish"; fiber: ModelFiberId }
| { tag: "divert"; fiber: ModelFiberId }
| { tag: "raise"; fiber: ModelFiberId }
| { tag: "leave"; fiber: ModelFiberId }
| { tag: "unload"; fiber: ModelFiberId }
| { tag: "retire"; fiber: ModelFiberId }
| { tag: "remove"; fiber: ModelFiberId };

```

Then define:

```
export function enabledRules(state: ModelState): readonly Rule[];
export function applyRule(state: ModelState, rule: Rule): ModelState;
```

`enabledRules` is the executable counterpart of rule premises. `applyRule` is the counterpart of the conclusion.

7.5.1 Worked example 7.1: L-Begin

```
function canBegin(state: ModelState, fiber: ModelFiber): boolean {
  return (
    fiber.lifecycle.tag === "inactive" &&
    !fiber.lifecycle.failed &&
    target(state, fiber) !== undefined
  );
}
```

```
function applyBegin(state: ModelState, fiberId: ModelFiberId): ModelState {
  const fiber = getFiber(state, fiberId);
  const committed = target(state, fiber);
  if (committed === undefined || !canBegin(state, fiber)) {
    throw new Error("rule not enabled");
  }

  return updateFiber(state, fiberId, {
    ...fiber,
    lifecycle: {
      tag: "loading",
      committed,
      remaining: fiber.effects.length
    }
  });
}
```

The model can make iteration success/failure a finite nondeterministic choice rather than actual I/O.

7.6 Well-formedness

Motivation. A theorem needs a clear domain. We do not promise good behaviour from arbitrary corrupt records.

Definition 7.2 (well-formed registry, simplified). A model state is well formed when:

1. every committed provider identity names an existing fiber;
2. distinct fibers do not claim the same provision key;
3. every installed fiber's committed view is total on its requirements;

4. every provider named by an installed consumer is itself installed;
5. every symbolic context entry is owned by an existing installed fiber;
6. a fiber writes only registrations and provisions attributed to itself.

```
export function wellFormed(state: ModelState): boolean {
  return [
    committedProvidersExist(state),
    provisionsAreDisjoint(state),
    committedViewsAreTotal(state),
    committedProvidersAreInstalled(state),
    contextOwnersExist(state),
    effectsAreConfined(state)
  ].every(Boolean);
}
```

7.6.1 Worked example 7.2: a malformed state

```
consumer c is ACTIVE
committed {db -> provider p}
provider p has already been removed from registry
```

This state violates clauses 1 and 4. The point of the unload guard and removal premises is to make it unreachable from a well-formed initial state.

7.7 Preservation

Definition 7.3 (preservation). A transition system preserves an invariant I when:

$$I(\gamma) \wedge \gamma \rightarrow \delta \implies I(\delta).$$

For well-formedness:

$$\text{WF}(\gamma) \wedge \gamma \rightarrow \delta \implies \text{WF}(\delta).$$

An executable checker is:

```
function checkPreservation(state: ModelState): Violation | undefined {
  if (!wellFormed(state)) return undefined;

  for (const rule of enabledRules(state)) {
    const next = applyRule(state, rule);
    if (!wellFormed(next)) {
      return {
        property: "preservation",

```



```

        state,
        rule,
        next
    };
}
}

return undefined;
}

```

7.7.1 Connecting premises to clauses

- Unique-provider premises preserve disjoint provisions.
- L-Begin commits only a total target view.
- L-Unload requires !relied, so no installed consumer names the provider that is about to become inactive.
- remove permits only inactive fibers and should ensure no parent/child reference remains.

A preservation proof is often a systematic case analysis over rules. The model checker automates the case generation for a finite state.

7.8 Temporal composability as a trace property

Local recovery says a fiber’s accumulator recovers its own sequential effects. Global temporal composability says this remains true despite interleaved independent steps of other fibers.

Definition 7.4 (fiber episode). An episode of fiber n is a maximal interval during which n is installed: from beginning loading until finishing unloading.

Definition 7.5 (recovery exactness, executable form). For an episode of n , remove the steps performed by n and replay the remaining foreign state transformations from the episode’s starting context. At the end of n ’s unload, the actual context should be equivalent to that foreign-only result.

In a symbolic model, record each context transformation as an owner-labelled event:

```

export interface ContextEvent {
  owner: ModelFiberId;
  apply(context: SymbolicContext): SymbolicContext;
}

```

Then compare:

```
const actual = observeContext(stateAfterUnload);
const foreignOnly = observeContext(
  replay(initialContext, events.filter((e) => e.owner !== fiberId))
);
expect(actual).toEqual(foreignOnly);
```

7.8.1 Worked example 7.3

```
A installs alpha
B installs beta
A installs service a
C installs gamma
A unloads
```

Foreign-only replay is:

```
B installs beta
C installs gamma
```

The context after A unloads should contain exactly beta and gamma, modulo observational equivalence.

7.8.2 Assumption switch

```
CHECK_INDEPENDENCE = false;
```

Allow two fibers to mutate the same ordered chain. The checker should find a trace in which removing one does not match foreign-only replay.

7.9 Spatial composability as trace properties

Spatial composability has two major global claims.

7.9.1 Activation ordering

A fiber begins only where every requirement has an active provider:

```
if (rule.tag === "begin") {
  expect(target(state, getFiber(state, rule.fiber))).toBeDefined();
}
```

7.9.2 Withdrawal ordering

If consumer m committed key k to provider n , then throughout m 's episode:

- the committed view continues to name n ;
- the binding remains available through n ;
- n 's episode began before m 's;

- n cannot finish unloading before m .

The model checker can tag episode boundaries and verify these order relations on every explored trace.

7.9.3 Resolution coherence

Every successful activation iteration runs against the committed view, or the transition diverts and its accumulated effects are recovered. A trace assertion can record the view used by each symbolic iteration and ensure it equals the fiber's committed view.

7.10 Progress

Motivation. A preservation-safe system might still get stuck forever in a non-quiescent state.

Definition 7.6 (progress for the lifecycle). If a well-formed state is not quiescent, at least one lifecycle rule is enabled:

$$\neg \text{quiet}(\gamma) \implies \exists \delta. \gamma \rightarrow \delta.$$

Executable form:

```
if (wellFormed(state) && !isQuiescent(state)) {
  expect(enabledLifecycleRules(state).length).toBeGreaterThan(0);
}
```

7.10.1 Why the unload guard seems dangerous

A provider in UNLOADING may be blocked by consumers. If every blocked provider is relied on by another blocked provider, the system appears deadlocked.

Follow the dependency edges:

```
blocked provider n0
  <- relied on by n1
  <- relied on by n2
  <- ...
```

In a finite acyclic graph, this sequence cannot continue forever or repeat. It reaches a maximal consumer with no dependent blocking it, so that consumer can unload. This releases the next guard, and so on.

7.10.2 Assumptions behind progress

The paper's result uses assumptions including:

- the provider precedence relation is acyclic;
- only finitely many fibers appear in the trace;

- each activation has bounded finite iteration length;
- in-flight iterations eventually land rather than remaining pending forever;
- the host offers all lifecycle rules required by the semantics.

The model checker should make these assumptions explicit in its generator.

7.11 Termination and quiescence

Progress says a move exists. Termination says lifecycle reaction eventually stops under finite input and bounded work.

A practical finite model can calculate a measure, for example:

```
sum over fibers of:
  number of remaining activation steps
  + mismatch penalty between lifecycle and target
  + blocked-unload depth
```

The paper gives a more detailed bound based on target changes and iterator length. For the lab, verify empirically that every maximal lifecycle-only path within the finite model reaches a quiescent state and contains no repeated canonical state.

```
function allMaximalPathsQuiesce(initial: ModelState): boolean {
  // Depth-first search with cycle detection in the bounded state graph.
}
```

If a repeated state appears under lifecycle-only rules, you have found non-termination in the model or an insufficient canonical key.

7.12 Breadth-first exploration

Breadth-first search finds shortest counterexample traces.

```
export function explore(initial: ModelState): Violation | undefined {
  const queue: ModelState[] = [initial];
  const seen = new Set<string>();

  while (queue.length > 0) {
    const state = queue.shift()!;
    const key = canonicalKey(state);
    if (seen.has(key)) continue;
    seen.add(key);

    const violation = checkAllProperties(state);
    if (violation) return violation;
  }
}
```

```

    for (const rule of enabledRules(state)) {
      queue.push(applyRule(state, rule));
    }
  }

  return undefined;
}

```

Store a predecessor pointer rather than copying the entire trace in a larger model.

7.13 Canonicalising states

Fresh fiber names can make structurally identical states appear different. Observational equivalence can also identify representation differences.

A canonical key can:

1. sort fibers by stable structural features rather than raw IDs;
2. rename IDs to $0, 1, 2, \dots$ in first-occurrence order;
3. sort maps and sets;
4. omit diagnostic fields outside the observation relation;
5. retain every field used by rule premises.

Pitfall. A canonicalisation that forgets a field read by a rule is too coarse. It may merge states with different enabled transitions and invalidate exploration.

7.13.1 Worked example 7.4: alpha-equivalent names

```

State A: provider id 17, consumer id 42, committed db -> 17
State B: provider id 3, consumer id 9, committed db -> 3

```

If all other structure agrees, a consistent renaming relates them. The model checker should explore one representative.

7.14 Fault switches and counterexample assignments

Add explicit switches to the model.

```

export interface ModelOptions {
  guardProviderUnload: boolean;
  useProviderIdentity: boolean;
  rollbackOnFailure: boolean;
  enforceUniqueProvision: boolean;
  preserveCommittedAccess: boolean;
}

```

```
    enforceIndependence: boolean;  
  }
```

7.14.1 Switch: guardProviderUnload = false

Expected smallest failure:

```
provider active  
consumer active and committed to provider  
provider leaves  
provider unloads immediately  
consumer still installed but provider no longer installed
```

Violation: preservation/spatial ordering.

7.14.2 Switch: useProviderIdentity = false

Expected failure:

```
consumer active against provider p1  
p1 replaced by p2 under same key  
satisfaction remains true  
consumer stays active with stale commitment
```

Violation: resolution coherence.

7.14.3 Switch: rollbackOnFailure = false

Expected failure:

```
fiber performs one registration  
next iteration fails  
fiber becomes failed inactive  
registration remains
```

Violation: temporal composability.

7.14.4 Switch: enforceUniqueProvision = false

Expected failure: target resolution depends on accidental provider selection or changes without a well-defined policy.

7.14.5 Switch: preserveCommittedAccess = false

Expected failure: consumer teardown cannot resolve the provider it committed to.

7.14.6 Switch: enforceIndependence = false

Expected failure: removing one component changes a foreign component's observable contribution.

7.15 Reporting counterexamples

A useful report contains:

```
Property: installed consumers name installed providers
Initial state: ...
Options: guardProviderUnload=false
Trace:
  1. insert provider P
  2. begin P
  3. finish P
  4. insert consumer C
  5. begin C
  6. finish C
  7. retire P
  8. leave P
  9. unload P
Failing state:
  C is ACTIVE with committed db -> P
  P is INACTIVE
Explanation:
  L-Unload lacked the !relied(P) premise.
```

The explanation should name the missing assumption or rule premise, not merely the line of code where an assertion failed.

7.16 Property-based generation versus exhaustive exploration

Use both.

Property-based testing generates larger random systems and shrinks failures. It is good for:

- maps and effect sequences;
- random component graphs;
- random orchestration histories;
- stress beyond the exhaustive bound.

Exhaustive exploration covers every schedule in a small finite universe. It is good for:

- lifecycle interleavings;
- shortest traces;
- confidence that every enabled rule combination was considered;
- comparing rule variants.

A combined workflow is:

property generator creates a small component system
 exhaustive checker explores all lifecycle schedules for that system
 failure shrinker reduces the generated system

7.17 Exercises

7.17.1 Pure model

1. Define finite keys, fibers, lifecycle states, views, and symbolic effects.
2. Implement `enabledRules` and `applyRule` for all lifecycle rules.
3. Implement orchestration inputs separately from lifecycle reactions.
4. Define `wellFormed` with at least the six clauses above.
5. Implement `isQuiescent`.
6. Implement a canonical state key up to fiber renaming.
7. Implement breadth-first exploration with predecessor reconstruction.

7.17.2 Property checks

8. Check preservation after every transition.
9. Check activation ordering for every `begin` rule.
10. Check that installed consumers name installed providers.
11. Check committed binding availability throughout episodes.
12. Check terminal recovery against foreign-only replay.
13. Check progress in every well-formed non-quiescent state.
14. Check that every maximal lifecycle-only path quiesces within the bounded model.

7.17.3 Counterexample assignments

15. Disable each fault switch independently and record the smallest counterexample.
16. For each counterexample, identify the theorem, invariant clause, or rule premise that fails.
17. Find a case where two switches interact to produce a smaller failure than either alone.
18. Deliberately make canonicalisation too coarse and show the checker missing or inventing behaviour.
19. Deliberately make canonicalisation too fine and measure state-space explosion.

7.17.4 Written theory

20. Explain preservation, progress, and confluence without using the phrase “the system works correctly.”
21. Explain why preservation alone does not imply progress.
22. Explain why progress alone does not imply termination.
23. Explain why a bounded model checker cannot prove the unbounded theorem.
24. Explain why a counterexample found within the model is still decisive for that model.
25. Relate one counterexample trace to a proof case in the paper.

7.18 Deliverable

Submit:

- a pure finite-state model separate from the production runtime;
- exhaustive schedule exploration;
- preservation, temporal, spatial, and progress checks;
- canonicalisation with a written justification of retained/forgotten fields;
- at least five automatically generated minimal counterexamples;
- a report mapping each failure to a missing assumption or premise.

7.19 Reading guide

7.19.1 Required

- Source paper: Sections 4.4.1-4.4.4.
- Source paper: Definitions 53, 58, 60, and 65; Theorems 59, 61, 63, 64, and 66.

7.19.2 Programming-languages foundations

- Pierce or Harper on preservation and progress.
- A short model-checking introduction covering state graphs, reachability, invariants, and breadth-first counterexample search.

7.19.3 Testing foundations

- fast-check introduction to properties, generators, and shrinking.
- Vitest documentation for example tests and type-level tests.

7.19.4 Read with these questions

1. Which theorem hypotheses are properties of components, and which are properties of the runtime state?
2. Why is the unload guard central to both preservation and spatial ordering?
3. How does a minimal trace help explain a formal proof obligation?

Lab 8: Confluence, Reconciliation, and Hot Replacement

8.1 Purpose and outcomes

The final question is about the history of the whole system. A dynamic runtime may load, unload, replace, fail, recover, and reorder independent transitions. Under suitable assumptions, its settled state should depend on the final desired composition rather than the accidental route taken to reach it.

After this lab, you should be able to:

- define normal form and confluence for the lifecycle relation;
- explain the connection between independent-step transposition and unique quiescent states;
- compare dynamic history with from-scratch assembly;
- define a declarative component configuration;
- implement keyed reconciliation;
- distinguish configuration identity from component runtime identity;
- implement a small transactional hot-replacement protocol;
- state the assumptions under which history independence is expected;
- design and evaluate a useful capstone built on the runtime.

8.2 Motivation: dynamic history should not accumulate scars

Consider two histories with the same final configuration.

History A

```
load Database V1
load Repository
load WebAPI
replace Database V1 with V2
unload WebAPI
reload WebAPI
```

History B

```
load Database V2
```

```
load Repository
load WebAPI
```

If both histories settle with the same components active, we want their observable contexts to agree. Otherwise every sequence of administrative actions creates a new, hard-to-reason-about system variant.

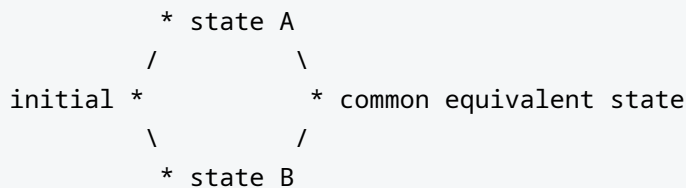
The desired claim is not that both histories emit the same log messages or network packets. It is that the recovered **context state** has no residual contribution from components that are no longer active.

8.3 Normal forms and confluence

Definition 8.1 (normal form). A state is a normal form for the lifecycle relation when no lifecycle rule applies. In this runtime, a well-behaved normal form is a quiescent state.

Definition 8.2 (confluence). A transition relation is confluent when, whenever one state can evolve along two paths, the paths can be continued to equivalent states.

Diagrammatically:



A stronger practical consequence, when lifecycle reaction also terminates, is a unique normal form up to the chosen equivalence.

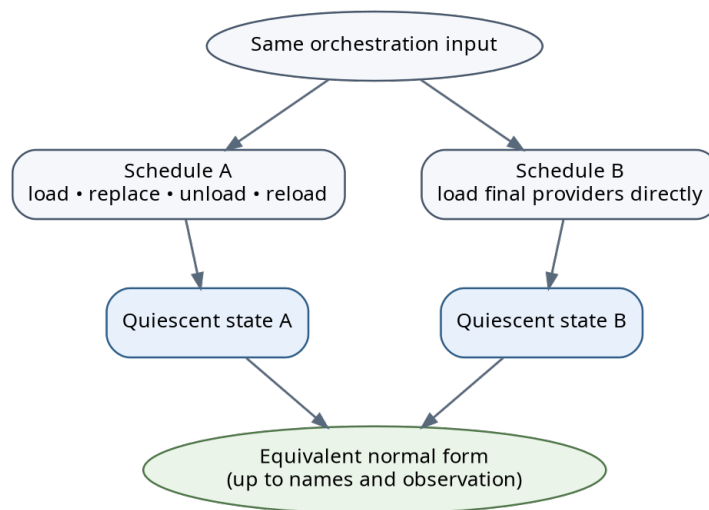


Figure 8.1: Different schedules and histories converge to an equivalent quiescent form under the theorem’s assumptions.

8.3.1 What confluence does not claim

It does not imply:

- identical timing;
- identical transient availability;
- identical log output;
- identical generated fiber names;
- identical external emissions;
- identical failure outcomes when failures depend on schedule.

The source paper excludes failed final states from its main confluence theorem because one schedule may encounter a state-dependent failure that another avoids.

8.4 From independent transpositions to schedule equivalence

Suppose two adjacent steps act on distinct independent fibers and neither changes the premises of the other. Then they can be transposed:

```
state --step A--> s1 --step B--> final
state --step B--> s2 --step A--> final
```

The endpoint is equivalent because:

- the context transformations commute;
- the lifecycle edits affect different fibers;
- the steps do not change each other's dependency resolution;
- state-dependent inverses/continuations remain coherent.

Repeatedly transposing independent adjacent steps can sort an arbitrary schedule into a canonical dependency-respecting schedule.

This is the operational role of trace equivalence from Lab 5.

8.5 From-scratch assembly

Definition 8.3 (from-scratch assembly). Given a final set of supported components, a from-scratch assembly:

1. begins with an empty runtime;
2. inserts the final component configuration;
3. activates each supported component once in dependency order;
4. never activates components that do not appear in the final support set;
5. reaches a quiescent state.

The confluence intuition is:

$\text{quiesce}(\text{dynamic history}) \simeq \text{assembleFromScratch}(\text{final configuration})$.

8.5.1 Worked example 8.1: delete a closed episode

Suppose a component A loads and later unloads completely. If temporal composability holds, its closed episode contributes nothing to the final context. Remove A's own steps from the trace. If A registered child fibers that never became supported, remove their vestigial steps as well. The remaining trace reaches an equivalent endpoint.

Repeating this deletion removes all closed episodes, leaving one open episode for every final active fiber. Independent transpositions then sort those episodes into dependency order.

You do not need to reproduce the paper's full proof, but this proof shape explains why recovery exactness, independence, progress, and support all appear in the confluence theorem.

8.6 Support and total provision

The final active set should be determined from stable declarations, not schedule accidents.

A component is **supported** when:

- it is not retired;
- its parent, if any, is supported;
- every required key has a supported provider.

The definition uses declared provisions p . To equate support with actually active fibers, a component should install every key it declares when activation finishes.

Definition 8.4 (total on provision). A component is total on its provision when every successful activation installs every key in its declared provides set.

8.6.1 Counterexample 8.1: conditional provision

```
const MaybeDatabase: Component = {
  provides: new Set([DatabaseKey]),
  activate(ctx) {
    if (ctx.config.enabled) {
      ctx.provide(DatabaseKey, db);
    }
  }
};
```

The declaration says a consumer can be supported, but one configuration may install nothing. The final active set then depends on configuration semantics beyond (d, p) .

You may support conditional provision, but the confluence/support statement must include the relevant configuration in its assumptions.

8.7 Confluence experiment in the model checker

Generate one orchestration input and explore every lifecycle schedule.

```
const normalForms = new Map<string, ModelState>();

for (const terminal of exploreToMaximalStates(initial)) {
  if (!isQuiescent(terminal)) {
    throw new Error("maximal path did not quiesce");
  }
  normalForms.set(observationalKey(terminal), terminal);
}

expect(normalForms.size).toBe(1);
```

Then compare to a canonical from-scratch schedule.

```
const canonical = assembleFromScratch(finalConfiguration);
expect(observationalKey(anyNormalForm)).toBe(
  observationalKey(canonical)
);
```

8.7.1 Assumption toggles

Confluence may fail when you disable:

- pairwise independence;
- total provision;
- acyclic dependency order;
- complete recovery;
- deterministic provider policy;
- failure exclusion.

For each toggle, seek two schedules reaching distinct quiescent observations.

8.8 Declarative configuration

An orchestrator often wants to describe what should exist rather than issue imperative lifecycle steps.

```
export interface Entry<Config = unknown> {
  readonly id: string;
  readonly component: Component;
  readonly config: Config;
  readonly disabled?: boolean;
```

```
    readonly children?: readonly Entry[];
}
```

Useful fields include:

- id: stable reconciliation identity;
- component/module identity;
- component configuration;
- disabled flag;
- parent/child grouping;
- optional isolation or interception metadata.

Definition 8.5 (authoritative configuration). A configuration is authoritative when the runtime treats it as the persistent desired composition and continuously reconciles fibers toward it.

The fiber registry is runtime state. The configuration is desired state.

8.9 Keyed reconciliation

Motivation. Rebuilding every component after each configuration edit is correct but unnecessarily disruptive.

For siblings in one configuration group, index by stable id.

```
interface ReconcilePlan {
  insert: readonly Entry[];
  retire: readonly FiberId[];
  update: readonly {
    fiber: FiberId;
    previous: Entry;
    next: Entry;
  }[];
}
```

Pseudocode:

```
oldById = map old entries by id
newById = map new entries by id

for each id in old but not new:
  retire old fiber

for each id in new but not old:
  insert new entry
```

```

for each id in both:
    if component identity changed:
        retire old fiber
        insert replacement
    else:
        apply least disruptive field updates

```

8.9.1 Worked example 8.2

Old:

```

- id: db
  component: sqlite
- id: api
  component: api

```

New:

```

- id: db
  component: postgres
- id: api
  component: api
- id: metrics
  component: metrics

```

Plan:

```

replace db fiber
keep api entry identity, but api will react to provider replacement
insert metrics fiber

```

The reconciler need not manually order API teardown around Database. The lifecycle and coeffect graph do that.

8.10 Least-disruptive updates

Different fields imply different operations.

Change	Typical action
stable id changes	treat as removal plus insertion
component/module identity changes	rebuild fiber
disabled becomes true	retire fiber
disabled becomes false	allow target and reload
configuration changes	component-defined update or rebuild
interception metadata changes	update in place if read dynamically
isolation realm changes	recompute resolution and notify affected fibers

A good reconciler delegates domain-specific configuration diffing to the component while preserving lifecycle invariants.

8.11 Configuration reconciliation property

For a final configuration C , compare:

```
start from current runtime R
apply a sequence of incremental reconciliation operations
settle
```

with:

```
start from empty runtime
load C from scratch
settle
```

The observations should agree under the confluence assumptions.

```
expect(observe(await reconcileAndSettle(existing, finalConfig))).toEqual(
  observe(await assembleAndSettle(finalConfig))
);
```

Property-based testing can generate a sequence of configuration edits and compare the final result to from-scratch assembly of the final tree.

8.12 Hot module replacement as component replacement

Hot module replacement (HMR) detects changed modules and swaps affected components without restarting the process.

In Mini-Cordis, a component already bounds:

- its context-mediated effects;
- its declared requirements and provisions;
- its live fiber identity and accumulator.

A simple HMR operation is therefore:

```
retire old fiber
load fresh component definition
insert replacement fiber
settle affected graph
```

The old component's local in-memory variables do not automatically survive. State that should survive must live in a longer-lived service or be explicitly migrated.

8.13 Transactional replacement

Module import can fail due to syntax, type-generation, or initialisation errors. A transactional protocol keeps the old component available as a rollback source.

```
async function hotReplace(entry: ManagedEntry): Promise<void> {
  const oldComponent = entry.component;
  const oldFiber = entry.fiber;

  // Import and validate first. If this fails, the running fiber is untouched.
  const freshComponent = await importFresh(entry.moduleUrl);

  let replacement: Fiber | undefined;
  try {
    runtime.retire(oldFiber);
    await runtime.settle();

    replacement = runtime.insert(freshComponent, entry.parent);
    entry.component = freshComponent;
    entry.fiber = replacement;
    await runtime.settle();
  } catch (error) {
    if (replacement !== undefined) {
      runtime.retire(replacement);
      await runtime.settle();
    }

    // Rebuild from the previous component definition only after the old fiber
    // has actually been retired. This avoids duplicating a still-live fiber.
    entry.component = oldComponent;
    entry.fiber = runtime.insert(oldComponent, entry.parent);
    await runtime.settle();
    throw error;
  }
}
```

A production implementation also manages module caches and the possibility that multiple entries depend on one changed module. The lab focuses on component-level rollback.

8.13.1 Worked example 8.3: syntax failure

1. watch detects plugin.ts changed
2. fresh import throws syntax error
3. old component definition remains available

4. no old fiber is retired, or rollback reinstalls it
5. runtime remains in old quiescent state

The safest ordering imports and validates the new module before retiring the old fiber when possible.

8.14 Rolling replacement through a broker

Replacing an exclusive provider perturbs every consumer. A broker can absorb churn.

Consumers -> DatabaseBroker -> backing providers V1, V2

Consumers commit to the stable broker. During a rolling update:

1. V2 registers with the broker;
2. traffic shifts from V1 to V2;
3. V1 drains in-flight work;
4. V1 unregisters and unloads;
5. consumers never change provider identity.

The broker's registration operations should be revertible and, ideally, commutative for distinct backing providers.

This pattern turns a system-level rolling update into an application-level composition pattern.

8.15 Capstone choices

Choose one domain where dynamic components have real utility.

8.15.1 Option A: CLI plugin host

Components can provide:

- commands;
- parsers;
- completion providers;
- output formatters.

Useful demonstration: replace a formatter while commands remain registered.

8.15.2 Option B: agent tool registry

Components can provide:

- tool definitions;
- credential providers;
- sandbox policies;
- memory stores;

- evaluators.

Useful demonstration: replace a tool provider and reactivate only dependent workflows.

8.15.3 Option C: HTTP route framework

Components can provide:

- route registries;
- middleware services;
- database repositories;
- metrics and tracing.

Useful demonstration: distinguish commutative route registration from order-sensitive middleware.

8.15.4 Option D: chatbot plugin host

Components can provide:

- platform adapters;
- command handlers;
- databases;
- moderation services;
- administrative UI extensions.

Useful demonstration: remove an adapter and drain dependents before closing it.

8.15.5 Option E: data-processing pipeline

Components can provide:

- sources;
- transforms;
- schemas;
- sinks;
- checkpoint stores.

Useful demonstration: replace a transform while retaining a longer-lived checkpoint service.

8.16 Capstone requirements

Your system must demonstrate:

1. dynamic insertion and retirement;
2. declared requirements and provisions;
3. automatic effect recovery;
4. provider replacement detection;

5. asynchronous multi-step activation;
6. failure rollback;
7. safe dependency withdrawal;
8. a trace or graph visualisation;
9. one declarative reconciliation scenario;
10. one confluence comparison against from-scratch assembly.

It must also include:

- a commutation matrix;
- a documented observational equivalence;
- at least three fault switches;
- automatically found counterexamples;
- a clear system-boundary statement.

8.17 Final report structure

A 3-5 page technical report should answer:

8.17.1 1. Runtime boundary

Which effects are tracked exactly? Which operations are emissions or external compensations?

8.17.2 2. Component algebra

Which primitives commute, under what observations and preconditions?

8.17.3 3. Lifecycle invariants

State at least three invariants and identify the rules that preserve them.

8.17.4 4. Theorem-to-system mapping

Explain three of:

- recovery exactness;
- activation/withdrawal ordering;
- resolution coherence;
- progress;
- confluence.

Use your own trace and code, not only the paper's notation.

8.17.5 5. Counterexamples

Show what fails when assumptions are removed.

8.17.6 6. Limitations

Identify unsupported effects, scaling limits, type-system gaps, failure modes, and security boundaries.

8.18 Exercises

8.18.1 Confluence experiments

1. Generate multiple lifecycle schedules for one orchestration input and compare normal forms.
2. Compare every normal form to from-scratch assembly.
3. Disable independence and find two distinct quiescent endpoints.
4. Add a schedule-dependent failure and show why the main confluence comparison excludes it.
5. Compare states up to fresh-name renaming.

8.18.2 Reconciliation implementation

6. Define configuration entries with stable IDs.
7. Implement keyed insert, retire, replace, and update planning.
8. Reconcile nested configuration trees.
9. Generate random edit sequences and compare the final settled state with from-scratch assembly.
10. Measure how many fibers reload under incremental reconciliation versus wholesale restart.

8.18.3 HMR implementation

11. Implement fresh module loading for a simple local plugin format.
12. Replace one component while retaining unrelated fibers and state.
13. Simulate an import failure and restore the prior component.
14. Make state persistence explicit through a longer-lived service.
15. Add a broker and demonstrate a rolling provider replacement without consumer reload.

8.18.4 Written theory

16. Explain how deleting closed episodes and transposing independent steps leads toward a canonical schedule.
17. Explain why total provision is relevant to identifying the final active set.
18. Explain which observable history confluence intentionally ignores.
19. Compare reconciliation with change propagation or incremental computation.
20. State one extension that would require strengthening the formal model.

8.19 Milestone D: metatheoretic runtime and capstone

A successful final milestone contains:

- a useful domain application;
- a complete dynamic lifecycle;
- bounded exhaustive verification;
- property-based configuration tests;
- reconciliation and replacement;
- a theorem-to-code report;
- counterexamples demonstrating the necessity of assumptions.

The strongest submissions will not merely reproduce the reference API. They will use the theory to justify an interface design: for example, making a registry set-like to gain commutativity, introducing a broker to stabilise consumer identity, or moving persistent state into a longer-lived coefficient.

8.20 Reading guide

8.20.1 Required

- Source paper: Sections 4.4.5, 5.2.1, and 5.2.2.
- Source paper: Theorem 73 and its surrounding definitions.

8.20.2 Applied comparison

- Vite HMR documentation: identify the module-level acceptance and invalidation model.
- Incremental computation or change-propagation literature: compare consistency with from-scratch evaluation.
- OSGi service dynamics or rolling-update literature for provider transitions.

8.20.3 Deepening

- Source paper Sections 6.2 and 7.3-7.4.
- Trace theory references for canonicalisation by swapping independent actions.

8.20.4 Read with these questions

1. Which earlier results are prerequisites for confluence?
2. Why does a declarative reconciler rely on final-state guarantees rather than manual load ordering?
3. When should a provider be replaced directly, and when should a stable broker absorb replacement?

Theory Companion and Reading Path

This chapter gathers the background needed to read the paper without turning the course into a general survey of category theory or programming-language semantics. The recommended order follows the labs rather than historical development.

9.1 Three reading tracks

Students differ in mathematical background. Use one of three tracks.

9.1.1 Minimum track

Use this when the main goal is to complete the runtime and understand the paper's claims operationally.

1. Functions, composition, monoids, relations, directed graphs, induction.
2. Small-step transition rules, invariants, preservation, progress.
3. Effects as environment modifications.
4. Coeffects as environment requirements.
5. Observational equivalence and commutation.
6. Trace equivalence and confluence intuition.

9.1.2 Standard track

Add:

- selected chapters from *Types and Programming Languages* or *Practical Foundations for Programming Languages*;
- Wadler on monads and state;
- Pretnar on algebraic effects and handlers;
- Petricek, Orchard, and Mycroft on coeffects;
- Milewski on categories, functors, monads, and comonads;
- Fong and Spivak on applied compositionality.

9.1.3 Deepening track

Add:

- Moggi on monadic semantics;
- Plotkin and Power on algebraic operations and monads;
- graded effects/coeffects;

- inverse arrows and reversible computation;
- Mazurkiewicz trace theory;
- operational metatheory and mechanisation in a proof assistant.

Reading discipline. Every reading should answer a question created by the implementation. Reading comonads before you understand the runtime dependency problem usually increases vocabulary without increasing insight.

9.2 Discrete mathematics for the paper

9.2.1 Motivation

The calculus relies more on discrete structures than on continuous mathematics. The essential tools are functions, finite maps, relations, graphs, induction, invariants, and well-founded order.

9.2.2 Core definitions

- **Finite partial map:** a map whose domain is a finite subset of possible keys.
- **Relation:** a predicate over pairs.
- **Partial order:** a reflexive, antisymmetric, transitive relation.
- **DAG:** a directed graph without cycles.
- **Well-founded relation:** a relation with no infinite descending chain; supports recursive definitions and induction.
- **Invariant:** a predicate true initially and preserved by every transition.

9.2.3 Applied example: provider precedence

Define:

$$n \prec m \iff p_n \cap d_m \neq \emptyset.$$

Read: fiber n may provide something required by fiber m . If $\prec$ is acyclic, providers can be ordered before consumers. During withdrawal, a maximal consumer can leave first, helping the unload guard release.

9.2.4 Recommended resource

Eric Lehman, F. Thomson Leighton, and Albert R. Meyer, *Mathematics for Computer Science* (MIT OpenCourseWare). Focus on:

- definitions and proofs;
- induction;
- sets and relations;
- directed graphs and partial orders;
- state-machine invariants.

9.2.5 Reading exercise

For each proof technique below, identify one handbook property it suits:

Technique	Possible use
counterexample	refute commutation or a proposed equivalence
induction on sequence length	composite recovery
induction on graph order	guarded withdrawal progress
invariant preservation	registry well-formedness
contradiction	show a provider cannot unload before a committed consumer

9.3 Operational semantics

9.3.1 Motivation

An implementation contains accidental detail: data structures, task queues, and error plumbing. Operational semantics isolates legal behaviour as transition rules.

9.3.2 Definition

A **small-step operational semantics** defines a relation $\gamma \rightarrow \delta$ where one rule application moves the system by one conceptual step.

A transition rule:

$$\frac{\text{premises}}{\text{conclusion}}$$

is an executable specification. Premises are guards; the conclusion is the state update.

9.3.3 Worked example

```
Premises:
  fiber is inactive
  target exists
Conclusion:
  fiber becomes loading with that committed target
```

This can be read as a rule, a pure enabled predicate plus apply function, or an imperative guarded branch.

9.3.4 Preservation and progress

Preservation asks whether legal steps maintain well-formedness.

$$\text{WF}(\gamma) \wedge \gamma \rightarrow \delta \implies \text{WF}(\delta).$$

Progress asks whether a non-final well-formed state can move.

$$\text{WF}(\gamma) \wedge \neg \text{quiet}(\gamma) \implies \exists \delta. \gamma \rightarrow \delta.$$

These words are familiar from type-safety theorems, but the paper applies the style to a component lifecycle.

9.3.5 Recommended resources

- Benjamin C. Pierce, *Types and Programming Languages*. Use the chapters that introduce syntax, small-step evaluation, and type-safety proof structure.
- Robert Harper, *Practical Foundations for Programming Languages*. Use as a systematic account of structural operational semantics and language definitions.

9.3.6 Applied exercise

Take one runtime branch and write:

1. its source lifecycle pattern;
2. every value it reads;
3. every premise required for safety;
4. every field it writes;
5. the invariant clauses affected by those writes.

9.4 Effects: three different ideas that share a name

9.4.1 Motivation

The word *effect* can refer to runtime mutation, a static effect annotation, or an algebraic operation. Distinguishing them prevents confusion.

9.4.2 Runtime effect in this handbook

A context transformation:

$$\Gamma \rightarrow \Gamma$$

or a revertible effect function:

$$\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma).$$

9.4.3 Static effect system

A typing judgement annotates what a term may do:

$$\Gamma \vdash t : T^\epsilon.$$

The annotation may track state regions, exceptions, I/O, or other effects before execution.

9.4.4 Algebraic effects and handlers

A program invokes abstract operations such as `get`, `put`, or `raise`. A handler interprets those operations and controls continuations. This separates an effect interface from its interpretation.

9.4.5 Relationship to the paper

The source paper does not simply implement an algebraic-effect handler. It lifts the broad idea of effects into runtime context transformations paired with inverses and tracked over component lifetimes.

9.4.6 Recommended resources

- Philip Wadler, *Monads for Functional Programming*: a programming-centred explanation of structuring state, exceptions, output, and nondeterminism.
- Matija Pretnar, *An Introduction to Algebraic Effects and Handlers*: a tutorial with operations, handlers, operational semantics, and effect typing.
- Eugenio Moggi, *Notions of Computation and Monads*: foundational deepening.
- Gordon Plotkin and John Power, *Notions of Computation Determine Monads*: algebraic-operation perspective.

9.4.7 Reading questions

1. Where does each account locate sequencing?
2. Does the mechanism track what may happen, interpret operations, or recover completed mutations?
3. Is the effect scoped lexically, dynamically, or by component lifetime?
4. Does the mechanism derive cleanup, or merely provide a place to write it?

9.5 Coeffects

9.5.1 Motivation

Effects describe output toward the environment. Coeffects describe input demanded from the environment.

A static coeffect judgement enriches the context:

$$\Gamma^\rho \vdash t : T.$$

The annotation may express which variables, resources, permissions, dataflow neighbours, or capabilities are required.

9.5.2 Runtime interpretation in Mini-Cordis

A component declares a set of dependency keys. The current service context resolves those keys. Each context change may alter satisfaction or provider identity, which drives lifecycle transitions.

9.5.3 Comonads: how much do you need?

A comonad is categorically dual to a monad and can model context-dependent computation. The paper reviews comonadic coeffects, but the labs do not require deriving the runtime from comonad laws.

At minimum, understand the environment comonad shape:

$$D(X) = E \times X,$$

with operations that extract a value from context and duplicate context for nested computation. Then return to the concrete question: which services must this component have?

9.5.4 Recommended resources

- Tomas Petricek, Dominic Orchard, and Alan Mycroft, *Coeffects: Unified Static Analysis of Context-Dependence*.
- Their later calculus paper for a deeper formal account.
- Selected comonad chapters from Milewski only after Reader/environment dependence is familiar.

9.5.5 Classification exercise

Classify each statement as effect, coeffect, both, or neither.

1. “This function may write the log.”
2. “This component requires a database.”
3. “This operation installs a database binding.”
4. “This computation may use its argument at most once.”
5. “This component requires permission to write /tmp.”
6. “This operation sends a network packet.”

The third is deliberately both: providing a coeffect binding is an effect on the coeffect context.

9.6 The unified context and the context paradigm

9.6.1 Motivation

Labs 1-2 used an effect context and Lab 3 used a coeffect/service context. Treating them as unrelated objects would permit a component to mutate one environment while declaring requirements against another. The paper instead makes one first-class context mediate both directions:

```
component -> context: performs tracked effects
context -> component: supplies and reacts to coeffects
```

Every environment interaction should be attributable to the component context on which it was invoked.

9.6.2 Definition

The paper presents a recursive unified context of the shape:

$$\Gamma_{\infty} = \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma.$$

Read the constituents as:

- recursive current context state;
- recovery accumulator for this level;
- coeffect context containing dependency information.

The fixed point makes child contexts have the same general structure as parents. A child can carry local effects, provisions, isolation, interception, and further children.

9.6.3 TypeScript approximation

```
export interface RuntimeContext {
  readonly parent?: RuntimeContext;
  readonly fiber: Fiber;
  readonly scope: DisposableScope;
  readonly bindings: BindingStore;
  readonly isolation: RealmTable;
  readonly interception: MetadataTable;
}
```

The fields need not literally encode the mathematical product. The implementation correspondence is semantic:

- scope carries the accumulator;
- stores/tables carry Σ ;
- parent gives recursive hierarchy;
- every public operation is attached to this context.

9.6.4 Worked example: nested plugin groups

```
root context
|
+-- group A context
|   +-- database provider
```

```

|      +-- repository consumer
|
+-- group B context
    +-- isolated database provider
    +-- repository consumer

```

Each repository uses the same component definition. The child context determines which provider it resolves. Unloading group B disposes its children, local bindings, and isolation overrides without disturbing group A.

9.6.5 The context paradigm

A programming paradigm is partly defined by how programs access and modify their environment.

- Pure functional state threading makes the environment explicit in every function type.
- Mainstream imperative programming makes one mutable ambient environment largely implicit.
- The context paradigm uses explicit first-class contexts at component boundaries while allowing ergonomic imperative operations through the context object.

This yields two forms of locality:

1. **effect locality:** the context attributes each mutation to the owning scope;
2. **coeffect locality:** the context authorises and resolves only declared dependencies.

9.6.6 Counterexample: escaping the assigned context

A component captures a global root context and performs an effect there rather than on its own child context. Its fiber accumulator does not own the effect, and its dependency declarations no longer describe what it can reach. The unified-context guarantee depends on preventing or detecting such escapes.

A language designed around the paradigm could make the current context implicit but unforgeable, rather than passing it as an ordinary object that can be stored globally.

9.6.7 Exercises

1. Draw the context tree for your capstone, including parent scopes and isolated realms.
2. For every public context operation, identify whether it mutates the current context, derives a child, or only observes a committed binding.
3. Demonstrate one context escape and the resulting effect leak or undeclared access.
4. Propose a language feature, lint rule, or capability restriction that would prevent the escape.
5. Explain how a longer-lived parent service can preserve state across child component replacement.

9.7 Category theory: the minimum useful map

9.7.1 Motivation

Category theory is useful here because it focuses attention on composition, identity, structure-preserving maps, opposites, and equivalence. It is not useful if treated as a vocabulary quiz detached from the runtime.

9.7.2 Category

A **category** has:

- objects;
- morphisms between objects;
- associative composition;
- an identity morphism for every object.

For programming intuition:

- objects can be types;
- morphisms can be total functions;
- composition is function composition;
- identity is $x \Rightarrow x$.

9.7.3 Endomorphism and monoid

An **endomorphism** is a morphism from an object to itself. Context transformations are endomorphisms on Γ . Endomorphisms form a monoid under composition.

9.7.4 Homomorphism

A monoid homomorphism preserves identity and composition:

$$h(e) = e, \quad h(a \star b) = h(a) \star h(b).$$

The paper's track construction preserves the composition of forward/inverse pairs. Programming interpretation:

Instrumenting an operation with recovery bookkeeping respects the way operations sequence.

9.7.5 Opposite

The opposite monoid reverses multiplication. Inverse accumulation uses the opposite order because teardown is last-in-first-out.

9.7.6 Functor

A functor maps objects and morphisms while preserving identity and composition. You do not need to force every implementation function into functor terminology. The useful habit is to ask whether a translation preserves composition.

9.7.7 Monad

A monad can structure effectful sequencing. Learn the `return/unit` and `bind/join` intuition, but do not conclude that every type constructor returning state and cleanup is automatically “the monad from the paper.” The paper’s effect composition is stated directly.

9.7.8 Comonad

A comonad structures context-dependent computation. Treat it as background for static coeffect theory, not as a prerequisite for the runtime dependency map.

9.7.9 Fixed-point type

The paper’s unified context is recursive in shape. A type equation such as:

$$\Gamma_{\infty} = \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma$$

uses μ to denote a recursive/fixed-point type. Programming intuition: each context level can contain state, recovery, dependency information, and derived child contexts of the same general form.

9.7.10 Quotient

Quotienting by $\simeq$ treats observationally equivalent states as one semantic state. The runtime may retain different representatives, but operations should respect the equivalence.

9.7.11 Paper-to-programming table

Paper construct	Mathematical lens	Programming lens
$\Gamma \rightarrow \Gamma$	endomorphism	state mutation
effect sequencing	monoid composition	execute operations in sequence
reversed inverses	opposite monoid	cleanup stack
track preserves composition	homomorphism	instrumentation respects sequencing
$\mathcal{E}_{\Gamma}$	state-dependent reversible arrow	operation returns cleanup closure
Σ	finite dependent partial map	typed service registry
coeffect	contextual requirement	dependency injection
$\simeq$	equivalence/quotient	ignore unobservable representation
independent effects	commuting transformations	safe reordering/selective unload

Paper construct	Mathematical lens	Programming lens
provider graph	order relation	activation/withdrawal constraints
trace equivalence	partial commutation	schedule equivalence
recursive context	fixed-point type	nested component scopes
confluence	unique normal form	same settled system despite history

9.7.12 Recommended resources

1. *Category Theory Illustrated*, sections on functions, categories, and monoids.
2. Bartosz Milewski, *Category Theory for Programmers*: selected chapters on categories, types/functions, monoids, functors, monads, and comonads.
3. Brendan Fong and David Spivak, *Seven Sketches in Compositionality*: for a broader applied view of compositional modelling.

9.7.13 Counterexample to bad CT usage

A common mistake is:

“This code has `map`, therefore it is a functor, therefore it is compositional.”

A lawful claim requires specified objects, morphisms, mapping, identities, composition, and laws. Naming a method does not establish the structure.

9.8 Reversible computation versus revertible effects

Reversible-computing models often require every computation to be invertible, sometimes with a two-sided inverse derived from the language or categorical structure.

Mini-Cordis asks less:

- only context-mediated atomic effects need supplied recovery;
- the inverse may be one-sided;
- the inverse may be chosen at the application state;
- external emissions may lie outside the boundary;
- a component accumulator recovers one lifetime rather than making the whole program run backwards.

Recommended deepening:

- Chris Heunen, Robin Kaarsgaard, and Martti Karvonen, *Reversible Effects as Inverse Arrows*.

Reading question:

Which global reversible laws are deliberately weakened by a runtime that records local cleanup closures?

9.9 Trace theory and concurrency

9.9.1 Motivation

A nondeterministic scheduler can produce many sequential interleavings of causally independent work. We want to identify schedules that differ only by harmless swaps.

Let I be an independence relation on action labels. Generate an equivalence by:

$$xaby \equiv xbay \quad \text{whenever } (a, b) \in I.$$

An equivalence class is a trace. The order of dependent actions is preserved; independent adjacent actions may swap.

9.9.2 Worked example

```
a = register command alpha
b = provide metrics
c = replace database
```

If a and b are independent but both may interact with c , then:

```
abc ~ bac
```

but neither is necessarily equivalent to acb or cab .

9.9.3 Recommended reading

- Antoni Mazurkiewicz, *Trace Theory*, or a modern introductory tutorial.
- A short secondary introduction is sufficient for the labs: focus on partial commutation, dependency graphs, and linearizations.

9.9.4 Applied exercise

Take one model-checker trace. Draw its dependency partial order. List all sequential schedules that are linearizations of that order. Verify that adjacent swaps correspond to independent steps.

9.10 Applied comparison readings

The following resources make the paper's design differences concrete.

9.10.1 React effects

Read the official `useEffect` and reactive-effect lifecycle documentation. Focus on:

- setup returning cleanup;
- cleanup before resynchronisation and unmount;
- the requirement that cleanup mirrors setup;
- hook call-order and async restrictions.

Compare with Mini-Cordis:

- atomic effects can be called through arbitrary helpers and iteration;
- inverses compose into a component accumulator;
- dependencies are runtime coeffects with provider identity;
- provider withdrawal coordinates multiple component lifecycles.

9.10.2 Dependency injection and service dynamics

Read Fowler on dependency injection and OSGi Declarative Services on dynamic service references.

Compare:

- initial injection versus reactive re-resolution;
- service availability versus provider identity;
- developer-authored deactivation versus accumulated inverses;
- synchronous callbacks versus asynchronous guarded teardown.

9.10.3 Hot module replacement

Read Vite's HMR API. Identify:

- how modules accept updates;
- how state is handed forward or disposed;
- what happens when an update is declined;
- which parts are framework-specific.

Compare with component-bounded recovery followed by clean reapplication.

9.10.4 Property-based testing

Read fast-check on generators, properties, and shrinking. Map:

```
mathematical universal claim
  -> generated inputs
  -> executable predicate
  -> smallest failing witness
```

9.11 Reading matrix by lab

	Lab	Source paper	Foundation	Applied bridge
	0	1.1, 2.3, opening 3.1	MCS; Milewski composition	state-machine examples
	1	3.1.1-3.1.2	Wadler; Pretnar	React setup/cleanup
	2	3.1 and 5.1.1	invariants and induction	RAII/bracket patterns
	3	2.2, 3.2.1-3.2.2	Petricek et al.	DI; OSGi services
	4	4.1-4.2	TAPL/PFPL operational semantics	component runtimes
	5	3.1.3, 3.3.2	equivalence; trace intuition	registries vs middleware
	6	4.3, 5.1.3	async state machines	React/OSGi/HMR lifecycle
	7	4.4.1-4.4.4	preservation, progress, model checking	fast-check
	8	4.4.5, 5.2	confluence and traces	reconciliation and HMR

9.12 Theory study exercises

1. Make a one-page concept map connecting monoid, opposite monoid, accumulator, and LIFO recovery.
2. Explain effects and coeffects using one example that is not a service registry.
3. Translate one static effect judgement and one static coeffect judgement into plain language.
4. Explain why the paper moves the concepts from compile-time annotations to runtime context objects.
5. Give an interface whose observational equivalence changes when one extra method is exposed.
6. Show how a dependency graph defines a partial order only when cycles are absent.
7. Explain how a homomorphism differs from an arbitrary wrapper function.
8. Compare a global two-sided inverse with a local one-sided witness.
9. Describe one trace-equivalence class for your capstone.
10. Write three questions you can now ask of a dynamic plugin system that ordinary life-cycle documentation does not answer precisely.

Reference Architecture

This chapter consolidates the interfaces developed incrementally across the labs. It is not a required exact implementation. Use it to check whether your design contains the semantic information required by the theory.

10.1 Core algebra

```
export type Endo<S> = (state: S) => S;

export type Effect<S> = (
  state: S
) => readonly [next: S, inverse: Endo<S>];

export interface Equivalence<S> {
  equivalent(left: S, right: S): boolean;
}

export function composeEffects<S>(
  later: Effect<S>,
  earlier: Effect<S>
): Effect<S>;

export function identityEffect<S>(): Effect<S>;
```

Required laws:

```
undo(effect(state).next) ~= state
identity <> effect = effect
effect <> identity = effect
(effectA <> effectB) <> effectC = effectA <> (effectB <> effectC)
```

10.2 Typed keys and bindings

```
export class Key<T> {
  readonly id: symbol;
  constructor(readonly description: string);
}

export interface Binding<T> {
  readonly key: Key<T>;
  readonly value: T;
  readonly provider: FiberId;
}

export interface BindingStore {
  has<T>(key: Key<T>): boolean;
  get<T>(key: Key<T>): Binding<T> | undefined;
  install<T>(binding: Binding<T>): void;
  remove<T>(key: Key<T>, provider: FiberId): void;
}
```

Required invariants:

```
one active binding per key per realm
binding provider exists
binding provider is installed
provider declares the key in its provision
```

10.3 Component and fiber

```
export type FiberId = symbol;
export type ProviderView = ReadonlyMap<Key<unknown>, FiberId>;
export type TargetView = ProviderView | undefined;

export interface Component<Config = unknown> {
  readonly name: string;
  readonly requires: ReadonlySet<Key<unknown>>;
  readonly provides: ReadonlySet<Key<unknown>>;
  activate(
    ctx: FiberContext,
    config: Config
  ): AsyncGenerator<Acquire, void, void>;
}
```

```
export interface Fiber<Config = unknown> {
  readonly id: FiberId;
  readonly component: Component<Config>;
  readonly config: Config;
  readonly parent?: FiberId;
  retired: boolean;
  state: Lifecycle;
}
```

10.4 Lifecycle

```
export type Lifecycle =
  | {
    tag: "inactive";
    error?: unknown;
  }
  | {
    tag: "loading";
    committed: ProviderView;
    scope: DisposableScope;
    transition: Promise<void>;
  }
  | {
    tag: "active";
    committed: ProviderView;
    scope: DisposableScope;
  }
  | {
    tag: "unloading";
    committed: ProviderView;
    scope: DisposableScope;
    transition: Promise<void>;
    outcome?: unknown;
  };

```

State predicates:

```
export function isInstalled(state: Lifecycle): boolean {
  return state.tag !== "inactive";
}

export function isProviding(state: Lifecycle): boolean {

```



```
    return state.tag === "active";
  }
```

The distinction is essential. UNLOADING is installed for committed consumers but not providing for new target resolution.

10.5 Fiber context

```
export type Acquire = () => Dispose | Promise<Dispose>;

export interface FiberContext {
  readonly fiberId: FiberId;

  require<T>(key: Key<T>): T;
  effect(acquire: Acquire): Promise<void>;
  provide<T>(key: Key<T>, value: T): Promise<void>;
  command(name: string, handler: Handler): Promise<void>;
  subscribe<E>(topic: string, listener: (event: E) => void): Promise<void>;
  interval(callback: () => void, milliseconds: number): Promise<void>;
  use<C>(component: Component<C>, config: C): Promise<FiberId>;
}
```

Access discipline:

```
require reads only declared keys
require resolves through committed provider identity
effect records inverse in current fiber scope
provide tags binding with current fiber identity
use registers child and records retirement as inverse
no operation writes another fiber's lifecycle fields
```

10.6 Runtime orchestration

```
export interface Runtime {
  insert<C>(
    component: Component<C>,
    config: C,
    parent?: FiberId
  ): FiberId;

  retire(id: FiberId): void;
  remove(id: FiberId): void;
  retry(id: FiberId): void;
```

```

target(id: FiberId): TargetView;
reliedUpon(id: FiberId): boolean;

step(): Promise<boolean>;
settle(): Promise<void>;

observe(): ObservableRuntime;
}

```

Orchestration rules should not directly force fibers active or inactive. They change retirement, existence, configuration, or component identity; lifecycle rules react.

10.7 Lifecycle algorithm

```

refresh(fiber):
    target := compute target view

    if fiber is INACTIVE without error and target exists:
        begin reload against target

    if fiber is ACTIVE and target != committed:
        mark UNLOADING immediately
        notify dependents
        start guarded recovery

    if fiber is LOADING:
        iteration runner checks target at boundaries
        stale target -> divert to UNLOADING
        failure -> UNLOADING with error outcome
        successful finish with stable target -> ACTIVE

    if fiber is UNLOADING:
        wait until no installed committed consumer names fiber
        run accumulator
        discard committed view
        if current target exists and no terminal error:
            begin fresh reload
        else:
            become INACTIVE

```

10.8 Disposable scope algorithm

```
export class DisposableScope {
  #recover: AsyncDispose = async () => {};
  #armed = true;

  add(inverse: Dispose): void {
    if (!this.#armed) {
      throw new Error("cannot add to disposed scope");
    }

    const previous = this.#recover;
    this.#recover = async () => {
      await inverse();
      await previous();
    };
  }

  async dispose(): Promise<void> {
    if (!this.#armed) return;
    this.#armed = false;

    const recover = this.#recover;
    this.#recover = async () => {};
    await recover();
  }
}
```

A production scope should specify an error policy. One robust option attempts every inverse and aggregates errors, while preserving the original order.

10.9 Tracing

A trace makes formal states visible during debugging.

```
export interface TraceEvent {
  readonly sequence: number;
  readonly rule: string;
  readonly fiber?: FiberId;
  readonly before: RuntimeDigest;
  readonly after: RuntimeDigest;
  readonly details?: unknown;
}
```

Useful fields include:

- lifecycle source/destination;
- target and committed provider IDs;
- changed keys;
- acquired/released resource labels;
- guard blockers;
- failure outcomes;
- parent/child registration.

A human-readable trace should support counterexample reports from Lab 7.

10.10 Invariant checklist

Run these checks after every model step and optionally in debug builds.

10.10.1 Registry

- every parent ID exists or is root;
- provision sets are disjoint under the current realm policy;
- retired flags move only from false to true unless explicit retry semantics say otherwise;
- inactive fibers carry no committed view or active scope.

10.10.2 Dependencies

- every installed fiber has a total committed view;
- every committed provider exists and is installed;
- every committed key belongs to the provider's declared provision;
- dependency access uses the committed provider.

10.10.3 Effects

- each context entry is attributable to one owner fiber;
- each successful atomic acquisition records one inverse;
- disposal runs at most once;
- a failed or diverted activation recovers its completed prefix.

10.10.4 Lifecycle

- only active fibers contribute to new target resolution;
- an unloading provider cannot finish while relied upon;
- one fiber has at most one in-flight transition;
- a target mismatch cannot leave a fiber stably active.

10.10.5 Quiescence

- inactive non-failed fibers have no target;
- active fibers have target equal to committed;
- no fiber remains loading or unloading.

10.11 Suggested repository milestones

```
v0.1 algebra
  immutable state and effect laws

v0.2 scopes
  automatic LIFO recovery and real resource wrappers

v0.3 coeffects
  typed keys, requirements, notification

v0.4 fibers
  target/committed views and base lifecycle

v0.5 independence
  observation and selective-withdrawal tests

v0.6 async lifecycle
  iteration, failure, inertia, guarded withdrawal

v0.7 model
  bounded exhaustive checker and fault switches

v0.8 loader
  reconciliation, replacement, capstone
```

Notation and Glossary

A.1 Mathematical notation

Notation	Meaning in this handbook
$x : X$	value x has type X
$f : X \rightarrow Y$	total function from X to Y
$f : X \rightharpoonup Y$	partial function, possibly undefined
$f \circ g$	composition; run g first, then f
id_X	identity function on X
Γ	runtime context/state type
γ, δ	particular context states
$\partial\Gamma$	effect context: state plus recovery accumulator
Σ	coeffect/service context
$\mathcal{E}_\Gamma$	effect functions over Γ
$\diamond$	sequential effect composition
d	component requirement/coeffect specification
p	component provision set
e	component activation/effect program
F_γ	fiber registry in state γ
ω	committed provider view
$\perp$	no target; fiber should not run
$\simeq$	observational equivalence
$\models$	satisfaction of a requirement by a context
$\mu X.F(X)$	recursive/fixed-point type
$\gamma \rightarrow \delta$	one lifecycle transition
$\rightarrow^*$	zero or more transitions

A.2 Core terms

Accumulator. A composed inverse that recovers all effects recorded in a scope, normally in LIFO order.

Activation. Execution of a component's effect program after its requirements resolve.

Active provider. A fiber in the ACTIVE state whose provisions participate in new target resolution.

Acquisition. The recoverable phase of obtaining or registering a resource, such as opening a handle or installing a listener.

Coeffect. An environmental requirement of a computation. In Mini-Cordis, a dynamically resolved dependency key.

Coeffect context. The finite typed service context against which requirements are resolved.

Committed view. The map from required keys to provider identities recorded when a fiber begins activation.

Commutation. The property $f \circ g \simeq g \circ f$.

Component. A reusable description (d, p, e) of requirements, possible provisions, and activation effects.

Confluence. The property that divergent transition paths can be continued to equivalent states; with termination, this yields unique normal forms up to equivalence.

Context. The runtime-mediated environment containing shared bindings, registries, and other tracked state.

Context discipline. The requirement that component/environment interactions pass through the assigned context API.

Deactivation. Recovery of a fiber's accumulated effects when its target changes or it is retired.

Derived context. A child context that changes resolution or metadata without mutating its parent, discarded on recovery.

Effect. A modification of the environment. In the core model, a transformation of Γ .

Effect context. A pair of current state and accumulated recovery function.

Effect iterator. A multi-step activation program that yields one inverse and a continuation boundary per successful iteration.

Emission. An operation whose information crosses the recoverable boundary, such as sending a packet. It may require withholding or compensation rather than exact inverse recovery.

Endomorphism. A function from a type to itself, such as $\Gamma \rightarrow \Gamma$.

Episode. A maximal interval during which a fiber is installed.

Equivalence relation. A reflexive, symmetric, transitive relation used to identify states considered semantically the same.

Fiber. A live component instance with identity, lifecycle, committed view, and accumulator.

Guarded withdrawal. Delaying a provider's final recovery until no installed consumer's committed view relies on it.

Independence. A relation between effects requiring generated forward/inverse transformations to commute and state-dependent yields/outcomes to remain stable under foreign transformations.

Inertia. The requirement that an already launched asynchronous iteration lands; stale results are recorded and then recovered rather than ignored.

Installed fiber. A loading, active, or unloading fiber carrying a committed view and accumulator.

Inverse at an application state. A function g returned for one effect application such that $g(\delta) \simeq \gamma$ for that application. It need not be a global two-sided inverse.

Lifecycle rule. An automatic reactive transition such as begin, finish, leave, or unload.

Monoid. A carrier with associative composition and an identity.

Normal form. A state with no applicable lifecycle transition; usually a quiescent state.

Observation. An allowed interaction or canonical summary used to distinguish states.

Observational equivalence. Indistinguishability by the allowed operations of the context interface.

Operational semantics. A rule-based definition of legal transitions between states.

Orchestration rule. An external input such as insert, retire, configuration update, or remove.

Partial function. A function that may be undefined for some inputs.

Preservation. The property that every legal step from a well-formed state reaches a well-formed state.

Progress. The property that a well-formed non-quiescent state has at least one legal lifecycle step.

Provider view. A map from required keys to the identities of their providers.

Provision. A set of keys a component may install while active.

Quiescence. A state in which every fiber's lifecycle agrees with its target and no transition remains in progress.

Reactive coeffect. A requirement whose satisfaction/provider resolution is re-evaluated after context changes and drives lifecycle transitions.

Reconciliation. Incrementally changing live fibers to match an authoritative declarative configuration.

Recoverable boundary. The portion of the environment the runtime can attribute, control, and restore up to equivalence.

Recovery exactness. The global property that removing a fiber leaves the state foreign steps would have produced without that fiber.

Requirement. The finite set of keys a component declares it needs.

Resolution coherence. The property that one activation runs against one committed provider view, or diverts and recovers when the view changes.

Retirement. An orchestration request that makes a fiber's target bottom; it is distinct from physical registry removal.

Satisfaction. The predicate that all keys in a requirement are currently resolved.

Selective withdrawal. Removing one effect/component while retaining later or interleaved effects from others.

Soundness invariant. The equation saying the current accumulator still recovers the scope's initial state.

Spatial composability. Declaring and reactively coordinating component dependencies, including safe activation and withdrawal order.

Target view. The provider map a fiber should currently run against, or bottom when retired/unsatisfied.

Temporal composability. Recovering a component's attributable context modifications on removal without disturbing independent components.

Trace. A sequence of transition labels/states. In trace theory, schedules differing only by swaps of independent actions may be equivalent.

Twisted composition. Composition of forward/inverse pairs where inverse order is reversed.

Well-formedness. The conjunction of registry, provider, committed-view, ownership, and lifecycle invariants defining valid runtime states.

Proof and Property Templates

B.1 Proving an invariant

Use this structure.

1. **State the predicate precisely.** Avoid “the registry is valid.” List the fields and relationships.
2. **Base case.** Show the initial state satisfies it.
3. **Inductive/step case.** Assume the predicate at γ and analyse every rule $\gamma \rightarrow \delta$.
4. **Localise writes.** Identify which fields the rule can change.
5. **Use premises.** Point to the premise that protects each affected invariant clause.
6. **Conclude.** All clauses hold at δ .

B.1.1 Example skeleton

Invariant: every installed consumer names installed providers.

Base:

the initial registry is empty, so the claim is vacuous.

Step:

consider each rule.

- begin: committed target contains active providers, hence installed.
- ordinary iteration: provider lifecycle fields are unchanged.
- provider unload: !relied premise ensures no installed consumer names it.
- remove: removed fiber is inactive and is named by no installed consumer.

Therefore every legal step preserves the invariant.

B.2 Proving composite recovery

Given effects:

$$e_1(\gamma) = (\delta, g_1), \quad e_2(\delta) = (\varepsilon, g_2),$$

with:

$$g_1(\delta) \simeq \gamma, \quad g_2(\varepsilon) \simeq \delta,$$

calculate:

$$(g_1 \circ g_2)(\varepsilon) = g_1(g_2(\varepsilon)) \simeq g_1(\delta) \simeq \gamma.$$

When $\simeq$ is not equality, cite that g_1 respects equivalence.

B.3 Proving selective withdrawal

For independent A and B:

```
initial γ
A -> Aγ
B -> B Aγ
undo A -> ?
```

Use commutation to move undo A past B:

$$g_A(B(A(\gamma))) \simeq B(g_A(A(\gamma))) \simeq B(\gamma).$$

Then check that B's yielded inverse/continuation is unchanged at the resulting state.

B.4 Designing a property test

A good property test has:

1. **generator:** finite states and legal operations;
2. **precondition:** assumptions such as distinct keys;
3. **execution:** both sides of a law;
4. **observation:** equality or chosen equivalence;
5. **diagnostic labels:** enough context to explain a failure.

```
fc.assert(
  fc.property(stateArb, effectArb, (state, effect) => {
    const [next, undo] = effect(state);
    expect(observe(undo(next))).toEqual(observe(state));
  })
);
```

Do not hide a theorem hypothesis inside a generator without documenting it. A reader should know which states are excluded.

B.5 Designing a counterexample

A useful counterexample is:

- small;
- reachable;
- observable through an allowed operation;
- tied to one missing assumption;
- represented as a trace.

Template:

Claim being challenged:

Assumption removed:

Initial state:

Steps:

Failing observation:

Why the original assumption prevents it:

Lab Submission Checklists

C.1 Milestone A checklist

- ☐ Atomic effects return inverses at application time.
- ☐ Composite inverse order is correct.
- ☐ Disposal is at most once.
- ☐ Nested scopes recover correctly.
- ☐ Property tests cover generated states.
- ☐ Direct untracked mutation is demonstrated as a leak.
- ☐ System boundary is documented.

C.2 Milestone B checklist

- ☐ Keys are nominal and typed.
- ☐ Requirements and provisions are declared.
- ☐ Service changes trigger reactive refresh.
- ☐ Fibers have fresh identities.
- ☐ Target views record provider identity.
- ☐ Committed views mediate access.
- ☐ Retirement is separate from removal.
- ☐ Quiescence is executable.

C.3 Milestone C checklist

- ☐ Observational equivalence is documented.
- ☐ Runtime primitives have a commutation matrix.
- ☐ Independent selective withdrawal is tested.
- ☐ Non-independent counterexamples exist.
- ☐ Activation is multi-step and asynchronous.
- ☐ Stale transitions divert and recover.
- ☐ Failures recover completed prefixes.
- ☐ Provider withdrawal is guarded.
- ☐ Consumer teardown retains committed access.

C.4 Milestone D checklist

- ☐ Pure bounded model exists separately from production runtime.
- ☐ Every rule has an enabled predicate and transition function.

- ☐ Preservation, temporal, spatial, and progress checks run.
- ☐ Fault switches produce minimal counterexamples.
- ☐ All bounded lifecycle paths quiesce under assumptions.
- ☐ Normal forms agree across schedules.
- ☐ Reconciliation agrees with from-scratch assembly.
- ☐ Hot replacement rolls back on failure.
- ☐ Capstone report maps theorems to code and traces.

C.5 Suggested rubric

Criterion	Excellent	Competent	Developing
Semantic fidelity	APIs and transitions clearly match stated definitions and laws	Core lifecycle works with minor unexplained deviations	Behaviour is mostly ad hoc or lifecycle states lack clear meaning
Recovery	Atomic and composite recovery tested under interleaving and failure	LIFO cleanup works in ordinary cases	Cleanup is handwritten, incomplete, or double-applied
Coeffects	Requirements, provider identity, committed access, and withdrawal order are explicit	Requirements trigger activation/deactivation	Dependencies are hidden lookups or only initial injection
Theory	Claims are stated precisely and tied to traces/tests	Main terms are explained correctly	Uses theorem names without stating properties
Counterexamples	Minimal automated traces identify missing hypotheses	Manual counterexamples demonstrate some limits	Failures are asserted but not reproduced
Verification	Exhaustive bounded exploration plus property testing	Several invariants and generated tests	Only happy-path example tests
Engineering	Clear ownership, diagnostics, and modular architecture	Usable small runtime	Tight coupling or untracked escape paths dominate

Bibliography and Resource Guide

The source paper is the primary text for every lab:

- Yifan Shi, Wei Zhang, and Tianyi Cui, *A Programming Paradigm for Spatiotemporal Composability*. Sections 1-5 are the core course material; Sections 6-7 provide design boundaries and related work.

D.1 Mathematical and PL foundations

- Eric Lehman, F. Thomson Leighton, and Albert R. Meyer, *Mathematics for Computer Science*. MIT OpenCourseWare. <https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-fall-2010/pages/readings/>
- Benjamin C. Pierce, *Types and Programming Languages*. MIT Press. <https://mitpress.mit.edu/9780262162098/types-and-programming-languages/>
- Robert Harper, *Practical Foundations for Programming Languages*. <https://www.cs.cmu.edu/~rwh/pfpl.html>

D.2 Effects

- Philip Wadler, *Monads for Functional Programming*. <https://homepages.inf.ed.ac.uk/wadler/papers/marktoberdorf/baastad.pdf>
- Matija Pretnar, *An Introduction to Algebraic Effects and Handlers*. <https://www.eff-lang.org/handlers-tutorial.pdf>
- Andrej Bauer and Matija Pretnar, *Programming with Algebraic Effects and Handlers*. <https://arxiv.org/abs/1203.1539>
- Eugenio Moggi, *Notions of Computation and Monads*. See the bibliography of the source paper for the journal citation.
- Gordon Plotkin and John Power, *Notions of Computation Determine Monads*. https://homepages.inf.ed.ac.uk/gdp/publications/Comp_Eff_Monads.pdf

D.3 Coeffects

- Tomas Petricek, Dominic Orchard, and Alan Mycroft, *Coeffects: Unified Static Analysis of Context-Dependence*. <https://tomaspetricek.net/academic/papers/coeffects/>
- Tomas Petricek, Dominic Orchard, and Alan Mycroft, *Coeffects: A Calculus of Context-Dependent Computation*. See the source paper bibliography for the formal publication.

D.4 Category theory and compositionality

- Bartosz Milewski, *Category Theory for Programmers*. Unofficial PDF/source prepared with permission. <https://github.com/hmemcpy/milewski-ctfp-pdf>
- *Category Theory Illustrated*. <https://abuseofnotation.github.io/category-theory-illustrated/>
- Brendan Fong and David I. Spivak, *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. <https://arxiv.org/abs/1803.05316>

D.5 Reversibility, traces, and concurrency

- Chris Heunen, Robin Kaarsgaard, and Martti Karvonen, *Reversible Effects as Inverse Arrows*. <https://arxiv.org/abs/1805.08605>
- Antoni Mazurkiewicz, *Trace Theory*. See the source paper bibliography for the LNCS publication.
- Volker Diekert and Anca Muscholl, *Trace Theory* tutorial material. <https://www2.informatik.uni-stuttgart.de/fmi/ti/veroeffentlichungen/pdf/DiekertMuscholl2011.pdf>

D.6 Applied comparisons

- Martin Fowler, *Inversion of Control Containers and the Dependency Injection Pattern*. <https://martinfowler.com/articles/injection.html>
- React, `useEffect` reference. <https://react.dev/reference/react/useEffect>
- React, *Lifecycle of Reactive Effects*. <https://react.dev/learn/lifecycle-of-reactive-effects>
- OSGi Alliance, Declarative Services Specification. <https://docs.osgi.org/specification/osgi.cmpn/8.0.0/service.component.html>
- Vite, HMR API. <https://vite.dev/guide/api-hmr>

D.7 Testing and tooling

- `fast-check`, property-based testing for JavaScript and TypeScript. <https://fast-check.dev/>
- Vitest documentation. <https://vitest.dev/guide/>
- TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>

D.8 How to read research papers in this course

Use four passes.

1. **Problem pass:** What failure or limitation motivates the section?
2. **Definition pass:** What are the types, domains, and quantified assumptions?
3. **Example pass:** Construct the smallest state that instantiates the definition.
4. **Theorem pass:** Translate the conclusion into an invariant or test, then identify why every hypothesis is needed.

For difficult notation, make a three-column note:

Formal symbol	Runtime representation	Executable check
$g(f(\gamma)) \simeq \gamma$	acquisition plus returned disposer	apply, dispose, compare observation
$\sigma \models d$	every required key resolves	satisfies(store, requires)
$\text{target}_n(\gamma)$	provider-ID map or bottom	compute target digest
$f \circ g \simeq g \circ f$	operations reorder safely	run both orders
$\neg \text{quiet}(\gamma)$	lifecycle mismatch exists	some enabled rule must exist

Closing Perspective

The central lesson of the labs is not that every side effect can be undone. It is that a useful class of dynamic systems becomes tractable when three design choices are made explicit.

First, context-changing operations are paired locally with recovery and accumulated structurally. Second, environmental requirements are declared and reactively resolved rather than hidden in optimistic lookup. Third, the runtime states the assumptions that let local guarantees survive interleaving: ownership, provider identity, observational equivalence, independence, dependency order, bounded progress, and failure boundaries.

The result is a different way to read plugin systems. A command registry is not only a map; its operations have commutation laws. A service lookup is not only dependency injection; it is a coeffect resolution with a lifecycle. A disposer is not only cleanup; it is a witnessed inverse whose validity depends on the state and on foreign interference. A scheduler is not only an implementation detail; its interleavings are traces whose equivalence determines confluence.

A student who completes the handbook should be able to move in both directions:

```
formal definition -> runtime mechanism -> executable property
```

and:

```
observed failure -> missing invariant -> strengthened definition or rule premise
```

That movement is the real purpose of the laboratory.